Bitácora de uso de inteligencia artificial

Proyecto: UberClone Driver
Tecnología: React Native, Expo y JavaScript
Repositorio: https://github.com/julianch07/uberclone-driver
Desarrollo: Trabajo individual

Entrada #1

- **Fecha:** 2026-11-05
- **Integrante:** Julian Cardona
- **Herramienta:** ChatGPT (OpenAI)
- **Objetivo:** Inicializar el proyecto UberClone Driver y configurar el entorno de desarrollo para comenzar su implementación.
- **Prompt:** No se conserva el texto literal de la solicitud inicial. La configuración se realizó con acompañamiento de IA.
- **Resumen de la respuesta:** Se proporcionó orientación para inicializar el proyecto con Expo, configurar el entorno de desarrollo y preparar el repositorio para trabajar con Git y GitHub.
- **Qué se usó / modificó / descartó:** Se utilizó la orientación para crear la estructura inicial del proyecto y configurar las herramientas necesarias para su ejecución.
- **Archivos y commit:** Archivos base del proyecto Expo y configuración de Git — `b6ee68e`. Mensaje: `chore: inicializar proyecto Expo y configuración base`.
- **Verificación:** Se ejecutó Expo Doctor y se obtuvieron 20 de 20 comprobaciones satisfactorias. Se verificó la autenticación de GitHub CLI, se realizó el primer commit y se publicó el repositorio. También se cambió la rama inicial `master` por `main`.
- **Aprendizaje:** Se comprendió el proceso de inicialización de un proyecto con Expo y la importancia de configurar correctamente Git y GitHub desde el comienzo.

Entrada #2

- **Fecha:** 2026-11-05
- **Integrante:** Julian Cardona
- **Herramienta:** ChatGPT (OpenAI)
- **Objetivo:** Organizar la arquitectura del proyecto mediante una estructura modular que facilite el desarrollo y mantenimiento de la aplicación.
- **Prompt:** No se conserva el texto literal de la solicitud original. Se solicitó orientación para organizar las carpetas y separar las responsabilidades del proyecto.
- **Resumen de la respuesta:** Se propuso una estructura modular para organizar los componentes, pantallas, servicios, configuración y documentación.
- **Qué se usó / modificó / descartó:** Se utilizó la estructura propuesta para organizar las carpetas del proyecto y se creó la bitácora para registrar el uso de inteligencia artificial durante el desarrollo.

- **Archivos y commit:** Carpetas de `src/` y `docs/`, incluyendo `docs/AI_LOG.md` — `49f1cf2`. Mensaje: `chore: establecer arquitectura modular y bitácora de IA`.
- **Verificación:** Se verificó la creación de las carpetas mediante el comando `find src docs -type d`.
- **Aprendizaje:** Se comprendió la importancia de separar las responsabilidades de una aplicación y mantener una estructura organizada para facilitar futuras modificaciones.

Entrada #3

- **Fecha:** 2026-11-05
- **Integrante:** Julian Cardona
- **Herramienta:** ChatGPT (OpenAI)
- **Objetivo:** Crear la estructura inicial del asistente de registro del conductor, organizando el proceso en diferentes pasos.
- **Prompt:** No se conserva el texto literal de la solicitud original. Se solicitó ayuda para implementar el flujo inicial de registro.
- **Resumen de la respuesta:** Se proporcionó una implementación inicial de un asistente de registro con dos pasos, un indicador de progreso y navegación entre las pantallas correspondientes.
- **Qué se usó / modificó / descartó:** Se utilizó la implementación como base visual y funcional para representar el proceso de registro del conductor. Los formularios todavía corresponden a una estructura de demostración.
- **Archivos y commit:** `App.js` y archivos relacionados con las pantallas y componentes del registro — `8d0b4ee`. Mensaje: `feat: crear asistente de registro del conductor`.
- **Verificación:** Se ejecutó el proyecto mediante Expo Go y se comprobó visualmente la pantalla de registro y la navegación entre los pasos.
- **Aprendizaje:** Se comprendió cómo dividir una funcionalidad en componentes y pantallas reutilizables para organizar mejor la interfaz de usuario.

Entrada #4

- **Fecha:** 2026-11-05
- **Integrante:** Julian Cardona
- **Herramienta:** ChatGPT (OpenAI)
- **Objetivo:** Crear la documentación README para explicar el propósito del proyecto, las tecnologías utilizadas y las instrucciones básicas de instalación y ejecución.
- **Prompt:** "que hago en readme"
- **Resumen de la respuesta:** Se explicó la importancia del archivo README y se propuso una estructura para documentar el proyecto, sus tecnologías, instalación y ejecución.
- **Qué se usó / modificó / descartó:** Se utilizó la orientación para crear el README del repositorio y describir la información principal del proyecto.
- **Archivos y commit:** `README.md` — `7e71e5f`. Mensaje: `docs: documentar el proyecto y registrar uso de IA`.
- **Verificación:** Se revisó el contenido del README y se verificó su inclusión en el repositorio.
- **Aprendizaje:** Se comprendió la importancia de documentar un proyecto para facilitar su instalación, comprensión y presentación a otros desarrolladores.

Entrada #5

- **Fecha:** 2026-11-06
- **Integrante:** Julian Cardona
- **Herramienta:** ChatGPT (OpenAI)
- **Objetivo:** Configurar Firebase en UberClone Driver e integrar los servicios iniciales de autenticación anónima y acceso a Firestore, utilizando variables de entorno para proteger la configuración.
- **Prompt:** No existe una única solicitud literal que represente todo el proceso. Se realizaron varias consultas para configurar Firebase, instalar sus dependencias, preparar la persistencia de autenticación y organizar los servicios de conexión.
- **Resumen de la respuesta:** Se proporcionaron instrucciones para crear un proyecto en Firebase, registrar una aplicación web, instalar Firebase SDK y AsyncStorage, configurar las variables de entorno e implementar la inicialización de Firebase, la autenticación y las operaciones básicas de Firestore.
- **Qué se usó / modificó / descartó:** Se utilizó la configuración propuesta para crear el proyecto Firebase `uberclone-driver-8e0bf`, registrar la aplicación web, crear la base de datos Firestore en la región `southamerica-east1`, instalar Firebase SDK versión `13.2.0` y AsyncStorage, configurar las variables de entorno y habilitar la autenticación anónima. También se implementaron los servicios de autenticación y Firestore, y se integraron las pruebas iniciales en `App.js`.
- **Archivos y commit:** `package.json`, `package-lock.json`, `.env.example`, `App.js`, `src/config/firebase.js`, `src/services/authService.js` y `src/services/firestoreService.js` — `8c2720d`. Mensaje: `feat: integrar Firebase y autenticación del conductor`. El archivo `.env` se configuró localmente y permanece excluido del repositorio. *Corrección (auditoría, entrada #26):* `.gitignore` no se modificó en este commit; la exclusión de `.env` ya existía desde `b6ee68e`.
- **Verificación:** Se verificó la instalación de Firebase mediante `npm ls firebase`. Se habilitó el proveedor de autenticación anónima desde Firebase Console. Se inició Expo con `npx expo start --clear` y se comprobó desde Expo Go que la aplicación autenticó al conductor anónimamente. En la consola se obtuvo su UID y el mensaje `Conexión con Firestore verificada correctamente.`. También se comprobó la escritura, lectura y eliminación del campo de prueba en Firestore. Se verificó que `.env` permanece excluido de Git.
- **Aprendizaje:** Se comprendió cómo inicializar Firebase en React Native, mantener la sesión de autenticación mediante AsyncStorage, separar los servicios de autenticación y Firestore, proteger las variables de entorno y comprobar las operaciones básicas de una base de datos desde la aplicación.

Entrada #6

- **Fecha:** 2026-11-06
- **Integrante:** Julian Cardona
- **Herramienta:** ChatGPT (OpenAI)
- **Objetivo:** Implementar los formularios de datos personales y del vehículo del conductor, incorporando campos, selectores, selección de fotografías y validaciones para controlar el avance del proceso de registro.
- **Prompt:** No existe una única solicitud literal que represente todo el proceso. Se realizaron varias consultas para desarrollar los componentes de registro, implementar las reglas de validación y conectar estas funcionalidades con la navegación entre los pasos.
- **Resumen de la respuesta:** Se proporcionó orientación y código para implementar los componentes reutilizables de los formularios, organizar los datos mediante estados de React, seleccionar fotografías desde la cámara o galería y validar la información ingresada antes de permitir avanzar al siguiente paso.

- **Qué se usó / modificó / descartó:** Se utilizaron las implementaciones propuestas para desarrollar los componentes `FormField`, `PersonalDataForm` y `VehicleDataForm`, así como las funciones de validación en `registrationValidation.js`. Se actualizaron las funciones de navegación y manejo de información en `RegistrationScreen.js`. La tercera pestaña de documentos y pagos se conserva como sección pendiente, de acuerdo con el alcance establecido para esta entrega.
- **Archivos y commit:** `src/screens/registration/RegistrationScreen.js`, `src/components/registration/FormField.js`, `src/components/registration/PersonalDataForm.js`, `src/components/registration/VehicleDataForm.js` y `src/utils/registrationValidation.js`. *Corrección (auditoría, entrada #26):* el commit también modificó `app.json` (plugin `@react-native-community/datetimepicker`), `package.json` y `package-lock.json` (dependencias `@react-native-community/datetimepicker` 9.2.1, `@react-native-picker/picker` 2.11.4 y `expo-image-picker` ~58.0.10). **Commit:** `0e37d23` (`feat: implementar formularios y validaciones del conductor`).
- **Verificación:** Se ejecutó la aplicación mediante Expo Go y se comprobó la navegación entre los pasos. Se probaron las validaciones de datos personales y del vehículo, incluyendo campos obligatorios, formato de placa, año del vehículo y capacidad mínima para la categoría XL. Se comprobó que los mensajes de error aparezcan debajo de los campos y que la aplicación impida avanzar cuando la información no cumple las reglas establecidas.
- **Aprendizaje:** Se comprendió cómo separar los formularios en componentes reutilizables, manejar información mediante `useState`, validar datos antes de avanzar en un asistente de registro y proporcionar mensajes de error para orientar al usuario.

Entrada #7

- **Fecha:** 2026-11-06
- **Integrante:** Julian Cardona
- **Herramienta:** ChatGPT (OpenAI)
- **Objetivo:** Integrar la subida de fotografías del conductor a Cloudinary y la persistencia de los datos personales y del vehículo en Firestore, permitiendo recuperar el registro al volver a abrir la aplicación.
- **Prompt:** No existe una única solicitud literal que represente todo el proceso. Se realizaron consultas para configurar Cloudinary, integrar la autenticación con Firebase, implementar la persistencia del registro y solucionar la incompatibilidad de FormData durante la carga de imágenes en React Native.
- **Resumen de la respuesta:** Se proporcionó orientación y código para implementar el servicio de subida de imágenes a Cloudinary, guardar y recuperar los datos mediante Firestore e integrar estas operaciones en el asistente de registro. Se adaptó el envío de fotografías utilizando la clase File de expo-file-system para resolver el error de compatibilidad de FormData.
- **Qué se usó / modificó / descartó:** Se integraron la autenticación anónima del conductor, la recuperación de información existente, la subida de fotografías a Cloudinary y el almacenamiento de las URL junto con los datos personales y del vehículo en Firestore. Se mantuvo la tercera pestaña de documentos y pagos como sección pendiente, de acuerdo con el alcance de esta entrega. La configuración local de Cloudinary permanece en .env y no se incluye en el repositorio.
- **Archivos y commit:** .env.example, App.js, src/screens/registration/RegistrationScreen.js, src/services/firestoreService.js y src/services/cloudinaryService.js — `d5efb91`. Mensaje: `feat: guardar registro y fotos del conductor`.
- **Verificación:** Se ejecutó npx expo export --platform ios y la compilación final terminó correctamente. Desde Expo Go se comprobó la autenticación anónima, el guardado exitoso del registro y la recuperación de la información y las fotografías después de cerrar y volver a abrir la aplicación.

- **Aprendizaje:** Se comprendió cómo integrar Cloudinary y Firestore en una aplicación React Native, almacenar URL de imágenes en lugar de URI locales, recuperar registros asociados al UID del usuario autenticado y utilizar File de expo-file-system para enviar archivos mediante FormData.

Entrada #8

- **Fecha:** 2026-11-06
- **Integrante:** Julian Cardona
- **Herramienta:** ChatGPT (OpenAI)
- **Objetivo:** Corregir el selector de fecha de nacimiento y aclarar visualmente que la tercera sección del registro está pendiente.
- **Prompt (resumen; no consta como texto literal):** Solicitud de revisión y corrección del formulario de datos personales y del indicador de pasos para la primera entrega.
- **Resumen de la respuesta:** Se identificó el uso incorrecto de `onValueChange` en `DateTimePicker` y se indicó reemplazarlo por `onChange`. También se ajustó el texto de la tercera sección para mostrar explícitamente su estado pendiente.
- **Qué se usó / modificó / descartó:** Se cambió el callback del selector de fecha y se modificó la etiqueta de la tercera sección, sin implementar documentos ni pagos porque están fuera del alcance de esta entrega.
- **Archivos y commit:** `src/components/registration/PersonalDataForm.js` y `src/screens/registration/RegistrationScreen.js` — `cb9ac34`. Mensaje: `fix: corregir formulario y pasos del registro`.
- **Verificación:** Cambios revisados mediante `git diff` y `git diff --check`. La prueba funcional y la compilación quedan pendientes.
- **Aprendizaje:** Se reforzó la importancia de utilizar las propiedades correctas de los componentes y comunicar claramente las funcionalidades pendientes.

Entrada #9

- **Fecha:** 2026-11-11
- **Integrante:** Julian Cardona
- **Herramienta:** ChatGPT (OpenAI)
- **Objetivo:** Implementar la tercera sección del registro del conductor, correspondiente a documentos y pagos, e integrar su validación, almacenamiento y navegación con las secciones anteriores.
- **Prompt:** No existe una única solicitud literal que represente todo el proceso. Se realizaron consultas para desarrollar el formulario de documentos y pagos, integrar sus datos con Firestore y Cloudinary, implementar las validaciones y permitir la navegación entre las tres secciones.
- **Resumen de la respuesta:** Se proporcionó orientación y código para crear el componente `DocumentsPaymentsForm`, validar los datos y documentos requeridos, incorporar la aceptación de términos y condiciones, guardar la información del registro y permitir navegar entre las secciones.
- **Qué se usó / modificó / descartó:** Se implementó el formulario de documentos y pagos, incluyendo número y fotografía de licencia de conducción, fechas de vencimiento, fotografías del SOAT y revisión técnico-mecánica, correo de Mercado Pago y aceptación de términos. Se actualizaron las validaciones, el servicio de Firestore y la pantalla principal del registro para integrar los tres pasos y guardar las fotografías en Cloudinary. La revisión técnico-mecánica se maneja según su aplicabilidad.

- **Archivos:** `src/components/registration/DocumentsPaymentsForm.js`, `src/screens/registration/RegistrationScreen.js`, `src/services/firestoreService.js`, `src/utils/registrationValidation.js`, `app.json`, `package.json` y `package-lock.json`.
- **Commit:** `be8ded2` (`feat: implementar sección de documentos y pagos`).
- **Verificación:** Se ejecutó `npx expo export --platform ios` y la exportación terminó correctamente. Desde Expo Go se comprobó la visualización y navegación entre las tres secciones, las validaciones de campos obligatorios, el guardado exitoso del registro y la recuperación de los datos después de cerrar y volver a abrir la aplicación.
- **Aprendizaje:** Se comprendió cómo ampliar un asistente de registro con nuevos componentes, centralizar las validaciones por sección, integrar fotografías y datos adicionales en los servicios de persistencia y controlar la navegación sin perder la información guardada.

Entrada #10

- **Fecha:** 2026-11-12
- **Integrante:** Julian Cardona
- **Herramienta:** Claude (Anthropic). La auditoría y el plan de módulos se prepararon previamente con ChatGPT y la planificación se revisó con Claude.
- **Objetivo:** Implementar el módulo 1 de la entrega final: estado de aprobación del conductor, de modo que todo registro quede `pending` y solo pueda pasar a `approved` desde Firebase Console.
- **Prompt:** "Comencemos por el módulo 1: estado de aprobación del conductor." Las decisiones previas del propietario (estado inicial `pending`, cambio a `approved` desde Firebase Console, el conductor no puede modificar su propio estado y protección mediante reglas de Firestore) se indicaron en la misma conversación.
- **Resumen de la respuesta:** Se realizó una auditoría del proyecto sin modificar archivos, se acordó el plan y se implementó el módulo 1: constantes y funciones del estado de aprobación, escritura de `approvalStatus: 'pending'` solo cuando el documento no lo tiene, reglas de Firestore que impiden al conductor cambiarlo y un aviso con el estado en la pantalla de registro.
- **Qué se usó / modificó / descartó:** Se creó `src/config/driverStatus.js` (estados `pending`, `approved` y `rejected`, textos en español, `getApprovalStatus` y `canDriverGoOnline`). Se modificó `saveDriverRegistration` para que escriba `approvalStatus` y `approvalRequestedAt` únicamente si el documento aún no tiene estado; así guardar nuevamente no reinicia una aprobación. Se agregó a `RegistrationScreen.js` un aviso con el estado actual y un botón "Actualizar estado". Se creó `firestore.rules`. Un registro sin campo de estado se trata como `pending`. Se descartó migrar a Expo Router y no se agregaron dependencias. Las reglas deben publicarse manualmente en Firebase Console y reemplazan las reglas existentes. Aún no se bloquea la conexión del conductor, porque la pantalla de inicio se implementa en módulos posteriores; este módulo solo deja disponible `canDriverGoOnline`.
- **Archivos y commit:** `src/config/driverStatus.js`, `firestore.rules`, `src/services/firestoreService.js`, `src/screens/registration/RegistrationScreen.js` y `docs/AI_LOG.md`. Commit: `f749800` (`feat: implementar estado de aprobación del conductor`).
- **Verificación:** Se ejecutó una prueba de las funciones de `driverStatus.js` con Node (9 aserciones, todas correctas). En una copia del proyecto se ejecutó `npm ci` y `npx expo export --platform ios`, que terminó correctamente. No se ejecutaron `npx expo-doctor`, la aplicación en Expo Go, las lecturas y escrituras reales en Firestore ni las reglas de Firestore, porque el entorno no dispone de esos recursos. Esas comprobaciones quedan pendientes para el propietario.
- **Aprendizaje:** Se comprendió que un campo de autorización no puede protegerse solo desde la aplicación y que debe validarse en las reglas de Firestore, y que conviene escribir el estado inicial una única vez para no reiniciar

aprobaciones al editar el registro.

Entrada #11

- **Fecha:** 2026-11-12
- **Integrante:** Julian Cardona
- **Herramienta:** Claude (Anthropic)
- **Objetivo:** Permitir que el conductor navegue directamente entre las tres secciones del registro sin perder información, como solicitó el profesor para la entrega final.
- **Prompt:** "Tarea: implementar navegación entre las secciones del registro de UberClone Driver", con los requisitos funcionales, las restricciones técnicas y las verificaciones indicados en la solicitud.
- **Resumen de la respuesta:** Se revisó la arquitectura existente. Los indicadores de sección ya eran pulsables y los datos viven en el estado de `RegistrationScreen`, por lo que se conservaron. Se movió la regla de navegación a una función pura, se agregó el estado "Completa" o "Incompleta" a cada sección y se muestra un mensaje cuando se redirige a una sección incompleta.
- **Qué se usó / modificó / descartó:** Se creó `src/utils/registrationNavigation.js` con `getSectionCompletion` y `resolveStepNavigation`. Reglas: ir a la misma sección o a una anterior siempre es posible y no borra datos; ir a una sección posterior exige que las anteriores estén completas, y si no, se redirige a la primera incompleta con sus errores y un mensaje; la sección 3 incompleta puede abrirse, pero no se puede guardar el registro hasta cumplir sus validaciones. En `RegistrationScreen.js` se reemplazó la lógica de `handleStepPress` y se actualizó el indicador de pasos (verde y ✓ cuando está completa, azul la actual, etiqueta de estado y atributos de accesibilidad). No se modificaron las validaciones, la carga y el guardado en Firestore, la subida a Cloudinary, el estado de aprobación ni las reglas de Firestore, y no se agregaron dependencias. No se agregó guardado automático al cambiar de sección: los datos se conservan en pantalla y se persisten al pulsar "Guardar registro".
- **Archivos y commit:** `src/utils/registrationNavigation.js`, `src/screens/registration/RegistrationScreen.js` y `docs/AI_LOG.md`. Commit: `5748009` (`feat: mejorar navegacion entre secciones del registro`). *Corrección (auditoría, entrada #26):* el commit también actualizó `expo` de `~58.0.24` a `~58.0.25` en `package.json` y `package-lock.json`; no se agregaron dependencias nuevas.
- **Verificación:** Se ejecutó con Node una prueba de las funciones de `registrationNavigation.js` contra los validadores reales (completitud, navegación libre con todo completo, retroceso sin validar, redirección a la primera sección incompleta y entrada a la sección 3 incompleta); todas las aserciones pasaron. Se ejecutó `git diff --check` sin observaciones y `npx expo export --platform ios` terminó correctamente. `npx expo-doctor` pasó 18 de 20 comprobaciones; las 2 restantes no pudieron ejecutarse por falta de acceso a la red del entorno (API de Expo). No se probó en Expo Go ni contra Firebase o Cloudinary reales, y no se realizó una prueba automatizada de la interfaz. Esas comprobaciones quedan pendientes para el propietario.
- **Aprendizaje:** Se comprendió que mantener los datos en el componente padre evita perder información al cambiar de sección, y que separar la regla de navegación en una función pura permite probarla sin ejecutar la aplicación.

Entrada #12

- **Fecha:** 2026-11-12
- **Integrante:** Julian Cardona
- **Herramienta:** ChatGPT (OpenAI)
- **Objetivo:** Crear el inicio del conductor, navegación interna y control de disponibilidad basado en aprobación.

- **Prompt (resumen; no consta como texto literal):** Implementar módulo 3 desde el ZIP actualizado, con Home, viajes y ganancias provisionales, perfil, acceso a edición del registro y control seguro de conexión.
- **Resumen de la respuesta:** Se implementó una pantalla de inicio con cuatro pestañas, integración con el registro persistido, lectura del estado de aprobación y disponibilidad guardada en Firestore mediante transacción. Se añadieron reglas de seguridad para impedir activar la disponibilidad sin aprobación. En una revisión posterior se separó el acceso al Home de la vigencia documental, se agregó un marcador de guardado exitoso y una confirmación antes de descartar cambios.
- **Qué se usó / modificó / descartó:** Se reutilizó el flujo de autenticación y los validadores; no se incorporó Expo Router ni dependencias adicionales. Las secciones de viajes y ganancias quedan como pantallas provisionales.
- **Archivos:** `App.js`, `src/screens/home/DriverHomeScreen.js`, `src/screens/registration/RegistrationScreen.js`, `src/services/availabilityService.js`, `src/services/firestoreService.js`, `src/utils/driverRegistration.js`, `firestore.rules`, `firebase.json`, `tests/firestore.rules.test.cjs`, `package.json`, `package-lock.json`, `.gitignore` y `docs/AI_LOG.md`.
- **Commit:** `bd26a6b` (`feat: implementar home y disponibilidad del conductor`). Integrado en `main` mediante el Pull Request #9, merge `49ab7ee`.
- **Verificación:** Se probó el módulo Home en Expo Go y se ejecutó `npx expo export --platform ios` correctamente. Se ejecutaron 12 pruebas automatizadas de reglas de Firestore en el emulador, con 12 resultados aprobados y 0 fallidos. Sin embargo, varios rechazos produjeron `evaluation error`, por lo que las validaciones negativas requieren revisión adicional. `git diff --check` terminó sin observaciones. Las reglas todavía no se han desplegado en Firebase Console y no se ha verificado su comportamiento en producción.
- **Aprendizaje:** La autorización para conectarse debe comprobarse en Firestore y no únicamente en la interfaz del dispositivo.

Entrada #13

- **Fecha:** 2026-11-12
- **Integrante:** Julian Cardona
- **Herramienta:** Claude (Anthropic). El prompt del módulo se preparó con ChatGPT.
- **Objetivo:** Mostrar la ubicación GPS real del conductor en un mapa de Google Maps en la pestaña Inicio y publicarla en Firestore solo mientras está conectado y aprobado.
- **Prompt:** "IMPLEMENTACIÓN DEL MÓDULO 4 — GPS Y GOOGLE MAPS", sobre el ZIP actualizado del repositorio, con arquitectura, reglas de Firestore, configuración de Google Maps, pruebas y verificaciones indicadas en la solicitud.
- **Resumen de la respuesta:** Se implementaron los servicios de ubicación y de publicación, un hook de seguimiento, el componente del mapa y el de permisos, la integración en `DriverHomeScreen`, las reglas de `driverLocations/{uid}`, la configuración nativa mediante `app.config.js` y pruebas de lógica y de reglas.
- **Qué se usó / modificó / descartó:** Se agregaron `expo-location` (~58.0.22) y `react-native-maps` (1.29.11). Los números de versión se tomaron de `node_modules/expo/bundledNativeModules.json` porque `npx expo install` no pudo consultar la API de Expo desde el entorno de trabajo. `locationService.js` consulta permisos y GPS, obtiene la posición y devuelve una función `stop()` que elimina la suscripción incluso si se llama antes de que `watchPositionAsync` termine de resolverse. `driverLocationService.js` escribe el documento completo (latitud, longitud, precisión y fecha del servidor) y decide cuándo publicar: mínimo 10 s entre escrituras, 15 m de desplazamiento o renovación cada 60 s; no publica antes de `start()` ni después de `stop()`, y `clear()` espera la escritura en curso antes de eliminar el documento. En `DriverHomeScreen` el seguimiento está activo en la pestaña Inicio o mientras el conductor está conectado; la publicación se detiene antes de pedir la desconexión y se elimina la ubicación una vez confirmada. El seguimiento se detiene al pasar la app a segundo plano.

`DriverMap` no dibuja el mapa hasta recibir una posición real y `LocationPermission` cubre los casos sin solicitar, rechazado, bloqueado y GPS desactivado. Las reglas exigen propietario, `drivers/{uid}` existente, aprobado y con `isOnline == true`, estructura exacta de cuatro campos, rangos válidos y `updatedAt == request.time`; el propietario puede leer y borrar su documento. Se mantuvieron las reglas de registro y disponibilidad. Android usa Google Maps; en iOS se usa Google Maps solo con `EXPO_PUBLIC_USE_GOOGLE_MAPS_IOS=true` en un development build con clave. No se implementó ubicación en segundo plano ni se incluyó ninguna clave real.

- **Archivos:** Creados: `app.config.js`, `src/components/map/DriverMap.js`, `src/components/map/LocationPermission.js`, `src/hooks/useDriverLocation.js`, `src/services/locationService.js`, `src/services/driverLocationService.js`, `src/utils/locationLogic.js`, `tests/location.test.cjs` y `tests/helpers/loadModule.cjs`. Modificados: `src/screens/home/DriverHomeScreen.js`, `firestore.rules`, `tests/firestore.rules.test.cjs`, `package.json`, `package-lock.json`, `.env.example`, `README.md` y `docs/AI_LOG.md`.
- **Commit:** `b67c3fa` (`feat: integrar ubicacion GPS y mapa del conductor`). Integrado en `main` mediante el Pull Request #11, commit de fusión `606bd56`.
- **Verificación:** Se ejecutaron 20 pruebas de lógica con `node --test tests/location.test.cjs` (permisos concedidos, denegados y bloqueados, coordenadas, errores de GPS, `stop()` antes de que se cree la suscripción, publicador con `start`, `stop` y `clear`, frecuencia, escrituras simultáneas, carrera con la desconexión y errores de conexión); las 20 pasaron. Al desactivar temporalmente dos protecciones del código, las pruebas correspondientes fallaron y luego se restauró el código. Se ejecutó `node --check` sobre los archivos de lógica y de pruebas, `git diff --check` sin observaciones y `npx expo export --platform ios` terminó correctamente. Con una clave falsa y una copia desechable se ejecutó `npx expo prebuild --no-install` y se comprobó que la clave llega a `Info.plist` y a `AndroidManifest.xml`, que solo se declara el permiso de ubicación en primer plano y que no se agregan permisos de ubicación en segundo plano; esa copia se eliminó. `npx expo-doctor` pasó 18 de 20 comprobaciones; las 2 restantes no pudieron ejecutarse por falta de acceso a la red (API de Expo y React Native Directory). Las 22 pruebas nuevas de reglas de Firestore y las 12 existentes no se ejecutaron en el entorno de Claude: el emulador de Firestore no se pudo descargar (respuesta 403 del proxy). Tampoco se ejecutó la aplicación en Expo Go ni en un development build, ni se probó el GPS, el mapa, Firestore real o la clave de Google Maps.
- **Revisión posterior (ChatGPT):** Se corrigió la descripción de alcance en README, se documentó el riesgo de ubicaciones obsoletas tras cierres inesperados y se añadió una advertencia visible cuando la desconexión funciona pero falla la eliminación de la ubicación. Estas correcciones no sustituyen las pruebas pendientes en dispositivo.
- **Verificaciones posteriores (ChatGPT, 2026-11-13):** El propietario ejecutó las pruebas de reglas de seguridad mediante `npx firebase-tools emulators:exec --project demo-uberclone-driver --only firestore "node --test tests/firestore.rules.test.cjs"`. Se obtuvieron **34 pruebas aprobadas y 0 fallidas**, incluyendo los 12 casos existentes y los 22 nuevos del módulo GPS. Inicialmente, 18 pruebas fallaron porque la función `assertDeniedByRule` rechazaba los mensajes que contenían `evaluation error`, aunque Firestore devolvía `permission-denied`. Se ajustó el verificador para comprobar la denegación sin exigir que el mensaje estuviera libre de errores de evaluación. Las pruebas pasaron después de ese cambio, pero los errores de evaluación siguen apareciendo en algunos rechazos y quedan pendientes de investigación. También se ejecutaron `npx expo-doctor` (**20/20 verificaciones aprobadas**), `npx expo export --platform ios` (exportación correcta) y `git diff --check` (sin observaciones). Posteriormente, el propietario realizó pruebas en un iPhone físico mediante Expo Go: se visualizó el mapa con la ubicación GPS real, se conectó un conductor aprobado y se confirmó la publicación de sus coordenadas en `driverLocations/{uid}` de Firestore de producción. Al desconectarse, se verificó que el documento de ubicación desapareció. En iOS con Expo Go se utiliza Apple Maps; la integración nativa de Google Maps mediante development build sigue pendiente de comprobar. Durante las pruebas se detectó que un registro antiguo no tenía `registrationCompletedAt`, lo que impedía conectarse bajo las reglas

actuales; se añadió manualmente ese campo al registro de prueba. Queda pendiente definir una solución para otros registros antiguos. También quedan por investigar algunos mensajes de evaluación del emulador y una advertencia de transporte de Firestore.
- **Aprendizaje:** Separar la lógica de ubicación en funciones puras y servicios inyectables permite probar permisos, errores y limpieza de suscripciones sin un dispositivo, y las reglas de Firestore deben comprobar tanto la estructura de los datos como el estado actual del conductor.

Entrada #14

- **Fecha:** 2026-11-13 (fecha declarada; el commit `a9c320f` tiene fecha 2026-11-12 22:16, UTC–05:00)
- **Integrante:** Julian Cardona
- **Herramienta:** ChatGPT (OpenAI)
- **Objetivo:** Corregir la incompatibilidad de registros antiguos de conductores aprobados que no contienen `registrationCompletedAt`, sin alterar su aprobación ni debilitar las reglas de seguridad de Firestore.
- **Prompt (resumen; no consta como texto literal):** Solucionar el error `Missing or insufficient permissions` que impedía conectar a conductores antiguos aprobados, evitando modificar manualmente sus documentos en Firebase Console y manteniendo las restricciones de seguridad.
- **Resumen de la respuesta:** Se implementó una regularización automática del campo `registrationCompletedAt` para registros antiguos aprobados y completos. La aplicación realiza una transacción para agregar únicamente el timestamp faltante y posteriormente otra transacción para actualizar la disponibilidad, comprobando nuevamente las condiciones de conexión.
- **Qué se usó / modificó / descartó:** Se agregó `validLegacyCompletionMigration()` a `firestore.rules`, permitiendo exclusivamente al propietario de un registro antiguo, aprobado, completo y desconectado agregar `registrationCompletedAt` con la fecha del servidor. Se modificó `src/services/availabilityService.js` para efectuar la regularización antes de conectarse. Se ampliaron las pruebas de seguridad y se creó `tests/availability.test.cjs` con pruebas del servicio mediante dependencias simuladas. Se descartó volver a guardar los formularios completos o modificar manualmente los documentos de producción como solución general. No se agregaron dependencias.
- **Archivos:** `firestore.rules`, `src/services/availabilityService.js`, `tests/firestore.rules.test.cjs`, `tests/availability.test.cjs` y `docs/AI_LOG.md`.
- **Commit:** `a9c320f` (`fix: compatibilizar registros antiguos aprobados`), rama `fix/compatibilidad-registros-antiguos`. Integrado en `main` mediante el Pull Request #13, merge `7d29445`.
- **Verificación:** Se ejecutaron 40 pruebas de reglas de Firestore en el emulador, todas aprobadas; 20 pruebas de lógica GPS y ubicación, todas aprobadas; y 6 pruebas nuevas del servicio de disponibilidad, todas aprobadas. `npx expo-doctor` superó 20 de 20 comprobaciones. `npx expo export --platform ios` generó correctamente el paquete JavaScript de iOS. `git diff --check` terminó sin observaciones. Las pruebas del servicio utilizan Firebase simulado y no sustituyen una prueba real de integración. Las reglas nuevas aún no se han desplegado en Firebase de producción y la regularización automática todavía no se ha verificado en un iPhone físico. Algunos mensajes `evaluation error` del emulador siguen pendientes de investigación.
- **Aprendizaje:** Los cambios de esquema deben considerar registros existentes. Una migración controlada puede preservar la aprobación y los datos del conductor, mientras las reglas limitan los campos modificables y verifican las condiciones de autorización.

Entrada #15

- **Fecha:** 2026-11-13

- **Integrante:** Julian Cardona
- **Herramienta:** ChatGPT (OpenAI)
- **Objetivo:** Implementar el módulo de solicitudes y aceptación de viajes para conductores aprobados y conectados, incorporando lógica de validación, transacciones de Firestore y reglas de seguridad.
- **Prompt (resumen; no consta como texto literal):** Implementar el módulo de viajes de UberClone Driver, integrando solicitudes pendientes en tiempo real, aceptación segura por parte del conductor, validaciones de los datos del viaje y pruebas automatizadas.
- **Resumen de la respuesta:** Se implementó el componente de solicitudes de viajes y se integró en la pestaña Viajes de la pantalla principal. Se desarrollaron servicios para consultar solicitudes pendientes y aceptar viajes mediante transacciones de Firestore. También se incorporaron funciones puras para validar viajes y sus estados, reglas de seguridad específicas y pruebas unitarias, de integración y de autorización.
- **Qué se usó / modificó / descartó:** Se creó `src/components/trips/TripRequests.js`, que presenta solicitudes pendientes, origen, destino, tarifa estimada y permite aceptar viajes. Se crearon `src/services/tripService.js` para escuchar solicitudes en tiempo real y `src/services/tripAcceptance.js` para aceptar viajes mediante transacciones. Se creó `src/utils/tripLogic.js` con los estados `pending`, `accepted`, `in_progress`, `completed` y `cancelled`, además de funciones de validación y transición. Se ampliaron las reglas de `firestore.rules` para validar ubicaciones, tarifa, estado del viaje, identidad del conductor, aprobación, disponibilidad y vigencia documental. Se integró `TripRequests` en `DriverHomeScreen.js`. Se agregaron pruebas para registro, lógica de viajes, aceptación y reglas de seguridad. Se actualizaron los scripts de pruebas de `package.json` y se retiró `firebase-admin` de las dependencias de desarrollo. No se implementaron todavía el mapa de cada solicitud, el temporizador de aceptación ni el ciclo completo del viaje activo.
- **Archivos:** Modificados: `firestore.rules`, `package.json`, `src/screens/home/DriverHomeScreen.js` y `docs/AI_LOG.md`. *Corrección (auditoría, entrada #26):* el commit `a090c85` no modifica `package-lock.json` y su cambio en `package.json` se limita a los scripts de pruebas; `firebase-admin` no aparece en ningún commit del repositorio, por lo que su retirada no es verificable en el historial. Creados: `src/components/trips/TripRequests.js`, `src/services/tripAcceptance.js`, `src/services/tripService.js`, `src/utils/tripLogic.js`, `tests/driverRegistration.test.cjs`, `tests/tripAcceptance.test.cjs`, `tests/tripLogic.test.cjs` y `tests/trips.rules.test.cjs`.
- **Commit:** `a090c85` (`feat: implementar solicitudes y aceptación de viajes`), rama `feat/modulo-viajes`. Integrado en `main` mediante el Pull Request #14, merge `561acc5`.
- **Verificación:** Se ejecutaron 47 pruebas unitarias, 1 prueba de integración de aceptación de viajes y 55 pruebas de reglas de seguridad de Firestore: 103 aprobadas y 0 fallidas. La prueba de integración utilizó Firestore Emulator. Se ejecutó `npx expo export --platform ios` y se generó correctamente el bundle de iOS con 729 módulos y un tamaño de 3,5 MB. `git diff --check` terminó sin observaciones. Durante algunas pruebas negativas de seguridad aparecieron mensajes `PERMISSION_DENIED` y errores de evaluación de reglas; las pruebas verificaron los rechazos esperados, pero esos mensajes de evaluación quedan pendientes de investigación. No se considera verificado el despliegue de las nuevas reglas en Firebase de producción.
- **Aprendizaje:** Las transacciones permiten evitar que dos conductores acepten simultáneamente una misma solicitud. La seguridad de los viajes debe protegerse en Firestore y no depender exclusivamente de las validaciones de la interfaz. La separación entre componentes, servicios y funciones puras facilita el mantenimiento y las pruebas del módulo.

Entrada #16

- **Fecha:** 2026-11-13
- **Integrante:** Julian Cardona

- **Herramienta:** ChatGPT (OpenAI)
- **Objetivo:** Implementar Context API para centralizar el estado del conductor y gestionar la recuperación de viajes activos.
- **Prompt (resumen; no consta como texto literal):** Solicité continuar el desarrollo de UberClone Driver mediante la implementación de Context API, integrando el estado de conexión del conductor, los viajes aceptados y su persistencia.
- **Resumen de la respuesta:** Se implementó DriverContext mediante React Context API, con estados para conexión, viaje activo, carga y errores. Se integró AsyncStorage para conservar el identificador del viaje aceptado y una suscripción a Firestore para sincronizar su información. Se conectó el contexto con App.js, DriverHomeScreen y TripRequests. También se incorporó una función para validar que un viaje activo pertenezca al conductor autenticado.
- **Qué se usó / modificó / descartó (hechos verificables en el commit `a439abc`):** Se utilizaron React Context API, AsyncStorage (`@react-native-async-storage/async-storage`, ya instalado desde la entrada #5) y suscripciones de Firestore. No se agregaron dependencias: el cambio en `package.json` se limita al script `test:unit`.
- **Archivos creados:** `src/context/DriverContext.js`, `src/utils/activeTripLogic.js` y `tests/activeTripLogic.test.cjs`.
- **Archivos modificados:** `App.js`, `src/screens/home/DriverHomeScreen.js`, `src/components/trips/TripRequests.js`, `src/services/tripService.js` y `package.json`.
- **Verificación:** 52 pruebas unitarias aprobadas, incluyendo cinco nuevas para validar viajes activos. También se ejecutaron previamente una prueba de integración y 55 pruebas de reglas de Firestore, todas satisfactorias. La exportación iOS se completó correctamente.
- **Limitaciones:** Queda pendiente verificar manualmente la recuperación del viaje después de reiniciar la aplicación y probar específicamente los fallos de AsyncStorage y de sincronización con Firestore.
- **Commit:** `a439abc` (`feat: implementar contexto global del conductor y persistencia de viajes`), rama `feat/context-api`. Integrado en `main` mediante el Pull Request #15, merge `ca45895`.
- **Aprendizaje:** Pendiente de completar por el integrante.

Entrada #17

- **Fecha:** 2026-11-13
- **Integrante:** Julian Cardona
- **Herramienta:** ChatGPT (OpenAI)
- **Objetivo:** Implementar el ciclo de vida de los viajes del conductor, permitiendo iniciar, finalizar y cancelar viajes aceptados de manera segura.
- **Prompt (resumen; no consta como texto literal):** Continuar el desarrollo de UberClone Driver implementando las transiciones de estado de los viajes, sus servicios, reglas de seguridad, pruebas automatizadas e integración con la interfaz.
- **Resumen de la respuesta:** Se implementaron operaciones transaccionales de Firestore para iniciar, finalizar y cancelar viajes. Se integraron estas operaciones en el servicio principal y se incorporaron botones en el componente de viajes, según el estado actual. Se ampliaron las reglas de seguridad para restringir las transiciones a los conductores asignados y proteger los campos modificables.
- **Qué se usó / modificó / descartó:** Se creó `src/services/tripLifecycle.js` con funciones para gestionar las transiciones mediante `runTransaction` y `serverTimestamp`. Se modificó `src/services/tripService.js` para exponer las operaciones. Se actualizó `src/components/trips/TripRequests.js` para mostrar los botones "Iniciar viaje", "Finalizar viaje" y "Cancelar viaje", bloquear pulsaciones mientras se procesa una operación y

presentar errores. Se ampliaron `firestore.rules` y `tests/trips.rules.test.cjs` para comprobar transiciones permitidas y rechazadas. Se optimizó la validación de aceptación para evitar superar el límite de expresiones del emulador de Firestore. Se reutilizaron `DriverContext` y las funciones existentes de `tripLogic.js`, sin crear pantallas nuevas ni agregar dependencias.

- **Archivos:** Creado: `src/services/tripLifecycle.js`. Modificados: `src/services/tripService.js`, `src/components/trips/TripRequests.js`, `firestore.rules`, `tests/trips.rules.test.cjs` y `docs/AI_LOG.md`.
- **Commit:** `ce6a048` (`feat: implementar ciclo de vida de viajes`), rama `feat/ciclo-vida-viajes`. Integrado en `main` mediante el Pull Request #17, merge `6e33cf2`.
- **Verificación:** Se ejecutó `npm run test:all` con 52 pruebas unitarias, 1 de integración y 67 de reglas de Firestore: **120 aprobadas y 0 fallidas**. No se detectaron avisos del límite de 1000 expresiones de Firestore. Se ejecutó `npx expo export --platform all`, generando correctamente los paquetes JavaScript de iOS y Android. `git diff --check` terminó sin observaciones antes de la última verificación. Las operaciones todavía no se han probado manualmente en un dispositivo con viajes reales o de prueba, y las reglas modificadas aún no se han verificado en Firebase de producción.
- **Aprendizaje:** Las transiciones de estado deben protegerse tanto en las transacciones del cliente como en las reglas de Firestore. Reutilizar el contexto global permite sincronizar el viaje activo sin duplicar su estado en otros componentes.

Entrada #18

- **Fecha:** 2026-11-13
- **Integrante:** Julian Cardona
- **Herramienta:** ChatGPT (OpenAI)
- **Objetivo:** Implementar y fortalecer la recuperación automática de viajes activos del conductor después de reiniciar la aplicación, evitando inconsistencias entre Firestore, AsyncStorage y el estado global.
- **Prompt (resumen; no consta como texto literal):** Revisar y corregir el módulo de recuperación de viajes activos, incluyendo consultas seguras a Firestore, persistencia local, protección frente a condiciones de carrera, pruebas automatizadas y conservación de las funcionalidades existentes.
- **Resumen de la respuesta:** Se incorporó la búsqueda de viajes activos del conductor en Firestore, utilizando los estados `accepted` e `in_progress`. Se modificó `DriverContext` para recuperar el viaje guardado localmente o consultar Firestore cuando no existe una referencia válida. Se añadieron controles de identidad y versiones para impedir que operaciones asíncronas antiguas sobrescriban selecciones recientes. También se incorporó un módulo que serializa las operaciones de AsyncStorage por clave.
- **Qué se usó / modificó / descartó:** Se reutilizaron Context API, Firebase, AsyncStorage y los servicios existentes. Se reforzó `TripRequests` para impedir la aceptación de nuevas solicitudes mientras se recupera o verifica un viaje activo. Se incorporaron reglas de seguridad para consultar únicamente viajes activos pertenecientes al conductor y un índice compuesto para respaldar la consulta. No se añadieron dependencias ni se reemplazó la arquitectura de navegación.
- **Archivos:** Modificados: `firebase.json`, `firestore.rules`, `package.json`, `src/components/trips/TripRequests.js`, `src/context/DriverContext.js`, `src/services/tripService.js` y `tests/trips.rules.test.cjs`. Creados: `firestore.indexes.json`, `src/utils/activeTripStorage.js`, `tests/activeTripService.test.cjs` y `tests/activeTripStorage.test.cjs`.
- **Commit:** `3c69583` (`fix: corregir recuperacion de viajes activos`), rama `fix/recuperacion-viaje-activo`. Integrado en `main` mediante el Pull Request #18, merge `67a7f66`.
- **Verificación:** Se ejecutaron 59 pruebas unitarias, 1 prueba de integración y 69 pruebas de reglas de Firestore mediante el emulador: **129 aprobadas y 0 fallidas**. Las pruebas incluyen recuperación de viajes activos,

protección de operaciones de almacenamiento y rechazo de consultas no autorizadas. `git diff --check` terminó sin observaciones en los archivos rastreados. Durante algunas pruebas negativas aparecieron mensajes `PERMISSION_DENIED` y errores de evaluación de reglas, que requieren revisión adicional. No se ha verificado todavía la recuperación manual después de reiniciar Expo Go ni el despliegue de las reglas y el índice en Firebase de producción.

- **Aprendizaje:** La recuperación de estado debe combinar persistencia local con una fuente de verdad remota. Las operaciones asíncronas necesitan controles de vigencia e identidad para evitar que resultados antiguos sobrescriban estados recientes. Las reglas de Firestore deben impedir el acceso a viajes de otros conductores.

Entrada #19

- **Fecha:** 2026-11-13
- **Integrante:** Julian Cardona
- **Herramienta:** ChatGPT (OpenAI)
- **Objetivo:** Corregir el bloqueo de la interfaz en "Recuperando viaje activo..." después de finalizar un viaje.
- **Prompt (resumen; no consta como texto literal):** Diagnosticar y corregir el problema que ocurre cuando un conductor finaliza un viaje, Firestore actualiza correctamente su estado a `completed`, pero la aplicación permanece indefinidamente recuperando el viaje activo.
- **Resumen de la respuesta:** Se identificó una condición de carrera en `DriverContext.js`. Al finalizar un viaje, la suscripción de Firestore detectaba que ya no estaba activo e iniciaba una reconciliación. Sin embargo, el cambio de `activeTripId` provocaba la limpieza de la suscripción y marcaba la operación como cancelada, impidiendo finalizar correctamente el estado de carga. Se separó la vigencia de la reconciliación de la cancelación de la suscripción, conservando las protecciones de identidad del conductor, versión de operación y selección de viajes.
- **Qué se usó / modificó / descartó:** Se reutilizaron React Context API, Firestore, AsyncStorage y los mecanismos existentes de recuperación. Se creó una función auxiliar para validar la vigencia de las reconciliaciones y facilitar sus pruebas. No se agregaron dependencias ni se modificaron las reglas de seguridad.
- **Archivos creados:** `src/utils/activeTripRecovery.js` y `tests/activeTripRecovery.test.cjs`.
- **Archivos modificados:** `src/context/DriverContext.js`, `package.json` y `docs/AI_LOG.md`.
- **Commit:** `de962fb` (`fix: corregir bloqueo al recuperar viaje finalizado`), rama `fix/bloqueo-recuperacion-viaje`. Integrado en `main` mediante el Pull Request #19, merge `fb76c1c`.
- **Verificación:** Se ejecutó `npm run test:all` con 64 pruebas unitarias, 1 prueba de integración y 69 pruebas de reglas de Firestore: **134 aprobadas y 0 fallidas**. Las cinco pruebas nuevas verifican la vigencia de las reconciliaciones y su invalidación ante cambios de conductor, operaciones posteriores, selección de otro viaje o desmontaje del contexto. Durante las pruebas negativas de seguridad aparecieron mensajes `PERMISSION_DENIED` y errores de evaluación de reglas, pendientes de investigación. Falta comprobar manualmente en Expo Go que finalizar un viaje permita regresar a la recepción de solicitudes sin bloquear la interfaz.
- **Aprendizaje:** La cancelación de una suscripción no necesariamente debe cancelar las operaciones asíncronas que inició. Separar sus ciclos de vida y validar la identidad y versión de cada operación ayuda a prevenir condiciones de carrera y estados de carga indefinidos.

Entrada #20

- **Fecha:** 2026-11-13

- **Integrante:** Julian Cardona
- **Herramienta:** ChatGPT (OpenAI)
- **Objetivo:** Implementar un temporizador de 20 segundos para aceptar solicitudes de viaje, evitando que el conductor acepte solicitudes después de agotarse el tiempo.
- **Prompt (resumen; no consta como texto literal):** Incorporar una cuenta regresiva de 20 segundos en las solicitudes pendientes de UberClone Driver, mostrar el tiempo restante, ocultar las solicitudes vencidas y evitar que las actualizaciones de Firestore reinicien el contador, conservando la funcionalidad existente.
- **Resumen de la respuesta:** Se implementó una utilidad para calcular el tiempo restante y determinar cuándo expira una solicitud. Se integró un contador visible en `TripRequests.js`, actualizado periódicamente, que oculta las solicitudes al agotarse los 20 segundos. Se incorporó una validación adicional antes de aceptar un viaje y se conservaron los tiempos iniciales de cada solicitud para evitar reinicios por actualizaciones de Firestore.
- **Qué se usó / modificó / descartó:** Se reutilizaron React Hooks, el componente de solicitudes y la suscripción existente a Firestore. La expiración se implementó localmente, sin cancelar ni modificar automáticamente los documentos de viajes pendientes en Firestore. No se añadieron dependencias ni se modificaron las reglas de seguridad.
- **Archivos creados:** `src/utils/tripRequestTimer.js` y `tests/tripRequestTimer.test.cjs`.
- **Archivos modificados:** `src/components/trips/TripRequests.js` y `docs/AI_LOG.md`.
- **Commit:** `8af239a` (`feat: implementar temporizador de solicitudes de viaje`), rama `feat/temporizador-solicitudes`. Integrado en `main` mediante el Pull Request #20, merge `7dd3317`.
- **Verificación:** Se ejecutaron 139 pruebas automatizadas con `node --test tests/*.test.cjs`, todas aprobadas y ninguna fallida. Las cinco pruebas nuevas verifican el inicio, la disminución, la expiración y los límites del contador. Se ejecutó `npx expo export --platform ios` correctamente, generando el bundle de iOS con 735 módulos. En Expo Go se comprobó manualmente que una solicitud pendiente muestra la cuenta regresiva y desaparece después de 20 segundos.
- **Limitaciones:** El temporizador comienza cuando el dispositivo observa la solicitud y no está sincronizado entre conductores ni respaldado por una fecha de expiración del servidor. Al reiniciar la aplicación, el plazo local puede comenzar nuevamente. Las solicitudes expiradas permanecen pendientes en Firestore hasta que otro proceso cambie su estado.
- **Aprendizaje:** Separar la lógica temporal en funciones puras facilita las pruebas. Conservar la primera aparición de cada solicitud evita que las actualizaciones en tiempo real reinicien el contador. Para una solución de producción, la expiración debe controlarse también desde el servidor.

Entrada #21

- **Fecha:** 2026-11-13
- **Integrante:** Julian Cardona
- **Herramienta:** ChatGPT (OpenAI)
- **Objetivo:** Implementar la visualización de mapas y rutas del conductor durante los viajes.
- **Prompt (resumen; no consta como texto literal):** Integrar mapas en UberClone Driver, mostrando la ubicación del conductor, los puntos de recogida y destino, las rutas por carretera, la distancia y el tiempo estimado.
- **Resumen de la respuesta:** Se implementó la visualización de rutas mediante un mapa interactivo y el servicio OSRM. Cuando el viaje está aceptado, la ruta conecta al conductor con el punto de recogida. Cuando está en curso, conecta al conductor con el destino. Se incorporaron marcadores y estimaciones de distancia y duración.
- **Archivos creados:** `src/services/routeService.js`, `src/utils/tripMapLogic.js` y `tests/tripMapLogic.test.cjs`.

- **Archivos modificados:** `src/components/map/DriverMap.js`, `src/screens/home/DriverHomeScreen.js` y `docs/AI_LOG.md`.
- **Qué se usó / modificó / descartó (hechos verificables en el commit `bf0b854`):** `src/services/routeService.js` consulta el servicio público OSRM (`https://router.project-osrm.org/route/v1/driving`) mediante `fetch`. `DriverMap` recibe el viaje activo desde `DriverContext` y la pestaña Viajes de `DriverHomeScreen` incorpora una tarjeta con el mapa del viaje. La lógica de puntos y tramos de la ruta se separó en `src/utils/tripMapLogic.js`, con pruebas en `tests/tripMapLogic.test.cjs`. No se agregaron dependencias: `package.json` no cambió en este commit.
- **Verificación:** Las 7 pruebas de lógica del mapa finalizaron correctamente. La exportación de iOS se completó sin errores. Se verificó manualmente la visualización de rutas en Expo Go.
- **Limitaciones:** El servicio público de OSRM es adecuado para pruebas, pero no garantiza disponibilidad para producción. Persiste un error de permisos al aceptar solicitudes en Firebase real que requiere diagnóstico independiente.
- **Aprendizaje:** Separar la lógica de rutas del componente visual facilita las pruebas y permite adaptar el recorrido al estado del viaje.
- **Commit:** `bf0b854` (`feat: implementar mapas y rutas de viajes del conductor`), rama `feat/rutas-mapa-viajes`. Integrado en `main` mediante el Pull Request #21, merge `ce80c52`.

Entrada #22

- **Fecha:** 2026-11-13
- **Integrante:** Julian Cardona (autor del commit `3dc1673`).
- **Herramienta:** Gemini CLI (Google)
- **Objetivo:** Finalización del proyecto: sistema de tarifas, pantallas de ganancias, historial, perfil, pagos, navegación y corrección de permisos.
- **Prompt:** No se conserva el texto literal.
- **Resumen de la respuesta:** Se implementó el sistema de tarifas con comisión del 20% en `fareLogic.js`. Se crearon las pantallas de `EarningsScreen`, `TripHistoryScreen`, `ProfileScreen` y `PaymentSummaryScreen`. Se integró `react-navigation` para la gestión de pantallas. Se corrigieron las reglas de Firestore para manejar mejor los formatos de fecha en la aceptación de viajes.
- **Qué se usó / modificó / descartó (hechos verificables en el commit `3dc1673`, completados en la auditoría de la entrada #26):** Se agregaron las dependencias `@react-navigation/native` ^7.5.1, `@react-navigation/native-stack` ^7.21.0, `@react-navigation/bottom-tabs` ^7.21.0, `react-native-safe-area-context` ~5.11.1 y `react-native-screens` ~4.30.1. `fareLogic.js` calcula la comisión de la plataforma (20 %) y la ganancia del conductor (80 %). `AppNavigator.js` define un navegador de pila para registro e inicio y, además, los navegadores de pestañas `MainTabs` y `EarningsTabs`, que no llegaron a renderizarse. `App.js` pasa a usar `NavigationContainer` y `AppNavigator`. `DriverHomeScreen.js` incorpora las subpestañas Resumen, Pagos e Historial y la pestaña Perfil. `tripService.js` incorpora `getDriverTripHistory`, que consulta viajes `completed` y `cancelled` con límite 50. En `firestore.rules`, `driverCanAcceptTrip` exige que las fechas de vencimiento tengan formato ISO 8601 antes de compararlas.
- **Archivos creados:** `src/utils/fareLogic.js`, `src/screens/home/EarningsScreen.js`, `src/screens/home/TripHistoryScreen.js`, `src/screens/home/ProfileScreen.js`, `src/screens/home/PaymentSummaryScreen.js` y `src/navigation/AppNavigator.js`.
- **Archivos modificados:** `App.js`, `src/screens/home/DriverHomeScreen.js`, `firestore.rules`, `src/services/tripService.js`, `package.json`, `package-lock.json` y `docs/AI_LOG.md`.

- **Commit:** `3dc1673` (`feat: agregar modulos financieros historial perfil y navegacion`), rama `feat/completar-modulos-conductor`. Integrado en `main` mediante el Pull Request #23, merge `7ee3ab0`.
- **Verificación:** Se instalaron las dependencias de navegación. Se actualizaron las reglas de seguridad. Se verificó la exportación de iOS. No consta la ejecución de pruebas automatizadas para estos cambios.
- **Incidencias posteriores:** La consulta de `getDriverTripHistory` no estaba autorizada por `firestore.rules`, lo que producía `Missing or insufficient permissions` en Historial, Ganancias y Pagos; se corrigió en la entrada #23. `DriverHomeScreen` anidaba listas virtualizadas dentro de un `ScrollView`, `AppNavigator` no usaba `loadError` y los navegadores de pestañas no se renderizaban; se corrigió en la entrada #24.
- **Aprendizaje:** La integración de navegación profesional mejora la experiencia del usuario. Las reglas de Firestore deben ser extremadamente robustas con los formatos de datos.

Entrada #23

- **Fecha:** 2026-11-13
- **Integrante:** Julian Cardona
- **Herramienta:** Claude Code (Anthropic), modelo Claude Opus 5.5
- **Objetivo:** Diagnosticar y corregir los errores `FirebaseError: Missing or insufficient permissions` que impedían completar el ciclo de viajes y consultar historial, ganancias y pagos.
- **Prompt (resumen; el texto literal se conserva en la sesión de Claude Code):** Investigar los errores de permisos de Firestore distinguiendo problemas de reglas, consultas, datos y despliegue; corregirlos manteniendo reglas restrictivas y compatibilidad con documentos existentes; añadir pruebas de reglas y de aceptación; no modificar datos reales ni desplegar reglas sin autorización.
- **Resumen de la respuesta:** Se reprodujeron los fallos en el emulador antes de corregirlos. (1) **Reglas/consulta:** `getDriverTripHistory` (usada por `TripHistoryScreen`, `EarningsScreen` y `PaymentSummaryScreen`) consulta `completed`/`cancelled` con límite 50, pero `allow list` solo autorizaba `accepted`/`in_progress` con límite 2; el emulador respondía `false for 'list'`. Se añadió una rama de lista exclusiva para viajes finalizados del propio conductor, con límite máximo de 50 y sin exigir conexión. (2) **Datos:** la regla de aceptación exigía que `driverId` existiera con valor `null`; un viaje creado en la consola sin ese campo pasaba la comprobación del cliente (`driverId != null`) pero era rechazado por las reglas. Ahora el campo ausente se trata como no asignado en reglas y en `tripLogic.js`. (3) **Mensajes:** `tripAcceptance.js` valida origen, destino y tarifa antes de escribir y muestra "La solicitud tiene datos incompletos o inválidos" en lugar del error de permisos. La transacción sigue modificando solo `status` y `driverId`, como exigen las reglas. Con las reglas anteriores y datos bien formados, aceptar, iniciar, finalizar y cancelar funcionaban en el emulador; si la aceptación falla en Firebase real, la causa está en las reglas publicadas o en los datos reales (ver pendientes).
- **Qué se usó / modificó / descartó:** Se mantuvieron todas las validaciones de conductor (aprobación, registro, conexión, documentos vigentes) y de viaje. No se usó `allow read, write: if true`. Se descartó filtrar viajes mal formados de la lista de pendientes para no ocultar problemas de datos. Se descartó ejecutar `npx expo lint` porque el proyecto no tiene ESLint configurado y el comando instalaría dependencias nuevas. Las pruebas de viajes usan proyectos propios del emulador para que `clearFirestore()` de archivos ejecutados en paralelo no borre datos ajenos.
- **Archivos creados:** `tests/tripCycle.integration.test.cjs`.
- **Archivos modificados:** `firestore.rules`, `src/services/tripService.js`, `src/services/tripAcceptance.js`, `src/utils/tripLogic.js`, `tests/trips.rules.test.cjs`, `tests/tripAcceptance.test.cjs`, `tests/tripLogic.test.cjs`, `tests/activeTripService.test.cjs`, `package.json` (scripts de pruebas) y `docs/AI_LOG.md`.

- **Commit:** `f5e70b3`, rama `feat/completar-modulos-conductor`. Integrado en `main` mediante el Pull Request #23, merge `7ee3ab0`.
- **Verificación:** `npm run test:all` con el emulador local: 79 pruebas unitarias, 11 de integración y 77 de reglas, **167 aprobadas y 0 fallidas** (línea base antes de los cambios: 134/134, sin cobertura del historial). La prueba de ciclo completo ejecutada contra las reglas anteriores falló en el historial (`false for 'list'`), y pasa con las nuevas. `npx expo export --platform ios` generó el bundle correctamente. `git diff --check` sin observaciones. Con autorización del integrante se ejecutó `npx firebase-tools deploy --only firestore:rules,firestore:indexes --project uberclone-driver-8e0bf` (2026-11-13): reglas compiladas y publicadas e índice de `trips` (`driverId`, `status`) desplegado, sin advertencias ni errores; `firestore:indexes` confirmó que el índice publicado coincide con el archivo local. El integrante confirmó en Expo Go que el error de permisos desapareció y que el ciclo de viajes, Ganancias, Pagos e Historial funcionan.
- **Limitaciones:** El historial sin `orderBy` devuelve hasta 50 viajes finalizados sin garantía de ser los más recientes; se ordenan en el cliente. Un conductor con más de 50 viajes finalizados verá ganancias parciales.
- **Aprendizaje:** Las reglas de Firestore no filtran resultados: cada consulta nueva necesita una regla que la autorice explícitamente, y conviene acompañarla de una prueba con la misma consulta del servicio. Firestore evalúa cada valor de un filtro `in` como una consulta separada, lo que debe tenerse en cuenta al diseñar pruebas negativas.

Entrada #24

- **Fecha:** 2026-11-13
- **Integrante:** Julian Cardona
- **Herramienta:** Claude Code (Anthropic), modelo Claude Opus 5.5
- **Objetivo:** Eliminar la causa de la advertencia `VirtualizedLists should never be nested inside plain ScrollViews with the same orientation` y auditar la navegación del conductor.
- **Prompt (resumen; el texto literal se conserva en la sesión de Claude Code):** Localizar listas virtualizadas anidadas en `ScrollView` verticales y corregir la estructura sin ocultar advertencias con `LogBox.ignoreLogs`; revisar `App.js`, `AppNavigator.js` y las pantallas del conductor para garantizar el acceso a Inicio, Ganancias, Historial, Perfil, Resumen de pagos y Viaje activo, sin alterar el registro ni la persistencia.
- **Resumen de la respuesta:** `DriverHomeScreen` envolvía todas sus pestañas en un único `ScrollView` vertical y dentro renderizaba `EarningsScreen` y `TripHistoryScreen` (ambas con `FlatList`), además de `PaymentSummaryScreen` y `ProfileScreen` (con su propio `ScrollView`). Ahora las pestañas Ganancias y Perfil se renderizan fuera del `ScrollView` principal, bajo un encabezado fijo, y cada pantalla gestiona su propio desplazamiento. Inicio y Viajes conservan el `ScrollView`. En `EarningsScreen` el resumen pasó a `ListHeaderComponent` para que se desplace junto con la lista. En la navegación se detectó que `AppNavigator` definía `MainTabs` y `EarningsTabs` que nunca se renderizaban (pantallas duplicadas) y que, desde el commit `3dc1673`, ignoraba `loadError`: si fallaba la consulta del registro se abría Inicio en lugar del mensaje de error. Se eliminaron los navegadores sin uso y se restauraron los estados de carga y error previos. También se añadió el manejo del botón "atrás" de Android (vuelve a Inicio desde otra pestaña en lugar de cerrar la app) y la tarjeta "Tu actividad" de Inicio ahora ofrece "Ver viaje activo" cuando existe uno, en lugar del texto obsoleto "estarán disponibles en un próximo módulo".
- **Qué se usó / modificó / descartó:** No se usó `LogBox.ignoreLogs`. Se descartó migrar las pestañas propias a `@react-navigation/bottom-tabs` para no cambiar la interfaz confirmada antes de la entrega. La dependencia `@react-navigation/bottom-tabs` quedó sin uso, pero no se desinstaló para no modificar dependencias sin autorización. No se modificaron reglas, servicios, registro ni persistencia.

- **Archivos modificados:** `src/screens/home/DriverHomeScreen.js`, `src/screens/home/EarningsScreen.js`, `src/navigation/AppNavigator.js`, `App.js` y `docs/AI_LOG.md`.
- **Commit:** `3cb5eb2`, rama `feat/completar-modulos-conductor`. Integrado en `main` mediante el Pull Request #23, merge `7ee3ab0`.
- **Verificación:** `npm run test:all`: 79 unitarias, 11 de integración y 77 de reglas, **167 aprobadas y 0 fallidas**. `npx expo-doctor`: 20/20 comprobaciones superadas. `npx expo export --platform ios`: bundle generado (987 módulos). `git diff --check` sin observaciones. El integrante confirmó en Expo Go que la advertencia ya no aparece y que Inicio, Viajes, Ganancias, Pagos, Historial y Perfil funcionan.
- **Limitaciones:** No hay pruebas automatizadas de interfaz; la estructura de pantallas se verifica manualmente. Con el cambio de estructura, las tarjetas de Ganancias, Pagos, Historial y Perfil quedan alineadas con el encabezado (20 px de margen) en lugar del doble margen anterior.
- **Aprendizaje:** Una pantalla con su propia `FlatList` no debe incrustarse dentro de un `ScrollView` vertical: además de la advertencia, la lista pierde la virtualización y renderiza todos sus elementos. El contenido fijo debe ir en `ListHeaderComponent`.

Entrada #25

- **Fecha:** 2026-11-13 (fecha del commit; registro retrospectivo añadido en la auditoría de la entrada #26)
- **Integrante:** Julian Cardona (autor del commit `df9b2a7`).
- **Herramienta:** No verificable. No existe evidencia en el repositorio, en el commit ni en el Pull Request #22 que permita determinar si se utilizó una herramienta de IA ni cuál.
- **Objetivo:** Validar la aceptación de viajes con datos representativos de un conductor real, en el contexto del error de permisos señalado en la entrada #21.
- **Prompt:** No consta.
- **Resumen del cambio (hechos verificables en el diff):** En la prueba de integración de `tests/tripAcceptance.test.cjs` se sustituyeron los datos del conductor y del viaje por valores fijos, descritos en el mensaje del commit como "datos representativos del conductor": fechas de vencimiento concretas (`2028-12-11T22:49:36.000Z` y `2029-02-15T22:49:51.000Z`) en lugar de fechas calculadas a cinco años, `registrationCompletedAt` fijo, otros datos personales y del vehículo, y otras direcciones y coordenadas de recogida y destino.
- **Archivos modificados:** `tests/tripAcceptance.test.cjs`.
- **Commit:** `df9b2a7` (`test: validar aceptacion con datos representativos del conductor`), rama `fix/permisos-aceptacion-viajes`. Integrado en `main` mediante el Pull Request #22, merge `ccffec8`. El Pull Request no tiene descripción.
- **Verificación:** No consta.
- **Observación:** Los datos representativos siguen en `tests/tripAcceptance.test.cjs` (reescrito en la entrada #23). No se añadió un comentario `AI-assisted` en el código porque no hay evidencia de asistencia de IA.

Entrada #26

- **Fecha:** 2026-11-13
- **Integrante:** Julian Cardona
- **Herramienta:** Claude Code (Anthropic), modelo Claude Opus 5.5
- **Objetivo:** Auditar la trazabilidad del código y del uso de IA desde el primer commit hasta `main`, y corregir la documentación y los comentarios `AI-assisted` sin modificar la lógica funcional.

- **Prompt (resumen; el texto literal se conserva en la sesión de Claude Code):** Auditar todos los archivos, commits, ramas y Pull Requests; construir una matriz de trazabilidad; comprobar los comentarios `AI-assisted` y todas las entradas de esta bitácora; completar los hashes pendientes; distinguir ChatGPT, Claude, Gemini CLI y Claude Code; marcar como no verificable lo que carezca de evidencia y no inventar prompts, autores, fechas ni resultados. Tras revisar el diagnóstico, el integrante autorizó las correcciones y decidió: registrar `df9b2a7` como no verificable, conservar la fecha de la entrada #14 con una nota, dejar pendiente el aprendizaje de la entrada #16, documentar solo hechos del diff en la entrada #21, actualizar `README.md` y añadir la referencia #22 en `tripService.js`.
- **Resumen de la respuesta:** Se auditaron 75 archivos rastreados, 57 commits (34 de trabajo y 23 de fusión), 23 Pull Requests y 47 referencias de rama. No hay archivos eliminados ni renombrados y `.env` nunca se versionó. De 58 archivos de código, 43 tenían referencias correctas; en 12 faltaban referencias o eran incorrectas. El commit `be8ded2` había sustituido las referencias #3, #6, #7 y #8 de `RegistrationScreen.js` por un comentario sin número. Las entradas #14, #15 y #17 a #22 no tenían hash; la entrada #22 no tenía integrante, prompt, uso detallado ni commit; y varias entradas contenían inexactitudes en sus listas de archivos.
- **Qué se usó / modificó / descartó:** Se añadieron o corrigieron referencias `AI-assisted` en 15 archivos de código y se verificó que todas las líneas modificadas en ellos son comentarios. No se añadió la referencia #8 en `RegistrationScreen.js` porque su cambio (etiqueta "pendiente" de la tercera sección) fue sustituido por la entrada #9. No se añadieron referencias en `index.js` ni en otros archivos de la plantilla de Expo. En esta bitácora se completaron los hashes y las fusiones, se completó la entrada #22 con hechos del diff, se corrigieron las listas de archivos de las entradas #5, #6, #11 y #15, se añadió una nota de fecha en la entrada #14, se marcaron como resumen los prompts que no constan como texto literal, se unificaron los nombres de las herramientas y se reorganizaron las consideraciones generales. Se actualizó `README.md` con las funcionalidades implementadas, la estructura real y el procedimiento de pruebas. No se reescribieron commits anteriores: los commits `f5e70b3`, `3cb5eb2` y `8cf42fd` conservan el trailer `Co-Authored-By` de Claude, coherente con las entradas #23 y #24.
- **Archivos modificados:** `App.js`, `README.md`, `docs/AI_LOG.md`, `firestore.rules` (solo comentarios; no requiere volver a publicar las reglas), `src/components/registration/DocumentsPaymentsForm.js`, `src/components/registration/VehicleDataForm.js`, `src/screens/home/DriverHomeScreen.js`, `src/screens/home/EarningsScreen.js`, `src/screens/home/PaymentSummaryScreen.js`, `src/screens/home/ProfileScreen.js`, `src/screens/home/TripHistoryScreen.js`, `src/screens/registration/RegistrationScreen.js`, `src/services/firestoreService.js`, `src/services/tripService.js`, `src/utils/fareLogic.js`, `src/utils/registrationValidation.js` y `tests/trips.rules.test.cjs`.
- **Commit:** `4bb9d62` (`docs: auditar trazabilidad del uso de IA y corregir referencias`), rama `docs/auditoria-trazabilidad-ia`. El hash se registró en un commit documental posterior. La integración en `main` queda pendiente de la revisión del Pull Request.
- **Verificación:** `npm run test:all` con el emulador local: 79 pruebas unitarias, 11 de integración y 77 de reglas, **167 aprobadas y 0 fallidas**; las pruebas de reglas compilan `firestore.rules` con los comentarios nuevos. `npx expo-doctor`: 20/20 comprobaciones superadas. `npx expo export --platform ios`: bundle generado (987 módulos). `git diff --check` sin observaciones. Un análisis automático del diff confirmó que en los archivos de código solo cambiaron comentarios.
- **Aprendizaje:** La trazabilidad se pierde cuando un commit posterior sustituye referencias existentes o cuando una entrada no registra su commit. Cruzar cada archivo con los commits que lo modificaron y con las entradas de la bitácora permite detectar estas omisiones sin depender de la memoria del desarrollo.

Entrada #27

- **Fecha:** 2026-11-13
- **Integrante:** Julian Cardona
- **Herramienta:** Claude Code (Anthropic), modelo Claude Opus 5.5
- **Objetivo:** Implementar el módulo de tarifas y comisiones de la guía (sección 5): configuración centralizada, función pura `calculateFare`, desglose guardado al finalizar cada viaje, protección en `firestore.rules` y pruebas.
- **Prompt (texto literal):**

```
# UberClone Driver — Implementación del módulo de tarifas y comisiones

Actúa como desarrollador senior de React Native, Expo, JavaScript y Firebase, con experiencia en lógica financiera, pruebas automatizadas y seguridad de Firestore.

## Contexto

Estamos desarrollando UberClone Driver para una entrega universitaria. El proyecto utiliza:

- Expo SDK 58, React Native y JavaScript, sin TypeScript.
- Firebase Authentication anónima.
- Cloud Firestore.
- Cloudinary.
- Navegación y flujo de viajes ya implementados.
- Pruebas unitarias, de integración y reglas de Firestore.
- Documentación de IA en `docs/AI_LOG.md`.

La rama de trabajo será `feat/tarifas-comisiones`, creada previamente por mí.

La rama principal ya contiene los PR #23 y #24.

## Reglas obligatorias

1. Trabaja exclusivamente sobre la rama actual. Verifica que sea `feat/tarifas-comisiones`.
2. NO ejecutes `git commit`, `git push`, `git merge`, `git rebase`, `gh pr create` ni otras operaciones de publicación.
3. No cambies de rama ni crees ramas.
4. No añadas `Co-Authored-By` ni mensajes automáticos de autoría.
5. No modifiques `.env` ni expongas credenciales.
6. No despliegues reglas, índices ni servicios Firebase.
7. No reescribas los módulos existentes si pueden ampliarse.
8. Mantén compatibilidad con los documentos de viajes ya guardados.
9. Usa exclusivamente JavaScript.
10. Documenta la asistencia de IA en `docs/AI_LOG.md` y agrega comentarios `AI-assisted` donde corresponda.
11. Puedes crear y modificar archivos, instalar dependencias estrictamente necesarias y ejecutar pruebas.
12. No avances al módulo de solicitudes o pagos. Debes entregar únicamente el módulo de tarifas y comisiones.

## Objetivo principal

Implementar completamente el sistema de cálculo de tarifas exigido por la guía docente,
```

integrándolo con el flujo actual de viajes y las ganancias del conductor.

1. Configuración de precios

Crear `src/config/pricing.js` con los valores:

- Tarifa base: 3.500 COP.
- Precio por kilómetro: 1.200 COP.
- Precio por minuto: 250 COP.
- Tarifa mínima: 6.000 COP.
- Económico: multiplicador 1.0.
- XL: multiplicador 1.4.
- Premium: multiplicador 1.8.
- Comisión de plataforma: 20%.
- Propina: 100% para el conductor.
- Redondeo de la tarifa al múltiplo de 100 COP más cercano.

Utiliza constantes claras y centralizadas.

2. Calculadora pura de tarifas

Crear una función `calculateFare({ distanceKm, durationMin, category, tip })`.

Debe:

- Validar los parámetros recibidos.
- Calcular el precio usando la tarifa base, kilómetros, minutos y multiplicador de categoría.
- Aplicar la tarifa mínima y el redondeo exigidos.
- Calcular la comisión del 20% únicamente sobre la tarifa, excluyendo la propina.
- Calcular la ganancia neta del conductor incluyendo el 100% de la propina.
- Devolver un objeto consistente y fácil de utilizar en toda la aplicación.

Revisar la guía original y comprobar sus ejemplos:

- Económico: tarifa 19.200 COP, comisión 3.840 COP, neto 15.360 COP.
- XL: tarifa 26.900 COP, comisión 5.380 COP, neto 21.520 COP.

Si los ejemplos presentan ambigüedades de distancia, duración o redondeo, documenta las diferencias; no fuerces resultados incorrectos.

3. Integración con el código existente

Audita antes de modificar:

- `src/utils/fareLogic.js`
- `src/services/tripService.js`
- `src/utils/tripLogic.js`
- `src/utils/tripLifecycle.js`
- `src/screens/home/EarningsScreen.js`
- `src/screens/home/DriverHomeScreen.js`
- `firestore.rules`
- Pruebas existentes.

Reutiliza lo que ya funciona.

No cambies innecesariamente el ciclo de estados del viaje.

Cuando se complete un viaje, debe quedar guardado un desglose coherente que incluya:

- `fare`
- `commission`
- `commissionRate`
- `driverEarnings`
- `tip`

Conserva compatibilidad con `estimatedFare` y con los viajes antiguos.

Define explícitamente cómo se obtiene una tarifa final si no existen distancia y duración verificadas; no inventes recorridos ni tiempos. Para datos históricos incompletos, usa una estrategia de compatibilidad documentada.

4. Seguridad y consistencia

Revisa que un conductor no pueda alterar arbitrariamente la tarifa, la comisión o sus ganancias mediante escrituras directas en Firestore.

Si necesitas modificar `firestore.rules`, hazlo localmente y agrega pruebas.

No publiques las reglas. Si la seguridad requiere un backend o una operación privilegiada que todavía no existe, señala esa dependencia y no presentes como seguro un cálculo exclusivamente del cliente.

Evita que la misma finalización de viaje duplique cobros o ganancias.

5. Pruebas obligatorias

Implementa pruebas para:

- Tarifa mínima.
- Redondeo.
- Cada categoría.
- Comisión del 20%.
- Propina completa para el conductor.
- Distancias y tiempos cero.
- Entradas inválidas.
- Compatibilidad con viajes antiguos.
- Persistencia del desglose.
- Reglas Firestore modificadas.
- Reintentos de finalización del viaje.

Después ejecuta:

- Todas las pruebas existentes.
- `npx expo-doctor`.
- `npx expo export --platform ios`.
- `git diff --check`.

No ocultes fallos. Si alguna prueba no puede ejecutarse, explica por qué.

Entrega final

Al terminar, presenta:

```
1. Resumen de lo implementado.
2. Archivos creados y modificados.
3. Decisiones técnicas y compatibilidad.
4. Resultado exacto de cada conjunto de pruebas.
5. Cambios en reglas Firestore y necesidad de despliegue.
6. Riesgos o limitaciones pendientes.
7. Comandos sugeridos para que yo revise los cambios.
8. Confirmación de que NO realizaste commits, push, PR, merge ni despliegues.

No avances al módulo 2.

Quiero un módulo completo, coherente, probado e integrado con el proyecto existente, no una demostración aislada.

 Importante: No existe una aplicación de pasajero y no debes crearla. Las solicitudes de viaje se simularán mediante documentos de prueba en Firestore. En este módulo concéntrate únicamente en tarifas y comisiones. Si encuentras dependencias relacionadas con solicitudes o pagos, documéntalas para los siguientes módulos sin implementarlas todavía.
```

- **Resumen de la respuesta:** Se creó `src/config/pricing.js` y se amplió `src/utils/fareLogic.js` con `calculateFare`, que usa aritmética entera y reproduce los ejemplos de la guía sin ambigüedades (Económico 19.200 / 3.840 / 15.360 y XL 26.880 → 26.900 / 5.380 / 21.520). Al finalizar un viaje se guarda el desglose en la misma transacción y `firestore.rules` lo recalcula con la misma fórmula, rechazando cualquier valor alterado. La finalización es idempotente ante reintentos y ante dos finalizaciones simultáneas.
- **Qué se usó / modificó / descartó:**
  - Se reutilizaron `calculateFareBreakdown`, `COMMISSIONS` y `formatCurrency` (ahora basados en la misma comisión) y el ciclo de estados existente, sin estados nuevos.
  - Estrategia de tarifa final: si el documento del viaje tiene `distanceKm`, `durationMin` y `category` válidos, se usa `calculateFare` (`fareSource: "calculated"`). Si no, se conserva `estimatedFare` sin recalcular, redondear ni inventar recorridos (`fareSource: "estimated"`). Los viajes finalizados antes de este módulo, sin desglose, se muestran con la comisión vigente sobre `estimatedFare` (`getTripEarnings`, origen `legacy`).
  - La propina se toma del documento del viaje (0 si no existe); el conductor no puede fijarla.
  - Seguridad: como no existe backend, se descartó confiar en el cálculo del cliente. Las reglas replican la fórmula (`categoryMultiplierPercent`, `hasPricingInputs`, `calculatedTripFare`, `validCompletionFareBreakdown`) y solo aceptan el desglose exacto. Los datos de entrada (distancia, duración, categoría, propina y tarifa estimada) solo pueden escribirse al crear el viaje, operación que las reglas prohíben a la aplicación.
  - Las pruebas detectaron que, con dos finalizaciones simultáneas, la segunda escritura recibía `permission-denied` (no reintentable). Ningún valor se duplicaba, pero la aplicación mostraba un error. Se añadió una comprobación posterior: si el viaje ya quedó finalizado por el mismo conductor, la operación se considera cumplida.
  - Se descartó integrar Google Distance Matrix, la tarjeta de solicitud con la ganancia estimada y los pagos, por pertenecer a los módulos siguientes. Los textos fijos de `PaymentSummaryScreen` ("Próximo pago estimado", "Lunes próximo") quedan para el módulo de pagos.
  - No se instalaron dependencias.
- **Archivos creados:** `src/config/pricing.js` y `tests/fareLogic.test.cjs`.

- **Archivos modificados:** `src/utils/fareLogic.js`, `src/services/tripLifecycle.js`, `src/screens/home/EarningsScreen.js`, `src/screens/home/PaymentSummaryScreen.js`, `src/screens/home/TripHistoryScreen.js`, `firestore.rules`, `tests/trips.rules.test.cjs`, `tests/tripCycle.integration.test.cjs`, `package.json` (script `test:unit`), `README.md` y `docs/AI_LOG.md`.
- **Commit:** `f7cbd9a` (`feat: implementar tarifas y comisiones con validacion en firestore`), rama `feat/tarifas-comisiones`.
- **Verificación:** `npm run test:all` con el emulador local de Firestore: 97 pruebas unitarias, 14 de integración y 92 de reglas, **203 aprobadas y 0 fallidas**. Las 18 pruebas unitarias nuevas cubren los ejemplos de la guía, cada categoría, la tarifa mínima, el redondeo (incluida una mitad exacta), distancia y tiempo cero, la comisión, la propina, entradas inválidas y la compatibilidad con viajes antiguos. Las 15 pruebas de reglas nuevas comprueban que las reglas aceptan el cálculo de la aplicación en 13 combinaciones y rechazan tarifas, comisiones, ganancias, porcentajes y propinas alterados, desgloses incompletos y reescrituras de un viaje finalizado. Las pruebas de integración verifican la persistencia del desglose, el reintento y la finalización simultánea; la de concurrencia se ejecutó cinco veces seguidas sin fallos. `npx expo-doctor`: 20/20. `npx expo export --platform ios`: bundle generado (988 módulos). `git diff --check` sin observaciones. No se probó en Expo Go ni contra Firebase de producción: las reglas nuevas no se han publicado.
- **Limitaciones y dependencias para los módulos siguientes:**
  - Al publicar estas reglas, una versión anterior de la aplicación que finalice viajes sin desglose será rechazada; la aplicación y las reglas deben actualizarse juntas.
  - La fórmula está duplicada en `src/config/pricing.js` y `firestore.rules`; cualquier cambio de tarifa exige actualizar ambos.
  - La seguridad depende de que los datos de entrada del viaje sean confiables. Hoy se crean en la consola; con una app de pasajero, la distancia y la duración deberían calcularse en un backend (Distance Matrix).
  - Pendiente para el módulo de solicitudes: mostrar la tarifa y la ganancia estimadas con `calculateFare`.
  - Pendiente para el módulo de pagos: registrar el método y el estado del pago, y calcular el saldo del conductor (comisión de viajes en efectivo).
- **Aprendizaje:** Cuando no existe un backend, las reglas de Firestore pueden recalcular un valor derivado de datos que el cliente no puede modificar, siempre que la fórmula use operaciones con resultados idénticos en ambos entornos (enteros y redondeo `floor(x + 0,5)`). Las pruebas de concurrencia contra el emulador revelan comportamientos de las transacciones que no aparecen en pruebas secuenciales.

Entrada #28

- **Fecha:** 2026-11-13
- **Integrante:** Julian Cardona
- **Herramienta:** Claude Code (Anthropic), modelo Claude Opus 5.5
- **Objetivo:** Completar las solicitudes de viaje en tiempo real (tarjeta completa, 15 s, aceptar y rechazar) y el ciclo completo del viaje (en camino, llegada al origen, pasajero a bordo, finalización), con reglas de Firestore, simulador de solicitudes y pruebas.
- **Prompt (texto literal):**

```
# UberClone Driver — Módulo 2: Solicitudes y ciclo completo del viaje

Actúa como desarrollador senior especializado en React Native, Expo SDK 58, JavaScript, Firebase
Authentication, Cloud Firestore, geolocalización, estados transaccionales y pruebas automatizadas.

## 1. Contexto del proyecto
```

Estamos desarrollando UberClone Driver, exclusivamente la aplicación del conductor.

Tecnologías actuales:
- React Native con Expo SDK 58.
- JavaScript, sin TypeScript.
- Firebase Authentication anónima.
- Cloud Firestore con reglas de seguridad.
- Cloudinary para documentos e imágenes.
- Ubicación mediante expo-location.
- Mapas y rutas con la integración existente de OSRM.
- Context API y hooks de React.
- Pruebas unitarias, integración y reglas de Firestore.

La rama principal contiene el PR #25, que implementó tarifas y comisiones con cálculo centralizado, desglose financiero y nuevas reglas de Firestore.

El último merge de main es `88f072c`.

La rama actual de trabajo es:

`feat/solicitudes-ciclo-viaje`

Ya fue creada por el estudiante.

El módulo de tarifas y comisiones reportó 203 pruebas aprobadas.

2. Reglas obligatorias de trabajo

1. Comprueba que estás en `feat/solicitudes-ciclo-viaje`.
2. No crees ramas ni cambies de rama.
3. No ejecutes git commit, git push, git merge, git rebase ni comandos de Pull Request.
4. El estudiante y ChatGPT se encargan exclusivamente del control de versiones y GitHub.
5. No añadas trailers `Co-Authored-By` ni firmas automáticas.
6. No modifiques `.env` ni expongas secretos.
7. No publiques reglas ni índices de Firebase.
8. No modifiques datos reales de Firestore sin autorización.
9. No reescribas módulos existentes innecesariamente.
10. Mantén compatibilidad con los viajes anteriores y el módulo de tarifas.
11. Utiliza JavaScript, no TypeScript.
12. Puedes crear y modificar archivos, ejecutar pruebas e instalar dependencias únicamente cuando sean imprescindibles.
13. Documenta la asistencia de IA en `docs/AI_LOG.md`, con una nueva entrada #28 y comentarios `AI-assisted` correspondientes.
14. No avances al módulo de pagos. Solo prepara los datos y contratos necesarios para su futura integración.
15. No implementes una aplicación de pasajero.

3. Auditoría técnica inicial

Antes de modificar archivos, inspecciona el código real y determina qué funcionalidades ya existen y cuáles deben completarse.

Revisa especialmente:

- `src/services/tripService.js`

- `src/services/tripLifecycle.js`
- `src/utils/tripLogic.js`
- `src/utils/tripRequestTimer.js`
- `src/utils/tripMapLogic.js`
- `src/utils/fareLogic.js`
- `src/config/pricing.js`
- `src/screens/home/DriverHomeScreen.js`
- Los componentes actuales de solicitudes.
- Los hooks y el contexto del conductor.
- `firestore.rules`
- Las pruebas unitarias y de integración existentes.

No presupongas que los nombres o ubicaciones de todos los archivos son exactos: inspecciona el repositorio y adapta la implementación a su estructura real.

4. Solicitudes completas en tiempo real

Conserva el sistema de suscripción Firestore mediante `onSnapshot`.

Implementa o completa el hook `useTripRequests`, reutilizando los servicios actuales.

Debe:

- Escuchar solicitudes disponibles para el conductor.
- Limpiar las suscripciones al desmontar componentes.
- Evitar suscripciones duplicadas.
- Controlar estados de carga y errores.
- No mostrar solicitudes expiradas, rechazadas o ya aceptadas por otro conductor.
- Recuperar correctamente el estado después de cerrar y abrir la aplicación.
- Mantener la compatibilidad con el Context API existente.

La tarjeta de solicitud debe mostrar:

- Origen.
- Destino.
- Distancia estimada.
- Duración estimada.
- Categoría del vehículo.
- Método de pago.
- Tarifa estimada.
- Comisión estimada.
- Ganancia neta estimada del conductor.
- Tiempo restante para responder.

Usa los cálculos del módulo de tarifas. No dupliques fórmulas ni inventes distancias o tiempos.

Para solicitudes antiguas sin todos los campos, presenta información disponible sin bloquear innecesariamente el funcionamiento.

5. Temporizador obligatorio de 15 segundos

La guía exige exactamente 15 segundos.

Actualiza la constante existente de 20.000 ms a 15.000 ms.

Requisitos:

- Cuenta regresiva visible.
- Basada en un timestamp persistente de la solicitud, no únicamente en el reloj de montaje del componente.
- Debe funcionar correctamente si la aplicación se minimiza o vuelve al primer plano.
- No debe reiniciarse al volver a renderizar.
- No debe permitir aceptar una solicitud vencida.
- Limpieza correcta de intervalos y efectos.
- Pruebas de los límites temporales.

Si el timestamp del servidor aún no está disponible, evita tratar una hora provisional del cliente como definitiva.

6. Aceptar y rechazar solicitudes

Implementa:

- Botón Aceptar.
- Botón Rechazar.
- Rechazo automático al vencer el tiempo.
- Estados de procesamiento para evitar pulsaciones duplicadas.
- Manejo de errores de conexión.
- `useCallback` para los manejadores de aceptación y rechazo.

Condiciones de seguridad:

- Una solicitud no puede ser aceptada por dos conductores.
- Una solicitud vencida no puede aceptarse.
- Un rechazo de un conductor no debe cancelar injustificadamente la solicitud para todos si el modelo permite varios conductores.
- El rechazo automático debe quedar persistido o reflejado de manera coherente en Firestore, no limitarse a ocultar una tarjeta.
- Si el conductor está desconectado, no debe aceptar nuevas solicitudes.
- Las reglas Firestore deben impedir transiciones no autorizadas.

Define claramente si las solicitudes son asignadas a un conductor o compartidas entre varios. Adapta el rechazo a ese modelo y documenta la decisión.

Utiliza transacciones cuando sea necesario.

7. Ciclo completo del viaje

Completa el flujo:

Solicitud pendiente → Aceptada → Conductor en camino → Llegué al origen → Pasajero a bordo → Viaje en curso → Viaje finalizado.

La guía contempla estados como:

- `requested`
- `accepted`
- `driver_arriving`
- `in_progress`
- `completed`
- `paid`
- `cancelled`

- `rejected`

El proyecto ya utiliza `pending` como equivalente de `requested`.

Conserva `pending` para evitar romper datos y reglas existentes, documentando la equivalencia.

Analiza qué estados adicionales deben persistirse y cuáles pueden representarse mediante acciones o marcas de tiempo sin duplicar estados.

Incluye:

- Botón para indicar que el conductor llegó al punto de recogida.
- Botón "Pasajero a bordo" para iniciar el viaje.
- Ruta al origen antes de recoger al pasajero.
- Ruta al destino después de recogerlo.
- Finalización del viaje con desglose de tarifas.
- Recuperación de viajes activos al reabrir la aplicación.
- Prevención de transiciones inválidas o duplicadas.
- Compatibilidad con viajes anteriores.

No implementes todavía el cobro ni marques automáticamente los viajes como pagados. Eso corresponde al módulo 3.

8. Integración con tarifas

Reutiliza:

- `src/config/pricing.js`
- `calculateFare`
- El desglose de tarifas.
- Las reglas y validaciones del PR #25.

Asegúrate de que:

- La tarjeta muestre importes consistentes.
- La finalización guarde el desglose correcto.
- Las propinas no se alteren.
- No se dupliquen comisiones.
- Los viajes antiguos sigan funcionando.

No modifiques la fórmula de precios sin una razón técnica justificada.

9. Simulador de solicitudes de prueba — SIN APP DE PASAJERO

No desarrolles una aplicación de pasajero.

Crea una herramienta de desarrollo independiente que permita generar solicitudes de prueba compatibles con el modelo de Firestore.

Puede ser:

- Una plantilla JSON documentada para introducirla manualmente en Firebase Console.
- Un script local de desarrollo para generar viajes de prueba.

Prioriza una solución sencilla y segura.

Condiciones:

- Separada del código de producción.
- No accesible desde la aplicación del conductor.
- No debe modificar las reglas para permitir a cualquier conductor crear solicitudes.
- No debe requerir credenciales privadas dentro de la app.
- No debe crear viajes reales por accidente.
- Debe permitir probar las categorías Económico, XL y Premium.
- Debe incluir distancia, duración, categoría, origen, destino, tarifa estimada y método de pago.
- Debe respetar los campos, timestamps y estados esperados.
- Debe permitir comprobar aceptación, rechazo, expiración y finalización.

Si eliges un script, que sea seguro por defecto y utilice preferentemente el emulador de Firestore.
No debe escribir en Firebase real sin autorización explícita.

Documenta cómo usarlo.

10. Seguridad de Firestore

Revisa y modifica las reglas locales cuando sea necesario.

Debes impedir:

- Aceptaciones duplicadas.
- Aceptación de solicitudes vencidas.
- Cambios de conductor no autorizados.
- Alteraciones de tarifa, distancia, duración, categoría o propina por parte del conductor.
- Transiciones de estados inválidas.
- Rechazos de solicitudes ajenas.
- Modificaciones posteriores a la finalización del viaje.
- Cambios arbitrarios de estados de pago.

Conserva las validaciones financieras implementadas en el módulo anterior.

No publiques las reglas.

Si alguna garantía exige un backend o una autoridad externa, documenta esa limitación sin simular que las reglas por sí solas la resuelven.

11. Pruebas obligatorias

Agrega o actualiza pruebas para:

Solicitudes
- Suscripción y limpieza.
- Tarjeta con todos los datos.
- Compatibilidad con documentos antiguos.
- Rechazo manual.
- Rechazo automático.
- Aceptación concurrente.
- Solicitud vencida.
- Conductor desconectado.

Temporizador
- 15 segundos exactos.
- Límites temporales.

- Reanudación después de segundo plano.
- No reinicio accidental.
- Limpieza de intervalos.

Ciclo del viaje
- Aceptación.
- Conductor en camino.
- Llegada al origen.
- Pasajero a bordo.
- Finalización.
- Recuperación de viaje activo.
- Transiciones inválidas.
- Operaciones repetidas.
- Compatibilidad con viajes antiguos.

Seguridad
- Reglas Firestore.
- Restricciones financieras.
- Rechazos no autorizados.
- Aceptación de solicitudes vencidas.
- Intentos de alterar datos protegidos.

Todas las pruebas anteriores deben seguir pasando.

Ejecuta:

- `npm run test:unit`
- `npm run test:integration`
- `npm run test:rules` con emulador.
- `npm run test:all` si el entorno está disponible.
- `npx expo-doctor`
- `npx expo export --platform ios`
- `git diff --check`

No ocultes fallos ni elimines pruebas para hacer que el conjunto pase.

12. Documentación y trazabilidad

Actualiza:

- `docs/AI_LOG.md` con la entrada #28.
- README con el modelo de solicitudes y la forma de probarlo.
- Comentarios `AI-assisted: see docs/AI_LOG.md #28` en el código intervenido.

Incluye el prompt literal cuando esté disponible y deja el hash del futuro commit pendiente para que lo registre el estudiante.

13. Entrega final

Cuando termines, presenta:

1. Resumen funcional de todo lo implementado.
2. Archivos creados y modificados.
3. Modelo de datos de Firestore.
4. Explicación de estados y transiciones.
5. Estrategia de rechazo y expiración.

```
6. Simulador o plantilla de solicitudes de prueba.
7. Resultados exactos de las pruebas.
8. Cambios locales en reglas Firestore.
9. Riesgos y limitaciones pendientes.
10. Instrucciones para probar en Expo Go.
11. Comandos de revisión Git, sin ejecutarlos para publicar.
12. Confirmación de que no hiciste commits, push, PR, merge ni despliegues.

No avances al módulo de pagos.

El objetivo es entregar un módulo de solicitudes y ciclo de viaje realmente funcional, seguro,
integrado y probado, no solamente una interfaz visual.
```

- **Resumen de la respuesta:** Se implementó un modelo de solicitudes compartidas con una oferta por conductor (`drivers/{uid}/tripOffers/{tripId}`). La oferta guarda `offeredAt` con la hora del servidor y de ella se cuentan los 15 s, de modo que el plazo es persistente y las reglas lo verifican con `request.time`. Se añadieron el estado `driver_arriving` y la marca `arrivedAt` ("Llegué al origen"), el hook `useTripRequests` y la tarjeta con todos los datos, que usa el desglose del módulo de tarifas. También se creó un simulador de solicitudes limitado al emulador.
- **Qué se usó / modificó / descartó:**
  - **Modelo:** las solicitudes son compartidas entre los conductores habilitados. El rechazo manual o por vencimiento se persiste en la oferta del conductor (`status: "rejected"`, `reason`, `respondedAt`) y no cancela la solicitud para los demás. Se descartó un estado `rejected` global, que requeriría una autoridad (backend) para decidir que nadie la aceptó.
  - **Estados:** se conservó `pending` como equivalente de `requested`. Se persistió `driver_arriving` (estado de la guía) y la llegada se registró como marca de tiempo (`arrivedAt`), sin crear un estado nuevo. Se eliminó la transición directa `accepted → in_progress`: "Pasajero a bordo" exige haber llegado al origen. `paid` queda para el módulo de pagos. `paymentMethod` (`cash`, `mercadopago`, `stripe`) queda como contrato de datos, fijado al crear la solicitud y protegido frente al conductor.
  - **Temporizador:** 15 s desde `offeredAt`. Mientras el servidor no confirma la marca, la tarjeta muestra "Preparando solicitud..." y no permite aceptar. El rechazo automático espera 1 s adicional para que la hora del servidor también haya superado el plazo, y al volver del segundo plano se recalcula de inmediato.
  - **Controlador:** la lógica de suscripciones, ofertas, temporizador y rechazo automático se separó de React en `createTripRequestFeed` para poder probar la suscripción, la limpieza y el intervalo en Node.
  - **Consulta de solicitudes:** se ordena por `createdAt` descendente para que las solicitudes antiguas que siguen en `pending` no ocupen los 20 resultados. Requiere un índice compuesto nuevo y que los viajes tengan `createdAt` de tipo timestamp.
  - **Corrección en las reglas:** las pruebas mostraron que el motor de reglas no cortocircuita `&&` ni `||`, de modo que cada actualización evaluaba la regla de aceptación y las cinco ramas del ciclo de vida, y se alcanzaba el límite de 1000 expresiones por petición. Se despachó con el operador `?:`, que solo evalúa la rama elegida, sin cambiar ninguna condición. *Corrección del informe de la entrada #27:* aquella ejecución ya registraba 34 avisos de este límite en las pruebas negativas, que la búsqueda usada entonces no detectó. Tras esta corrección quedan 0.
  - **Simulador:** `scripts/dev/simulateTripRequests.cjs` (`create`, `list`, `clear`, `demo`) solo usa el emulador, exige un proyecto `demo-*` y no requiere credenciales. Para Expo Go se añadió la plantilla `scripts/dev/tripRequestTemplate.json`.
  - No se instalaron dependencias.

- **Archivos creados:** `src/hooks/useTripRequests.js`, `src/services/tripOfferService.js`, `src/services/tripRequestFeed.js`, `src/utils/tripRequestLogic.js`, `scripts/dev/simulateTripRequests.cjs`, `scripts/dev/tripRequestTemplate.json` y `tests/tripRequests.test.cjs`.
- **Archivos modificados:** `src/components/trips/TripRequests.js`, `src/services/tripService.js`, `src/services/tripAcceptance.js`, `src/services/tripLifecycle.js`, `src/utils/tripLogic.js`, `src/utils/tripRequestTimer.js`, `src/utils/activeTripLogic.js`, `src/utils/tripMapLogic.js`, `firestore.rules`, `firestore.indexes.json`, `tests/trips.rules.test.cjs`, `tests/tripAcceptance.test.cjs`, `tests/tripCycle.integration.test.cjs`, `tests/tripRequestTimer.test.cjs`, `tests/tripLogic.test.cjs`, `tests/activeTripService.test.cjs`, `package.json` (script `test:unit`), `README.md` y `docs/AI_LOG.md`.
- **Commit:** `b7cfcba` (`feat: implementar solicitudes y ciclo completo de viajes`), rama `feat/solicitudes-ciclo-viaje`. Integrado en `main` mediante el Pull Request #26, merge `e33a87f`. *Corrección (entrada #29):* el commit `4637669` había insertado esta línea dentro del texto literal del prompt; se trasladó a este campo sin otros cambios.
- **Verificación:**
  - `npm run test:unit`: 122/122. `npm run test:integration`: 24/24. `npm run test:rules` con el emulador: 121/121. `npm run test:all`: **267 aprobadas y 0 fallidas**.
  - 0 avisos del límite de 1000 expresiones en todas las salidas.
  - Pruebas existentes actualizadas por cambio de requisito: el temporizador pasa de 20 a 15 s, `canStartTrip` ya no acepta `accepted`, la consulta de recuperación incluye `driver_arriving` y las aceptaciones requieren oferta. Ninguna prueba se eliminó.
  - `npx expo-doctor`: 20/20. `npx expo export --platform ios`: bundle generado (992 módulos). `git diff --check` sin observaciones.
  - El simulador se ejecutó contra el emulador (`create`, `list`, `demo`, `clear`) y rechazó un proyecto que no empieza por `demo-`.
  - No se probó en Expo Go: las reglas y el índice nuevos no se han publicado.
- **Limitaciones:**
  - Sin backend, el plazo de 15 s comienza cuando el cliente del conductor registra la oferta; una aplicación modificada podría retrasar ese registro. Un despachador en el servidor que asigne las ofertas lo resolvería.
  - Una solicitud que ningún conductor acepta permanece en `pending`.
  - La cuenta regresiva usa el reloj del dispositivo frente a `offeredAt`; un desfase del reloj altera lo que se muestra, aunque las reglas deciden con la hora del servidor.
  - Las solicitudes sin `createdAt` de tipo timestamp no aparecen en la lista.
  - Al publicar, una versión anterior de la app no podrá aceptar viajes ni iniciar un viaje desde `accepted`.
- **Aprendizaje:** En las reglas de Firestore, `&&` y `||` no evitan evaluar el resto de la expresión, así que las reglas extensas deben despacharse con `?:` para no superar el límite de expresiones. Al verificar la ausencia de un aviso conviene buscar el texto exacto que produce la herramienta. Separar la lógica de suscripciones y temporizadores de React permite probar su limpieza sin un entorno de interfaz.

Entrada #29

- **Fecha:** 2026-11-13
- **Integrante:** Julian Cardona
- **Herramienta:** Claude Code (Anthropic), modelo Claude Opus 5.5. La revisión de los servicios financieros y las reglas, que originó los ajustes de completitud, la realizó el integrante con ChatGPT (OpenAI).

- **Objetivo:** Implementar el módulo 3: registro de pagos en efectivo, comisión del 20 % adeudada a la plataforma y saldo del conductor, con reglas de Firestore, pruebas y compatibilidad con el futuro módulo de Mercado Pago.
- **Prompt (texto literal):**

```
# PROMPT MAESTRO — MÓDULO 3: PAGOS EN EFECTIVO, COMISIONES Y SALDO DEL CONDUCTOR

## 1. ROL Y OBJETIVO

Actúa como un ingeniero de software senior especializado en React Native, Expo, JavaScript,
Firebase Authentication, Cloud Firestore, arquitectura de aplicaciones móviles y pruebas
automatizadas.

Estás trabajando en **UberClone Driver**, una aplicación universitaria para conductores de una
plataforma de transporte.

Tu objetivo es **implementar de manera integral el módulo 3: pagos en efectivo, comisión de la
plataforma y saldo pendiente del conductor**, reutilizando la arquitectura existente y manteniendo
compatibilidad con todas las funcionalidades ya desarrolladas.

No debes comenzar el proyecto desde cero ni reemplazar componentes funcionales innecesariamente.

## 2. CONTEXTO ACTUAL DEL PROYECTO

**Repositorio:** [https://github.com/julianch07/uberclone-driver]
(https://github.com/julianch07/uberclone-driver)

**Rama actual:** `feat/pagos-efectivo-saldo-conductor`

**Último PR fusionado:** #26 — Solicitudes y ciclo completo de viajes.

**Tecnologías:**

- React Native con Expo SDK 58.
- JavaScript, sin TypeScript.
- Firebase Authentication.
- Cloud Firestore.
- Cloudinary para fotografías.
- Arquitectura basada en componentes, hooks, servicios y utilidades.

**Funcionalidades existentes:**

- Registro de conductores en tres secciones.
- Persistencia de información en Firestore.
- Estado de aprobación del conductor.
- Navegación principal.
- Disponibilidad y ubicación.
- Solicitudes de viajes.
- Aceptación y rechazo de solicitudes.
- Ciclo de viaje hasta su finalización.
- Cálculo de tarifas y comisión de plataforma del 20%.
- Pantallas de ganancias, historial y perfil.

El módulo 2 incorporó ofertas individuales de viajes, vencimiento de 15 segundos, aceptación
mediante transacciones, recuperación de viajes activos y pruebas automatizadas.
```

Importante: inspecciona el código actual y verifica qué funcionalidades están realmente implementadas antes de asumir su comportamiento.

3. REGLAS OBLIGATORIAS

1. Lee `AGENTS.md`, `README.md`, `package.json`, `docs/AI_LOG.md` y todos los archivos relevantes antes de modificar código.
2. Trabaja exclusivamente sobre la rama `feat/pagos-efectivo-saldo-conductor`.
3. No ejecutes `git add`, `git commit`, `git push`, `git merge` ni crees Pull Requests.
4. No modifiques `.env` ni expongas credenciales.
5. No despliegues reglas, índices ni servicios de Firebase.
6. No modifiques configuraciones de producción sin autorización.
7. No reescribas funcionalidades existentes innecesariamente.
8. No instales dependencias nuevas salvo necesidad técnica justificada.
9. Conserva compatibilidad con JavaScript, Expo 58 y las dependencias actuales.
10. Respeta los modelos y estados de viaje ya implementados.
11. No elimines pruebas existentes ni reduzcas sus verificaciones.
12. Documenta la asistencia de IA siguiendo `docs/AI_LOG.md`.
13. Añade comentarios `// AI-assisted: see docs/AI_LOG.md #N` en los archivos correspondientes, usando la siguiente entrada disponible.
14. No añadas atribuciones de Claude como coautor de commits.
15. No simules operaciones financieras reales que no tengan un respaldo técnico verificable.

4. FUNCIONALIDAD A — PAGOS EN EFECTIVO

Implementa el manejo de viajes completados cuyo método de pago sea efectivo.

Requisitos:

- Identificar correctamente el método de pago registrado en el viaje.
- Registrar que el conductor recibió el efectivo directamente del pasajero.
- Asociar el cobro con el viaje y el conductor correspondiente.
- Impedir cobros duplicados.
- Manejar reintentos y doble pulsación.
- Mostrar un resumen del importe recibido.
- Conservar trazabilidad del registro.

Diferencia claramente entre:

- Dinero recibido directamente por el conductor.
- Comisión que corresponde a la plataforma.
- Saldo pendiente del conductor con la plataforma.

El registro de recepción de efectivo no debe presentarse como una verificación bancaria independiente.

5. FUNCIONALIDAD B — COMISIÓN DEL 20%

La plataforma UberClone cobra una comisión del **20% sobre el importe correspondiente del viaje**, según el modelo de tarifas existente.

Debes:

1. Inspeccionar el cálculo actual de tarifas y comisiones.
2. Reutilizarlo como fuente de verdad.
3. Evitar cálculos contradictorios entre pantallas y servicios.

4. Registrar la comisión correspondiente a cada viaje en efectivo.
5. Garantizar importes consistentes en pesos colombianos.
6. Definir una política explícita de redondeo.
7. Evitar registros duplicados de comisiones.
8. Mantener los importes históricos estables después de finalizar el viaje.

Ejemplo ilustrativo, sujeto al modelo de tarifas existente:

Concepto	Valor
Tarifa del viaje	$20.000 COP
Comisión de plataforma (20%)	$4.000 COP
Efectivo recibido por el conductor	$20.000 COP
Comisión adeudada a la plataforma	$4.000 COP
Ganancia neta del conductor	$16.000 COP

No confundas la ganancia neta con el efectivo físicamente recibido.

6. FUNCIONALIDAD C — SALDO DEL CONDUCTOR

Implementa la consulta del saldo relacionado con los viajes cobrados en efectivo.

La interfaz debe permitir visualizar, cuando los datos existentes lo soporten:

- Total cobrado en efectivo.
- Total de comisiones generadas.
- Comisión pendiente de pago a la plataforma.
- Ganancia neta acumulada.
- Historial de comisiones por viaje.
- Estados de carga, error y ausencia de registros.

Diseña un modelo de datos que permita incorporar posteriormente pagos o conciliaciones de comisiones.

Restricción importante: no implementes un botón que permita al conductor marcar unilateralmente su deuda como pagada.

Si no existe un backend administrativo seguro, conserva las comisiones como pendientes y documenta cómo podrían conciliarse posteriormente.

7. FUNCIONALIDAD D — SEGURIDAD E INTEGRIDAD DE FIRESTORE

La integridad de los registros financieros es prioritaria.

Implementa o adapta los mecanismos necesarios para:

- Garantizar que un mismo viaje no genere múltiples comisiones.
- Evitar duplicaciones por reintentos.
- Validar la identidad del conductor.
- Respetar las reglas de acceso a Firestore.
- Impedir modificaciones indebidas de registros históricos.
- Impedir que un conductor modifique arbitrariamente importes o estados financieros.
- Conservar consistencia entre viajes completados y registros de cobro.
- Utilizar transacciones o escrituras atómicas cuando corresponda.

Advertencia de arquitectura: las transacciones de Firestore ejecutadas desde el cliente no

constituyen por sí solas una garantía contra clientes maliciosos.

Evalúa si las reglas de seguridad pueden validar las operaciones necesarias. Si alguna funcionalidad requiere lógica privilegiada de servidor, documenta esa limitación y plantea una solución segura.

No introduzcas una falsa sensación de seguridad.

8. FUNCIONALIDAD E — INTERFAZ DE USUARIO

Integra las funcionalidades dentro de las pantallas y componentes existentes.

Prioriza reutilizar:

- Pantalla de viaje activo o finalizado.
- Resumen de viaje.
- Pantalla de ganancias.
- Historial de viajes.
- Componentes visuales ya disponibles.

La interfaz debe:

- Mostrar valores monetarios en COP.
- Diferenciar claramente cobro bruto, comisión y ganancia neta.
- Evitar acciones duplicadas.
- Mostrar estados de procesamiento.
- Informar errores de forma comprensible.
- Mantener coherencia visual con UberClone Driver.
- Funcionar correctamente en Expo Go.

No construyas pantallas nuevas si la funcionalidad puede integrarse adecuadamente en las existentes.

9. FUNCIONALIDAD F — COMPATIBILIDAD CON MERCADO PAGO

El próximo módulo integrará Mercado Pago mediante un flujo de pago digital.

Por eso:

- Mantén separados los flujos de efectivo y pago digital.
- No registres pagos digitales como efectivo.
- No generes automáticamente una deuda por comisión de efectivo para un viaje pagado digitalmente.
- Conserva un modelo de datos extensible.
- No implementes todavía Checkout Pro ni credenciales de Mercado Pago.
- No simules confirmaciones de pago digital.

El módulo 3 debe quedar preparado para coexistir con el módulo 4.

10. PRUEBAS OBLIGATORIAS

Agrega pruebas automatizadas para los casos que correspondan:

Pruebas unitarias:

- Cálculo y redondeo de comisiones.
- Importes en COP.

- Separación de valores brutos y netos.
- Cálculo de saldo pendiente.

Pruebas de integración:

- Registro único de un cobro.
- Reintentos.
- Doble pulsación.
- Operaciones concurrentes.
- Persistencia y recuperación.
- Viajes completados y no completados.
- Separación entre efectivo y pago digital.

Pruebas de seguridad:

- Acceso exclusivo del conductor autorizado.
- Protección de importes.
- Prevención de cambios indebidos.
- Restricciones sobre estados financieros.

Al finalizar, ejecuta:

```bash
npm run test:all
npx expo-doctor
npx expo export --platform ios
```

Si alguna verificación falla, investiga y corrige el problema cuando pertenezca al alcance del módulo.

No afirmes que una prueba fue aprobada si no se ejecutó realmente.

11. DOCUMENTACIÓN

Actualiza:

- `README.md`, con las funcionalidades nuevas y sus limitaciones.
- `docs/AI_LOG.md`, con una nueva entrada siguiendo exactamente la estructura existente.

La entrada debe incluir la herramienta utilizada, objetivo, prompt o resumen fiel, decisiones, archivos modificados, verificaciones y resultados.

Deja el hash del commit pendiente: lo añadiremos posteriormente cuando creemos el commit con Git.

No modifiques entradas históricas sin necesidad.

12. METODOLOGÍA DE IMPLEMENTACIÓN

Trabaja en las siguientes fases:

Fase 1 — Auditoría técnica

Inspecciona el repositorio, los servicios, las pantallas, las reglas de Firestore y los modelos de viaje.

Presenta brevemente:

- Qué funcionalidades pueden reutilizarse.
- Qué archivos necesitan cambios.
- Qué archivos nuevos serían necesarios.
- Cómo almacenarás los cobros y las comisiones.
- Qué riesgos de seguridad identificaste.

Fase 2 — Implementación

Desarrolla los servicios, validaciones, reglas, componentes y hooks necesarios.

Integra las funcionalidades en la aplicación.

Fase 3 — Pruebas

Ejecuta las verificaciones y corrige los fallos relacionados con el módulo.

Fase 4 — Documentación

Actualiza README y bitácora de IA.

Fase 5 — Informe final

Entrega un resumen que incluya:

1. Funcionalidades implementadas.
2. Archivos creados.
3. Archivos modificados.
4. Modelo de datos utilizado.
5. Medidas de seguridad implementadas.
6. Pruebas ejecutadas y resultados.
7. Limitaciones y riesgos pendientes.
8. Recomendaciones para el módulo 4.

13. CRITERIO FINAL DE ACEPTACIÓN

El módulo estará listo para revisión cuando:

- Un viaje completado en efectivo pueda registrar su cobro.
- El registro sea idempotente y no genere duplicados.
- Se calcule y conserve la comisión del 20%.
- Se pueda consultar el saldo pendiente del conductor.
- Los importes sean consistentes entre viajes, ganancias e historial.
- Las reglas de Firestore protejan las operaciones dentro de las garantías que realmente puedan ofrecer.
- Las pruebas correspondientes se ejecuten satisfactoriamente.
- La documentación esté actualizada.
- No se hayan creado commits ni realizado despliegues.

Prioridad máxima: calidad, compatibilidad con el proyecto existente, integridad financiera y seguridad.

No sacrifiques estas condiciones por implementar más funcionalidades.

Comienza inspeccionando el repositorio y presenta el diagnóstico técnico antes de realizar cambios

```
importantes. Claude,  tu realizarás la implementación local. Cuando termine, me pasas su informe y
los resultados de las pruebas. Yo revisaré la coherencia con el proyecto, los posibles riesgos de
Firestore y el cumplimiento de lo solicitado antes de hacer los commits.
```

- **Revisión con ChatGPT (resumen; el mensaje la incluía antes del prompt de ajustes):** auditoría favorable de la seguridad de los pagos en efectivo, con dos ajustes recomendados: el saldo se calculaba con un máximo de 100 asientos y los cobros pendientes con un máximo de 20 viajes completados filtrados en el cliente.
- **Prompt de ajustes (texto literal):**

```
# AJUSTES FINALES — MÓDULO 3 UBERCLONE DRIVER

He revisado junto con ChatGPT los servicios financieros y las reglas de Firestore del módulo 3.

La auditoría es favorable. No debes reescribir el módulo ni modificar funcionalidades que ya
funcionan.

Antes de crear los commits necesitamos resolver dos problemas de completitud de datos.

## 1. Saldo limitado a 100 movimientos

Actualmente `cashPaymentService.js` consulta los últimos 100 asientos contables y
`summarizeDriverBalance()` calcula el saldo a partir de ellos.

Esto puede presentar una deuda incorrecta cuando existan más de 100 movimientos.

**Objetivo:** garantizar que el saldo presentado como total sea completo y correcto.

Evalúa la solución más adecuada para el proyecto:

- Paginación completa de asientos para el cálculo del saldo.
- Agregados financieros mantenidos de forma segura.
- Otra alternativa que conserve integridad y compatibilidad con Firestore.

No utilices agregados actualizables libremente por el conductor.

Si implementas paginación, diferencia entre la consulta limitada para mostrar el historial y la
consulta completa necesaria para calcular el saldo.

No presentes datos parciales como si fueran totales.

## 2. Cobros pendientes limitados a 20 viajes

Actualmente se consultan hasta 20 viajes completados y después se filtran los pagos en efectivo.

Esto puede ocultar viajes pendientes de cobro.

**Objetivo:** permitir recuperar todos los viajes en efectivo pendientes, sin introducir consultas
incompatibles con las reglas de Firestore.

Evalúa una consulta específica por método de pago y estado, junto con paginación cuando sea
necesaria.

Comprueba si hacen falta índices compuestos y actualiza `firestore.indexes.json` únicamente cuando
corresponda.
```

```
No despliegues reglas ni índices.

## 3. Pruebas obligatorias

Añade pruebas para:

- Más de 100 asientos contables.
- Más de 20 viajes pendientes.
- Mezcla de viajes digitales y en efectivo.
- Cálculo correcto de la deuda histórica.
- Errores de paginación o lecturas incompletas.
- Compatibilidad con las reglas existentes.

Ejecuta nuevamente:

- `npm run test:all`
- `npx expo-doctor`
- `npx expo export --platform ios`
- `git diff --check`

## 4. Restricciones

- Mantén la rama `feat/pagos-efectivo-saldo-conductor`.
- No ejecutes `git add`, `git commit`, `git push` ni crees PR.
- No modifiques `.env`.
- No despliegues Firebase.
- No instales dependencias innecesarias.
- No alteres el flujo de Mercado Pago.
- Conserva el formato de `docs/AI_LOG.md` y actualiza la entrada #29.
- Mantén los comentarios de asistencia de IA.

## 5. Entrega

Implementa las correcciones y entrégame un informe con los archivos modificados, la solución
escogida, los resultados reales de las pruebas y los riesgos pendientes.

No hagas commits. ChatGPT y yo realizaremos la revisión final y el control de versiones.
```

- **Resumen de la respuesta:** Tras una auditoría, se implementó el cobro en efectivo como una transacción que pasa el viaje de `completed` a `paid` (`paymentStatus: "cash"`) y crea el asiento `drivers/{uid}/ledgerEntries/cash_{tripId}`, con los importes copiados del desglose del módulo de tarifas. Las reglas exigen ambas escrituras juntas y con los mismos importes, e impiden duplicarlas, modificarlas o liquidarlas. Se añadieron una tarjeta de cobro en la pestaña Viajes y el saldo real en la pantalla Pagos. En los ajustes, el saldo pasó a calcularse con agregaciones de Firestore sobre todos los asientos y los cobros pendientes con una consulta filtrada en el servidor y paginada.
- **Qué se usó / modificó / descartó:**
  - **Fuente de verdad:** se reutilizó el desglose guardado al finalizar (#27): `fare`, `commission` con redondeo a la mitad hacia arriba en pesos enteros, `commissionRate`, `driverEarnings` y `tip`. El cobro no recalcula importes, por lo que el historial permanece estable.
  - **Modelo:** el asiento distingue `cashAmount` (efectivo recibido: tarifa + propina), `commission`/`amount` (deuda con la plataforma) y `driverEarnings` (ganancia neta). El ID determinista `cash_{tripId}` garantiza una

sola comisión por viaje. El tipo `cash_commission` y el estado `pending`/`settled` dejan el modelo abierto a liquidaciones y pagos digitales.
    - **Seguridad:** las reglas validan identidad, método `cash`, estado `completed`, ausencia de pago previo, importes idénticos al viaje y la atomicidad con `existsAfter`/`getAfter`. Se documentó que el registro es una declaración del conductor y que la liquidación de comisiones requiere un backend con credenciales de servidor. No se implementó ningún botón para marcar la deuda como pagada.
    - **Separación digital/efectivo:** los viajes `mercadopago`/`stripe` no pueden registrarse como efectivo ni pasar a `paid` desde la app; se muestran como "Pago digital pendiente". Los estados `approved`, `pending` y `rejected` quedan reservados para la pasarela.
    - **Estado `paid`:** se añadió a `TRIP_STATUS` y se incluyó en la consulta y la regla del historial y en los filtros de Ganancias y Pagos, para que los viajes cobrados no desaparezcan.
    - **Interfaz:**
        - En `PaymentSummaryScreen` se reemplazaron los textos fijos sin implementación ("Próximo pago estimado", "Fecha: Lunes próximo", "Los pagos se procesan semanalmente…") por el saldo real y la lista de comisiones por viaje.
        - Los cobros por confirmar se muestran en la pestaña Viajes, porque `DriverContext` deja de mostrar el viaje al finalizarlo.
    - **Hallazgos de las pruebas:**
        - Con el saldo vacío el resultado era `-0`; se corrigió en el código.
        - La primera instantánea de `onSnapshot` puede venir de la caché local; la prueba espera a la condición.
        - Leer un viaje inexistente o ajeno ya es rechazado por las reglas (`permission-denied`), por lo que el cobro tiene su propio mensaje de error.
    - **Ajustes de completitud (tras la revisión con ChatGPT):**
        - **Saldo:** los totales los calcula Firestore en el servidor sobre todos los asientos (`getAggregateFromServer` con `count` y `sum`), sujeto a las reglas de lectura del propietario. Se descartaron los agregados escritos por el cliente, que serían manipulables, y la paginación completa en el cliente, que es más costosa e igual de exacta. La lista visible sigue limitada a los 100 asientos más recientes e indica cuando hay más. Los totales se recalculan con cada cambio y, si el cálculo falla, la pantalla muestra un error y ningún saldo.
        - **Cobros pendientes:** se filtran en el servidor por conductor, `completed` y `cash`, y se leen todas las páginas de 50 (el límite de la regla) con `startAfter`. La primera página se escucha en tiempo real y cada cambio relee la lista completa. Si falla una página, no se entrega una lista parcial.
        - **Índices:** las consultas nuevas solo usan filtros de igualdad, que Firestore resuelve combinando índices de un solo campo; no se modificó `firestore.indexes.json`.
    - **Simulador:** el comando `demo` registra también el cobro en efectivo.
    - **Corrección documental:** el commit `4637669` había insertado el hash de la entrada #28 dentro del texto literal de su prompt; se trasladó a su campo "Commit".
    - No se instalaron dependencias ni se desplegó nada.
- **Archivos creados:** `src/utils/cashPaymentLogic.js`, `src/services/cashPaymentService.js`, `src/services/firestorePagination.js`, `tests/pagination.test.cjs`, `src/hooks/useDriverPayments.js`, `src/components/payments/PendingCashPayments.js`, `tests/cashPayment.test.cjs`, `tests/cashPayment.integration.test.cjs` y `tests/payments.rules.test.cjs`.
- **Archivos modificados:** `src/utils/tripLogic.js`, `src/services/tripService.js`, `src/components/trips/TripRequests.js`, `src/screens/home/PaymentSummaryScreen.js`, `src/screens/home/EarningsScreen.js`, `src/screens/home/TripHistoryScreen.js`, `firestore.rules`,

`scripts/dev/simulateTripRequests.cjs`, `tests/activeTripService.test.cjs`, `package.json` (scripts de pruebas), `README.md` y `docs/AI_LOG.md`.

- **Commit:** `bfa3197` (`feat: implementar pagos en efectivo y saldo del conductor`), rama `feat/pagos-efectivo-saldo-conductor`.
- **Verificación:**
  - `npm run test:all` con el emulador local: 139 unitarias, 37 de integración y 150 de reglas, **326 aprobadas y 0 fallidas**.
  - Pruebas nuevas del módulo: 17 unitarias, 13 de integración y 29 de reglas. Los ajustes añadieron 130 asientos (con liquidados mezclados), 65 cobros pendientes, 30 viajes digitales, 10 pagados y 3 antiguos; se comprobó que los totales del servidor coinciden con la suma de todos los asientos y difieren de la suma de los 100 más recientes. También se probaron las lecturas sin permiso, sin datos parciales.
  - Las pruebas de integración se ejecutaron varias veces seguidas sin fallos.
  - Ningún aviso "maximum of 1000 expressions" en las salidas (búsqueda del texto exacto del emulador).
  - Una prueba existente se actualizó por cambio de requisito: la consulta del historial incluye `paid`.
  - `npx expo-doctor`: 20/20. `npx expo export --platform ios`: bundle generado (996 módulos). `git diff --check` sin observaciones.
  - No se probó en Expo Go ni contra Firebase de producción: las reglas nuevas no se han publicado.
- **Limitaciones:**
  - La confirmación del efectivo depende de la declaración del conductor.
  - Las comisiones permanecen `pending` hasta que exista un backend administrativo.
  - Los viajes finalizados antes del módulo de tarifas, o sin `paymentMethod`, no pueden registrar cobro.
  - Al publicar las reglas, la app y las reglas deben actualizarse juntas.
  - Las agregaciones no son en tiempo real: se recalculan cada vez que cambia la lista de asientos recientes.
  - El emulador no verifica índices. Si en producción Firestore pidiera un índice compuesto para las consultas de igualdad, el error incluye el enlace para crearlo.
- **Aprendizaje:** Las reglas de Firestore pueden garantizar la coherencia entre dos documentos escritos en la misma operación (`existsAfter`/`getAfter`) y la inmutabilidad de un registro, pero no la veracidad de un hecho del mundo real como la entrega del efectivo. Copiar los importes ya validados, en lugar de recalcularlos, mantiene los registros financieros consistentes entre pantallas y en el tiempo.

Entrada #30

- **Fecha:** 2026-11-13
- **Integrante:** Julian Cardona
- **Herramienta:** Claude Code (Anthropic), modelo Claude Opus 5.5
- **Objetivo:** Diagnosticar el error `The query requires an index` en la consulta de solicitudes pendientes y revisar si otras consultas de la app requieren índices compuestos.
- **Prompt (texto literal):**

```
# Claude Code — UberClone Driver: diagnóstico y corrección de índices Firestore

Actúa como ingeniero senior de React Native, Expo, Firebase y Firestore, especializado en
depuración, seguridad y pruebas automatizadas.

## Contexto
```

Proyecto: UberClone Driver.

Stack: Expo SDK 58, React Native, JavaScript, Firebase Authentication anónima, Cloud Firestore y Cloudinary.

Firebase Project ID: `uberclone-driver-8e0bf`.

El proyecto ya implementa registro del conductor, aprobación, pantalla Home, disponibilidad, ubicación, solicitudes de viajes, ciclo de viajes, tarifas, cobros en efectivo, ganancias, historial y perfil.

Las reglas locales de Firestore se probaron con `npm run test:rules`: **150 pruebas superadas, 0 fallidas y 0 omitidas**.

Posteriormente se ejecutó exitosamente:

`npx --yes --package firebase-tools firebase deploy --only firestore:rules --project uberclone-driver-8e0bf`

Las reglas se publicaron correctamente.

El error anterior `Missing or insufficient permissions` al recuperar viajes activos dejó de aparecer en los últimos registros, pero apareció este nuevo error:

```text
ERROR Error al escuchar viajes pendientes:
[FirebaseError: The query requires an index.]

ERROR Error en las solicitudes de viaje:
[FirebaseError: The query requires an index.]
```

Firebase solicita un índice compuesto en la colección `trips` con los campos:

- `status`: Ascending.
- `createdAt`: Descending.
- `__name__`: Descending (implícito en el enlace de Firebase).

Objetivo principal

Corregir la falta de índices compuestos de Firestore y detectar preventivamente otros índices requeridos por las consultas actuales, sin alterar la lógica funcional ni debilitar las reglas de seguridad.

Instrucciones obligatorias

1. Lee primero `AGENTS.md`, `package.json`, `firebase.json`, `firestore.indexes.json`, `firestore.rules`, los servicios de Firestore y los archivos de pruebas relevantes.
2. Revisa todas las consultas Firestore de la aplicación, especialmente las relacionadas con viajes pendientes, viajes activos, historial, ganancias y cobros.
3. Determina cuáles requieren índices compuestos y comprueba si ya están declarados.
4. Actualiza `firestore.indexes.json` únicamente con los índices necesarios, sin duplicados.
5. Verifica que los índices sean compatibles con las consultas reales. No modifiques consultas funcionales solo para evitar crear índices.
6. Ejecuta `npm run test:unit`, `npm run test:integration` y `npm run test:rules`. Si alguna prueba falla, investiga y corrige la causa dentro del alcance.

```
7. Ejecuta `npx expo-doctor` y `npx expo export --platform ios`, si el entorno lo permite.
8. No cambies `.env`, credenciales, configuración sensible ni el proyecto Firebase.
9. No modifiques las reglas de seguridad salvo que descubras un problema real y documentes su justificación. No amplíes permisos indiscriminadamente.
10. No hagas commits, push, merges ni despliegues automáticos.
11. Conserva la compatibilidad con Expo SDK 58, JavaScript y los documentos existentes en Firestore.
12. Documenta los cambios asistidos por IA en `docs/AI_LOG.md`, respetando la numeración consecutiva de `Entrada #N` y manteniendo `### Consideraciones generales` al final. No inventes resultados ni números de commits. Usa los comentarios `// AI-assisted: see docs/AI_LOG.md #N` cuando corresponda.
13. Trabaja sobre el repositorio existente. No reconstruyas el proyecto ni instales dependencias innecesarias.

## Entregables

Al finalizar, presenta:

- Causa raíz del error.
- Consultas inspeccionadas.
- Índices existentes y nuevos.
- Archivos modificados.
- Resultado de cada suite de pruebas.
- Riesgos o limitaciones pendientes.
- Comando exacto recomendado para desplegar exclusivamente los índices en el proyecto correcto.
- Procedimiento para comprobar que los índices estén en estado `Enabled` y verificar la aplicación.

**Importante:** Puedes realizar los cambios locales necesarios y ejecutar las pruebas, pero debes detenerte antes de cualquier despliegue a Firebase o modificación remota. Yo revisaré los resultados y autorizaré el despliegue.

Prioriza resolver el error actual de manera segura y rápida. Evita refactorizaciones ajenas al problema.
```

- **Resumen de la respuesta:**
  - **Causa raíz:** el índice compuesto `trips (status ASC, createdAt DESC)` que exige la consulta de solicitudes pendientes ya estaba declarado en `firestore.indexes.json` desde el commit `b7cfcba` (entrada #28), pero no estaba publicado. El despliegue se hizo con `--only firestore:rules`, que no publica índices.
  - **Comprobación de solo lectura:** `npx firebase-tools firestore:indexes --project uberclone-driver-8e0bf` mostró que en el proyecto solo existe `trips (driverId ASC, status ASC)`.
  - No fue necesario modificar `firestore.indexes.json`, las consultas ni las reglas.
- **Qué se usó / modificó / descartó:**
  - **Consultas revisadas (8):**
    1. Solicitudes pendientes: `status ==` + `orderBy(createdAt desc)`. Requiere el índice declarado y no publicado.
    2. Viaje activo: `driverId ==` + `status in`. Cubierta por el índice publicado.
    3. Historial: `driverId ==` + `status in`. Cubierta por el índice publicado.
    4. Cobros en efectivo pendientes: tres filtros de igualdad, paginada con `startAfter`.
    5. Ofertas: `orderBy(offeredAt desc)`. Índice automático de un solo campo.
    6. Asientos recientes: `orderBy(createdAt desc)`. Índice automático de un solo campo.

7. Total del saldo: `type ==` + `count` y tres `sum`.
8. Comisión pendiente: `type ==` + `status ==` + `sum`.

- **Documentación oficial consultada:**
  - Las consultas con varias igualdades pueden servirse combinando índices existentes (*index merging*).
  - Para agregaciones sobre varios campos, la documentación indica que la consulta "podría" requerir un índice compuesto y que Firestore lo sugiere en ese caso, sin especificar cuál.
- **Decisión:** no se añadieron índices especulativos para las consultas 4 y 7, porque el emulador no verifica índices y no puede comprobarse su composición exacta; quedaron documentadas como riesgo con un procedimiento de verificación. No se modificó ninguna consulta para evitar índices.
- **Corrección (registrada en la entrada #31):** la consulta 8 también debió quedar documentada como riesgo. En el informe se dio por cubierta mediante la combinación de índices existentes, pero la documentación oficial solo describe esa combinación para filtros de igualdad con un `orderBy` opcional, no para agregaciones. En el proyecto real, Firestore pidió un índice compuesto para esa consulta.

- **Archivos modificados:** `docs/AI_LOG.md`.
- **Commit:** Pendiente (lo registra el integrante).
- **Verificación:** `firestore.indexes.json` es JSON válido, con 2 índices y sin duplicados. `npm run test:unit`: 139/139. `npm run test:integration`: 37/37. `npm run test:rules`: 150/150 (0 omitidas). Ningún aviso "maximum of 1000 expressions". `npx expo-doctor`: 20/20. `npx expo export --platform ios`: bundle generado (997 módulos). No se desplegó nada: la publicación de los índices queda pendiente de la autorización del integrante.
- **Aprendizaje:** Un índice declarado en el repositorio no existe en Firebase hasta que se despliega con `firestore:indexes`; desplegar solo las reglas no lo publica. El emulador de Firestore no valida índices, por lo que esa comprobación solo puede hacerse contra el proyecto real.

Entrada #31

- **Fecha:** 2026-11-13
- **Integrante:** Julian Cardona
- **Herramienta:** Claude Code (Anthropic), modelo Claude Opus 5.5
- **Objetivo:** Diagnosticar el rechazo `permission-denied` al pasar un viaje pendiente a `in_progress` y los errores de índice de las agregaciones del saldo del conductor; aplicar solo las correcciones locales confirmadas.
- **Prompts (texto literal):**

Primer prompt (auditoría de solo lectura):

```
Realiza una auditoría SOLO LECTURA de UberClone Driver. No modifiques archivos, no hagas commits ni despliegues.

Problema 1: Firestore rechaza con permission-denied la actualización de trips/viaje_prueba_001 que cambia status a in_progress y startedAt a serverTimestamp(). Antes, el documento tenía status=pending y driverId=null. Revisa firestore.rules, las transiciones de estados, el servicio de viajes, la asignación de driverId y el flujo de aceptar/iniciar viaje. Determina la causa exacta y qué datos o pasos se necesitan para una prueba válida. No propongas relajar reglas sin justificación.

Problema 2: consultas de drivers/{uid}/ledgerEntries fallan. Una consulta filtra type=cash_commission y status=pending, agregando sum(commission), y solicita un índice. Otra filtra
```

```
type=cash_commission y agrega count y sum(cashAmount, commission, driverEarnings), con failed-
precondition. Revisa firestore.indexes.json, los servicios de ganancias y el diseño de consultas.
Indica los índices concretos que faltan y si se requiere algún cambio de código.

Ejecuta únicamente pruebas locales disponibles, si son necesarias. Devuélveme diagnóstico con
archivos y líneas, plan mínimo de corrección y comandos de verificación. Conserva compatibilidad
con Expo SDK 58, JavaScript, Firebase y la bitácora docs/AI_LOG.md.
```

Segundo prompt (implementación):

```
Excelente diagnóstico. Ya tenemos el enlace oficial del primer índice de comisión pendiente:
https://console.firebase.google.com/v1/r/project/uberclone-driver-8e0bf/firestore/indexes?
create_composite=Clxwcm9qZWN0cy91YmVyY2xvbmUtZHJpdmVyLThlMGJmL2RhdGFiYXNlcy8oZGVmYXVsdCkvY29sbGxlY3
Rpb25Hcm91cHMvbGVkZ2VyRW50cmllcy9pbmRleGVzL18QARoKCgZzdGF0dXMQARoICgR0eXBlEAEaDgoKY29tbWlzc2lvbhABG
gwKCF9fbmFtZV9fEAE

Implementa únicamente las correcciones locales respaldadas por el diagnóstico: agrega el índice
confirmado de ledgerEntries a firestore.indexes.json, añade la prueba de regresión que rechaza
pending → in_progress, corrige la omisión en Entrada #30 de docs/AI_LOG.md y agrega Entrada #31 en
orden cronológico, antes de Consideraciones generales.

Para la segunda agregación, verifica si existe un enlace oficial de creación de índice o una forma
fiable de determinar su definición. No inventes el índice ni lo agregues si no está confirmado;
déjalo documentado como pendiente.

No cambies firestore.rules, servicios de viajes, lógica de pagos ni dependencias. No hagas commits,
push ni despliegues. Ejecuta las pruebas locales pertinentes y reporta archivos modificados,
resultados y cualquier pendiente. Conserva Expo SDK 58, JavaScript y los comentarios AI-assisted
cuando corresponda.
https://console.firebase.google.com/project/uberclone-driver-8e0bf/firestore/databases/-
default-/indexes?
create_composite=Clxwcm9qZWN0cy91YmVyY2xvbmUtZHJpdmVyLThlMGJmL2RhdGFiYXNlcy8oZGVmYXVsdCkvY29sbGVjdG
lvbkdyb3Vwcy9sZWRnZXJFbnRyaWVzL2luZGV4ZXMvXxABGgoKBnN0YXR1cxABGggKBHR5cGUQARoOCgpjb21taXNzaW9uEAEaD
AoIX19uYW1lX18QAQ
```

- **Resumen de la respuesta:**
  - **Problema 1, `pending → in_progress`:** el rechazo es el comportamiento esperado y no requiere cambiar las reglas.
    - La regla de actualización de `trips` se decide por el estado actual. Desde `pending` solo se evalúa la aceptación, que exige pasar a `accepted`, asignarse `driverId`, modificar únicamente `status` y `driverId`, y tener una oferta vigente de menos de 15 s.
    - El ciclo de vida exige además que el conductor asignado haga la petición y que el viaje esté en `driver_arriving` con `arrivedAt` para poder pasar a `in_progress`.
    - La app tampoco intenta ese salto: `canStartTrip` aplica las mismas condiciones.
    - Una prueba válida sigue el ciclo desde la app: crear la solicitud con la plantilla `scripts/dev/tripRequestTemplate.json` (con `paymentMethod`, que después ya no puede añadirse), y luego Aceptar en menos de 15 s, "Voy en camino", "Llegué al origen" y "Pasajero a bordo".
  - **Problema 2, agregaciones del saldo:** las dos consultas de `getDriverBalanceTotalsWithDatabase` combinan un filtro con `sum()`. `firestore.indexes.json` no declaraba ningún índice de `ledgerEntries`. No hace falta cambiar código.

    - **Índice de la comisión pendiente (confirmado):**
        - Se decodificó el parámetro `create_composite` del enlace oficial: `ledgerEntries`, ámbito de colección, `status ASC, type ASC, commission ASC`, y el `__name__ ASC` implícito, que no se declara.
        - De las dos copias del enlace, la primera estaba alterada (`colllectionGroups`, con la longitud del campo descuadrada). La segunda es válida. Ambas indican los mismos campos.
    - **Índice de los totales (pendiente):** no se añadió porque no está confirmado.
        - No hay ningún enlace oficial disponible para él.
        - La documentación oficial de agregaciones solo indica que una consulta con agregaciones sobre varios campos "might require a composite index" y que Firestore lo sugiere. No especifica qué campos incluye ni en qué orden.
        - El servicio lanza las dos agregaciones en paralelo con `Promise.all`, que solo propaga el primer rechazo. Eso explica que solo se viera el enlace de la comisión pendiente. Con ese índice habilitado, al abrir Ganancias → Pagos el registro "Error al consultar el saldo del conductor" de Metro mostrará el enlace de la consulta de totales.
- **Qué se usó / modificó / descartó:**
    - Se añadió a `firestore.indexes.json` el índice confirmado, con los campos en el mismo orden que el enlace.
    - Se añadió la prueba de regresión "Impide iniciar un viaje pendiente sin aceptarlo".
        - Usa un conductor habilitado con una oferta vigente y comprueba que se rechaza el paso a `in_progress`, también cuando la misma escritura se asigna el `driverId`.
        - Como control, comprueba que la aceptación normal sí se permite, de modo que el rechazo se debe al salto de estado.
    - Se corrigió la entrada #30: la consulta de comisión pendiente se había dado por cubierta mediante la combinación de índices existentes, sin quedar documentada como riesgo.
    - **Descartado:**
        - Deducir la definición del segundo índice a partir de la documentación.
        - Lanzar consultas contra el proyecto real con credenciales de administrador para obtener el enlace.
        - Cambiar las consultas para evitar índices.
        - Modificar `firestore.rules`, los servicios de viajes o la lógica de pagos.
- **Archivos modificados:** `firestore.indexes.json`, `tests/trips.rules.test.cjs` y `docs/AI_LOG.md`.
- **Commit:** 69bddc6 (fix: agregar indice de comisiones y prueba de regresion de viajes).
- **Verificación:**
    - **`firestore.indexes.json`:** JSON válido, con 3 índices y sin duplicados.
    - **Suites:**
        - `npm run test:unit`: 139/139.
        - `npm run test:integration`: 37/37.
        - `npm run test:rules`: 151/151 (0 omitidas). Ningún aviso "maximum of 1000 expressions".
    - **Prueba de mutación:** la prueba nueva falla contra una copia de las reglas que permite cualquier cambio desde `pending`, y pasa con las reglas reales. La copia se creó en un directorio temporal; `firestore.rules` no cambió.
    - **Sin ejecutar:** `npx expo-doctor` y `npx expo export`, porque no cambiaron ni el código de la app ni las dependencias.
    - **Sin desplegar:** no se desplegó nada.
- **Pendientes:**

    - Desplegar los índices con `--only firestore:indexes`, cuando el integrante lo autorice.
    - Obtener el enlace oficial del índice de los totales y añadirlo a `firestore.indexes.json`.
    - Verificar en el proyecto real la consulta paginada de cobros pendientes.
- **Aprendizaje:** El enlace `create_composite` que genera Firestore codifica la definición exacta del índice, por lo que puede decodificarse y declararse en el repositorio sin crearlo a mano en la consola. Cuando varias consultas se lanzan en paralelo con `Promise.all`, solo se ve el primer error, y los índices que faltan aparecen de uno en uno.

Entrada #32

- **Fecha:** 2026-11-13
- **Integrante:** Julian Cardona
- **Herramienta:** Claude Code (Anthropic), modelo Claude Opus 5.5
- **Objetivo:** Declarar el índice compuesto que Firebase pidió para la agregación de totales del saldo del conductor, pendiente desde la entrada #31.
- **Prompt (texto literal):**

```
Estamos en UberClone Driver, rama main, actualizada y limpia tras el merge anterior.

Firebase confirmó que falta un segundo índice compuesto para la agregación de totales de
ledgerEntries. Enlace oficial:
https://console.firebase.google.com/v1/r/project/uberclone-driver-8e0bf/firestore/indexes?
create_composite=Clxwcm9qZWN0cy91YmVyY2xvbmUtZHJpdmVyLThlMGJmL2RhdGFiYXNlcy8oZGVmYXVsdCkvY29sbGVjdG
lvbkdyb3Vwcy9sZWRnZXJFbnRyaWVzL2luZGV4ZXMvXxABGggKBHR5cGUQARoOCgpjYXNoQW1vdW50EAEaDgoKY29tbWlzc2lvb
hABGhIKDmRyaXZlckVhcm5pbmdzEAEaDAoIX19uYW1lX18QAQ

Realiza únicamente estos cambios locales:
1. Verifica los campos y orden del enlace oficial y añade el índice ledgerEntries (type ASC,
cashAmount ASC, commission ASC, driverEarnings ASC) a firestore.indexes.json. Conserva los tres
índices existentes sin duplicarlos.
2. Añade una nueva Entrada #32 en docs/AI_LOG.md documentando el error, la causa, el índice
añadido, las pruebas y el uso de Claude Code. Mantén todas las entradas numeradas en orden y la
sección Consideraciones generales al final.
3. Verifica que el JSON sea válido, ejecuta git diff --check y las pruebas locales pertinentes.
4. Informa los archivos modificados y resultados.

No modifiques firestore.rules, servicios, pantallas, dependencias ni pruebas salvo que detectes una
razón imprescindible, en cuyo caso informa antes de modificar. No hagas commits, push ni
despliegues. No modifiques .env. Conserva JavaScript y Expo SDK 58.
```

- **Resumen de la respuesta:**
    - **Error:** con el índice de la comisión pendiente (entrada #31) ya disponible, la pantalla Pagos seguía sin calcular el saldo. Firestore rechazaba con `failed-precondition` la agregación de totales de `drivers/{uid}/ledgerEntries` y proporcionó el enlace para crear el índice.
    - **Causa:** la consulta de `getDriverBalanceTotalsWithDatabase` filtra por `type == "cash_commission"` y calcula `count()` y `sum()` sobre `cashAmount`, `commission` y `driverEarnings`. Una agregación sobre varios campos con filtro requiere un índice compuesto, que no estaba declarado.
    - **Por qué faltaba desde la entrada #31:** las dos agregaciones del saldo se lanzan en paralelo con `Promise.all`, que solo propaga el primer error. Por eso Firestore solo había mostrado el enlace de la

comisión pendiente. La documentación oficial no permite deducir la composición de este índice, de modo que en la entrada #31 quedó pendiente hasta disponer del enlace.
    - **Índice añadido:**
        - Se decodificó el parámetro `create_composite` del enlace oficial: `ledgerEntries`, ámbito de colección, `type ASC, cashAmount ASC, commission ASC, driverEarnings ASC`, y el `__name__ ASC` implícito, que no se declara.
        - Coincide con la definición indicada en el prompt y se declaró con los campos en ese orden.
- **Qué se usó / modificó / descartó:**
    - Se añadió el índice a `firestore.indexes.json`, que pasa de 3 a 4 índices; los anteriores no cambian.
    - No fue necesario modificar las consultas, `firestore.rules`, los servicios, las pantallas, las pruebas ni las dependencias.
- **Archivos modificados:** `firestore.indexes.json` y `docs/AI_LOG.md`.
- **Commit:** Pendiente (lo registra el integrante).
- **Verificación:**
    - **`firestore.indexes.json`:** JSON válido, con 4 índices y sin duplicados.
    - **Suites:**
        - `npm run test:unit`: 139/139.
        - `npm run test:integration`: 37/37.
        - `npm run test:rules`: 151/151 (0 omitidas). Ningún aviso "maximum of 1000 expressions".
    - **`git diff --check`:** sin errores.
    - **Limitación:** el emulador no verifica índices, de modo que el efecto del índice solo puede comprobarse en el proyecto real tras desplegarlo.
    - **Sin desplegar:** no se desplegó nada.
- **Pendientes:**
    - Desplegar los índices con `--only firestore:indexes`, cuando el integrante lo autorice.
    - Comprobar que el índice esté "Habilitado" y que Ganancias → Pagos muestre el saldo.
    - Verificar en el proyecto real la consulta paginada de cobros pendientes (entrada #31).
- **Aprendizaje:** Cuando varias consultas que requieren índices se ejecutan en paralelo, los índices que faltan aparecen de uno en uno. Por eso conviene volver a probar la pantalla después de habilitar cada índice, hasta que deje de aparecer el error.

Entrada #33

- **Fecha:** 2026-11-13
- **Integrante:** Julian Cardona
- **Herramienta:** Claude Code (Anthropic), modelo Claude Opus 5.5
- **Objetivo:** Crear un script de desarrollo que inserte seis solicitudes de viaje de prueba en el Firestore real, sin debilitar las reglas ni modificar el simulador del emulador.
- **Prompt (texto literal):**

```
Necesito preparar seis solicitudes de viaje de prueba para UberClone Driver.

**Contexto:**
- Proyecto React Native con Expo SDK 58, JavaScript y Firestore.
- Firebase: `uberclone-driver-8e0bf`.
```

```
- Ya existe `scripts/dev/simulateTripRequests.cjs`, que funciona exclusivamente con el emulador.
- Existe `scripts/dev/tripRequestTemplate.json`.
- La aplicación consulta solicitudes pendientes ordenadas por `createdAt`.
- Las reglas de Firestore implementan un ciclo de vida seguro para los viajes.

**Objetivo:**
Crear un script independiente de desarrollo que permita insertar seis viajes de prueba en el
Firestore real para verificar la aplicación.

**Instrucciones:**

1. Primero revisa `firestore.rules`, `src/services/tripService.js`, `src/utils/fareLogic.js`,
`scripts/dev/simulateTripRequests.cjs` y la plantilla JSON.
2. No modifiques el simulador existente.
3. Crea un script nuevo dentro de `scripts/dev/` que genere seis viajes con rutas variadas de
Medellín.
4. Utiliza dos solicitudes Economy, dos XL y dos Premium, con diferentes métodos de pago.
5. Calcula las tarifas con la lógica existente y utiliza timestamps reales.
6. Identifica el mecanismo de autenticación adecuado para escribir en Firestore sin debilitar las
reglas. No introduzcas credenciales en el repositorio.
7. Exige confirmación explícita antes de escribir en el proyecto real.
8. Marca los documentos como datos de prueba y evita duplicados accidentales.
9. No modifiques documentos existentes, reglas de Firestore, pantallas ni dependencias salvo
necesidad justificada.
10. Documenta el procedimiento en `docs/AI_LOG.md`, manteniendo las entradas consecutivas y
`Consideraciones generales` al final.
11. No ejecutes escrituras reales, no hagas commits, push ni despliegues.

**Importante:** si las reglas actuales impiden crear solicitudes desde el cliente, explícame la
alternativa segura. No modifiques las reglas para saltarte las restricciones.

Al finalizar, informa los archivos creados, las verificaciones realizadas y el comando exacto que
debería ejecutar para generar los seis viajes. No lo ejecutes todavía.
```

- **Resumen de la respuesta:**
  - **Restricción de las reglas:** `firestore.rules` impide crear viajes desde cualquier cliente (`match /trips/{tripId}`: `allow create, delete: if false`). La autenticación anónima de la app no puede crear solicitudes, y no debe poder, porque en el sistema completo las crea la app del pasajero o un backend.
  - **Mecanismo elegido:** la API REST de Firestore con un token OAuth de corta duración (alrededor de 1 hora) de la cuenta propietaria del proyecto, obtenido con `gcloud auth print-access-token`.
    - Es acceso de servidor autorizado por IAM, que no pasa por las reglas; estas no se modifican.
    - El token se pasa en la variable `FIRESTORE_ACCESS_TOKEN` de la orden; el script no lo guarda ni lo imprime, y no hay credenciales en el repositorio.
    - Como ese acceso omite las reglas, el script valida cada documento con las mismas condiciones que la app y las reglas antes de escribir.
  - **Script nuevo:** `scripts/dev/seedTestTripRequests.cjs`.
    - **Viajes:** 6 rutas de Medellín: Económico en efectivo y con Mercado Pago, XL en efectivo y con Stripe, Premium con Mercado Pago y en efectivo; tres con propina.
    - **Tarifas:** calculadas con `calculateFare()` de `src/utils/fareLogic.js`. `createdAt` lo fija el servidor al escribir (`REQUEST_TIME`).

        - **Marcas de prueba:** `testData: true`, `testBatch` (lote) y `testSource`.
        - **Sin duplicados:** IDs deterministas `prueba_<lote>_<n>_<categoría>` con la precondición "el documento no existe", en una única escritura atómica. Repetir el comando no duplica ni sobrescribe nada, y si un viaje del lote ya existe no se crea ninguno.
        - **Protecciones:** sin `--escribir` solo muestra la vista previa. En el proyecto real exige `--proyecto uberclone-driver-8e0bf` y escribir el ID del proyecto en una confirmación interactiva, y rechaza cualquier otro proyecto.
        - **Opciones:** `--estado` consulta los viajes del lote sin escribir; `--solo <n>` crea un solo viaje; `--lote <id>` crea otro conjunto; `--emulador` escribe en el emulador, solo con proyectos `demo-`.
- **Qué se usó / modificó / descartó:**
    - **Reutilizado:** `calculateFare`, `buildCompletionBreakdown`, `hasPricingInputs` y las etiquetas de la app, cargados con `tests/helpers/loadModule.cjs`, igual que el simulador. Se usa `fetch` de Node 24, sin dependencias nuevas.
    - **Descartado:**
        - **Modificar `firestore.rules`** para permitir crear viajes desde la app.
        - **Firebase Admin SDK con una clave de cuenta de servicio:** añade una dependencia y una clave privada de larga duración.
        - **Reutilizar las credenciales guardadas por `firebase-tools`:** no hay un mecanismo admitido para obtener un token a partir de ellas.
        - **Borrado de los documentos desde el script:** solo crea documentos nuevos.
    - **Sin modificar:** el simulador del emulador, las reglas, las pantallas, los servicios y las dependencias.
- **Procedimiento (proyecto real):**
    1. Instalar Google Cloud CLI, fuera del repositorio: `brew install --cask gcloud-cli`.
    2. Iniciar sesión con la cuenta propietaria del proyecto: `gcloud auth login`.
    3. Revisar la vista previa: `node scripts/dev/seedTestTripRequests.cjs`.
    4. Crear los viajes y confirmar escribiendo el ID del proyecto: `FIRESTORE_ACCESS_TOKEN="$(gcloud auth print-access-token)" node scripts/dev/seedTestTripRequests.cjs --proyecto uberclone-driver-8e0bf --escribir`.
    5. Comprobar con `--estado`, en lugar de `--escribir`, y en la pestaña Viajes de la app.
    6. Para borrar los datos de prueba, el propietario los elimina desde Firebase Console filtrando por `testData == true`.
- **Archivos modificados:** `scripts/dev/seedTestTripRequests.cjs` (nuevo) y `docs/AI_LOG.md`.
- **Commit:** Pendiente (lo registra el integrante).
- **Verificación:**
    - **Vista previa:** muestra los 6 viajes; las tarifas coinciden con la fórmula de la guía.
    - **Protecciones:** el script se detiene sin conectarse si falta el destino o el token, si el proyecto no es `uberclone-driver-8e0bf`, si no hay una terminal interactiva, si la confirmación es incorrecta (comprobado con una terminal simulada y un token falso) o si se usa `--emulador` con un proyecto que no es `demo-`.
    - **Emulador, con una verificación ad hoc fuera del repositorio:**
        - **Escritura:** se crean los 6 viajes y repetir el lote se rechaza. Un lote con un viaje ya existente no crea ninguno de los otros 5, y un documento ajeno no cambia.
        - **Contenido:** `createdAt` es un timestamp del servidor y hay 2 viajes por categoría con los 3 métodos de pago.
        - **Compatibilidad con la app:** la consulta de la app devuelve los 6 viajes con las reglas, y un viaje en efectivo y otro con Stripe completan el ciclo con los servicios reales (oferta, aceptación, "Voy en

camino", llegada, inicio, finalización y cobro en efectivo). Las tarifas guardadas al finalizar coinciden con `estimatedFare`.
    - **No ejecutado:** no se ejecutó ninguna escritura real ni ninguna llamada al proyecto real.
- **Limitaciones:**
    - La autenticación con el token y la API REST del proyecto real no se probó; se verificará al ejecutar el comando.
    - La distancia, la duración y las coordenadas de las rutas son valores de prueba aproximados.
    - Cada conductor tiene 15 s desde que ve una solicitud. Si aparecen las 6 a la vez, vencen juntas; para recorrer el ciclo conviene crearlas de una en una con `--solo`.
    - Las solicitudes que nadie acepta quedan en `pending` (limitación conocida) y son visibles para todos los conductores aprobados y conectados hasta que se borren.
- **Aprendizaje:** Cuando las reglas impiden con razón que un cliente cree ciertos documentos, los datos de prueba deben crearse con una identidad de servidor autorizada por IAM, no relajando las reglas. Como esa identidad no pasa por las reglas, la herramienta debe validar los datos y protegerse de escrituras accidentales por sí misma.
- **Revisión posterior (mismo día), prompt (texto literal):**

```
Continuemos con el script `scripts/dev/seedTestTripRequests.cjs` de UberClone Driver.

La vista previa de los seis viajes se ejecutó correctamente. Antes de utilizar Firebase real,
necesito una revisión final de dos puntos:

1. **Lectura REST (`batchGet`):** revisa `callFirestore()` y `printStatus()`. La API de Firestore
puede devolver una secuencia de objetos JSON en lugar de un arreglo JSON convencional. Corrige el
procesamiento si es necesario, sin afectar `commit`.

2. **Escritura atómica:** verifica que la estructura de `documents:commit`, incluyendo `update`,
`currentDocument.exists: false` y `updateTransforms` con `REQUEST_TIME`, sea válida según la API
REST de Firestore. Comprueba que `createdAt` quede almacenado como Timestamp.

Después:
- Ejecuta las pruebas pertinentes contra el emulador.
- Verifica la sintaxis y `git diff --check`.
- Actualiza la entrada #33 de `docs/AI_LOG.md` si realizas cambios.
- Mantén las entradas cronológicas y `Consideraciones generales` al final.
- No modifiques `firestore.rules` ni la lógica de producción.
- No ejecutes escrituras en Firebase real.
- No hagas commits, push ni despliegues.

Al finalizar, informa qué corregiste, qué verificaste y si el script está listo para una prueba
controlada en Firebase real.
```

- **Resultado de la revisión:**
    - **`batchGet`:** la referencia oficial lo describe como una respuesta "streamed", sin fijar el formato del cuerpo, y no garantiza el orden de los documentos. El emulador lo devuelve como arreglo JSON.
        - **Antes:** el script solo admitía un único valor JSON y buscaba los errores únicamente en un objeto `{ error }` con estado HTTP de error.
        - **Ahora:** `parseJsonMessages()` admite un arreglo, un objeto o una secuencia de objetos concatenados o separados por saltos de línea, y detecta un mensaje `{ error }` dentro de la secuencia aunque el estado HTTP sea 200.

        - `printStatus()` distingue los documentos encontrados (`found`), los inexistentes (`missing`) y los que no recibieron respuesta. En este último caso se detiene, en lugar de presentarlos como inexistentes.
    - **`commit`:** la estructura no cambió; solo se trasladó a `buildCommitBody()`. Según la referencia de `Write`:
        - `updateTransforms` "can be set only when the operation is `update`".
        - `currentDocument.exists: false` exige que "the target document must not exist".
        - `REQUEST_TIME` es "the time at which the server processed the request".
        - Las escrituras de `commit` se aplican "atomically and in order".
        - Ahora el script comprueba que la respuesta incluya un resultado por viaje con su `timestampValue` antes de dar el lote por creado.
    - **Otros cambios:** el mensaje "el lote ya existe" solo se aplica a `commit`, y los errores `UNAUTHENTICATED` y `PERMISSION_DENIED` se reconocen también por su estado. Las funciones puras se exportan, y el programa solo se ejecuta cuando se invoca directamente (`require.main`).
- **Verificación de la revisión:**
    - **Emulador:** `commit` devuelve `transformResults` con `timestampValue`. `batchGet` muestra `createdAt` como `timestampValue`, y el SDK lo lee como `Timestamp`. Un duplicado responde 409 `ALREADY_EXISTS`.
    - **Formatos de respuesta, con verificación ad hoc fuera del repositorio:**
        - Se probaron un arreglo, un objeto, objetos separados por saltos de línea, objetos concatenados, llaves y comillas dentro de cadenas, y respuestas incompletas o mal formadas.
        - Con un servidor HTTP falso se ejecutó el programa completo. Se comprobaron `--estado` con una respuesta desordenada separada por saltos de línea, con un error dentro de una respuesta HTTP 200 y con una respuesta incompleta. También se comprobaron un `commit` 409 con el error dentro de un arreglo y un `commit` con resultados incompletos.
    - **Prueba completa contra el emulador, repetida:** se crean los 6 viajes, repetir el lote se rechaza, la escritura es atómica, un documento ajeno no cambia, y los viajes completan el ciclo con las reglas y los servicios reales.
    - **Repositorio:** sintaxis válida (`node --check`) y `git diff --check` sin errores. No se ejecutó ninguna escritura ni llamada al proyecto real.

Entrada #34

- **Fecha:** 2026-11-14
- **Integrante:** Julian Cardona
- **Herramienta:** Claude Code (Anthropic), modelo Claude Opus 5.5
- **Objetivo:** Encontrar la forma más rápida y segura de ejecutar `scripts/dev/seedTestTripRequests.cjs` contra el Firestore real sin instalar Google Cloud CLI, reutilizando credenciales existentes.
- **Prompt (texto literal):**

```
Claude, necesito que resolvamos de una vez la creación de los seis viajes de prueba en Firestore
REAL del proyecto `uberclone-driver-8e0bf`.

**Contexto:**
- Ya creaste `scripts/dev/seedTestTripRequests.cjs`.
- El script pasó las pruebas locales y la vista previa muestra los seis viajes.
- Los viajes todavía NO se han creado en Firebase real.
- Intentamos instalar Google Cloud CLI en mi Mac Intel con macOS 12 Monterey.
- Homebrew falló por `RuntimeError: Will not overwrite .../executables.txt`.
```

```
- Descargamos y descomprimimos Google Cloud CLI en `~/Downloads/google-cloud-sdk`.
- El instalador falló porque está usando Python 3.9.
- No quiero seguir perdiendo tiempo instalando herramientas innecesarias.

**Tu tarea:**

1. Haz un diagnóstico breve del entorno y determina la forma más rápida y segura de ejecutar el
script contra Firestore real.
2. Prioriza reutilizar herramientas y credenciales existentes. Si es posible, evita instalar
Python, Homebrew o Google Cloud CLI.
3. Si propones otra autenticación, debe ser segura, temporal y con permisos adecuados. No solicites
que pegue tokens, contraseñas o claves privadas en el chat.
4. No modifiques las reglas de seguridad de Firestore ni desactives sus restricciones.
5. Conserva los dos viajes existentes y evita duplicados.
6. No cambies el funcionamiento de la aplicación ni reescribas módulos.
7. Si necesitas modificar el script, haz cambios mínimos, justificados y probados.
8. No ejecutes escrituras reales hasta mostrarme el plan y solicitar mi autorización explícita.
9. No hagas commits, push ni despliegues automáticamente.
10. Mantén `docs/AI_LOG.md` ordenado, con las entradas numeradas consecutivamente y
`Consideraciones generales` al final.

**Objetivo inmediato:** conseguir que los seis viajes de prueba aparezcan en Firestore real y
comprobar que UberClone Driver puede recibirlos.

Quiero que trabajes de manera autónoma en el diagnóstico y las pruebas locales, sin hacerme
ejecutar comandos innecesarios. Al finalizar, indícame el procedimiento mínimo para autorizar y
realizar la escritura real de forma segura.Una cosa importante, bro: Claude Code tampoco puede
saltarse los permisos de Firebase. Para escribir en Firestore real necesitaremos alguna credencial
autorizada, pero puede encontrar una alternativa más sencilla que la instalación que estamos
intentando.
```

- **Resumen de la respuesta:**
  - **Diagnóstico:**
    - **Google Cloud CLI descargado (593.0.0):** exige Python 3.10 o superior (`MIN_SUPPORTED_PY3_VERSION = (3, 10)` en `lib/gcloud.py`) y no incluye Python propio. El sistema (macOS 12.7.6, Intel) solo tiene Python 3.9.6, por lo que `gcloud` no funciona sin instalar otro Python.
    - **Firebase CLI ya tiene sesión iniciada:** `firebase-tools` 15.36.0, la misma sesión que se usa para desplegar. Su sesión tiene el permiso `cloud-platform`, que cubre la API de Firestore. Su access token dura una hora y Firebase CLI lo renueva al ejecutar cualquier comando.
  - **Solución:** reutilizar esa sesión. No hace falta instalar nada ni copiar tokens.
  - **Cambio en el script:** se añadió la opción explícita `--credencial-firebase-cli`.
    - Lee de `~/.config/configstore/firebase-tools.json` solo el access token vigente. Nunca lee el refresh token, ni imprime o copia el token.
    - Exige el permiso `cloud-platform` y al menos 5 minutos de vigencia. Si no se cumplen, indica cómo renovar la sesión con un comando de solo lectura.
    - El resto del script no cambió: vista previa, confirmación interactiva, precondición "el documento no existe" y escritura atómica.
  - **Viajes existentes:** se conservan. Los IDs del script (`prueba_<lote>_<n>_<categoría>`) no coinciden con los existentes, y la precondición impide modificar cualquier documento que ya exista.
- **Qué se usó / modificó / descartó:**

    - **Descartado:**
        - Instalar Python 3.10 o superior, reparar Homebrew o completar la instalación de Google Cloud CLI.
        - OAuth Playground: obliga a copiar el token a mano.
        - Clave de cuenta de servicio: es una credencial privada de larga duración.
        - Cloud Shell: obliga a subir el script y sus dependencias.
    - **Riesgo:** el archivo de sesión de Firebase CLI es un detalle interno de esa herramienta y no una interfaz documentada. Si cambia, el script se detiene con un mensaje y sigue disponible la vía con `FIRESTORE_ACCESS_TOKEN`.
- **Archivos modificados:** `scripts/dev/seedTestTripRequests.cjs` y `docs/AI_LOG.md`.
- **Commit:** Pendiente (lo registra el integrante).
- **Verificación:**
    - **Credencial, con una carpeta de usuario falsa:** el script se detiene sin conectarse si no hay sesión, si falta el access token, si falta el permiso `cloud-platform`, si el token está vencido o vence en menos de 5 minutos, o si no se indica ninguna credencial. Con una credencial válida, una confirmación incorrecta se rechaza antes de cualquier llamada.
    - **Sin regresiones:** se repitieron las verificaciones ad hoc. Análisis de respuestas: 9/9. Prueba completa contra el emulador: 10/10.
    - **Proyecto real, solo lectura:**
        - `npx firebase-tools firestore:indexes --project uberclone-driver-8e0bf` renovó el token y mostró los 4 índices desplegados.
        - `--estado` del lote `20261114` funcionó con la credencial de Firebase CLI: Firestore aceptó la autenticación y la cabecera del proyecto, el script interpretó la respuesta real de `batchGet` y los 6 IDs aparecen como inexistentes.
    - **Sin escrituras:** no se ejecutó ninguna escritura real; queda pendiente de la autorización del integrante.
- **Aprendizaje:** Antes de instalar herramientas nuevas conviene revisar qué credenciales autorizadas existen ya en el equipo. Una consulta de solo lectura contra el proyecto real valida la autenticación y el formato de las respuestas sin riesgo, antes de pedir autorización para escribir.
- **Verificación posterior por el integrante:**
    - Claude Code no realizó ninguna escritura real. El integrante ejecutó personalmente el comando de creación del lote `20261114` en `uberclone-driver-8e0bf`, con `--credencial-firebase-cli` y la confirmación interactiva.
    - Se crearon los seis documentos. Firestore devolvió `createdAt: 2026-11-14T05:08:03.404Z`.
    - En la consulta posterior con `--estado` y `--credencial-firebase-cli`, los seis documentos existen con `status: pending`, sin `driverId` asignado y con `createdAt` válido.
    - Los viajes anteriores no se modificaron.

Entrada #35

- **Fecha:** 2026-11-14
- **Integrante:** Julian Cardona
- **Herramienta:** Claude Code (Anthropic), modelo Claude Opus 5.5
- **Objetivo:** Diagnosticar, en solo lectura, por qué la pestaña Viajes mostraba «Sin solicitudes por ahora» después de crear viajes de prueba en Firestore real.
- **Prompt (texto literal):**

```
Claude, necesito un diagnóstico técnico de solo lectura sobre un problema en UberClone Driver.

**Situación confirmada:**
- Proyecto Firebase real: `uberclone-driver-8e0bf`.
- El conductor está aprobado, en línea y con la pantalla Viajes abierta.
- El script `scripts/dev/seedTestTripRequests.cjs` creó correctamente seis viajes del lote
`20261114`.
- Después creé un viaje individual:
  - ID: `prueba_prueba-viaje-01_1_economy`
  - Categoría: `economy`
  - Estado inicial: `pending`
  - `driverId: null`
  - `createdAt: 2026-11-14T05:48:34.671Z`
- La aplicación continuó mostrando «No hay solicitudes por ahora».
- El script confirmó la escritura real sin errores.

**Tu tarea:**

1. Revisa el código real de la suscripción a solicitudes, el hook correspondiente, la pantalla
Viajes, los filtros de elegibilidad y las reglas de Firestore.
2. Comprueba cómo funciona el contador de 15 segundos y si la antigüedad de `createdAt` impide
mostrar solicitudes.
3. Compara exactamente la estructura del viaje generado con los campos que espera la aplicación.
4. Revisa si el conductor aprobado y conectado cumple los filtros de categoría, disponibilidad y
asignación.
5. Determina si el problema está en la consulta, el filtrado, el listener, las reglas, la caducidad
de la oferta o la interfaz.
6. Ejecuta únicamente pruebas locales o consultas de solo lectura cuando sean necesarias.
7. No modifiques archivos, no crees más viajes reales, no cambies reglas, no hagas commits ni
despliegues.

**Entrega esperada:** diagnóstico con archivos y líneas concretas, causa confirmada o hipótesis
claramente identificadas y la corrección mínima recomendada. No implementes cambios todavía.

Quiero que distingas entre «el viaje no llegó al listener» y «el viaje llegó, pero fue descartado
por algún filtro».
```

- **Resumen de la respuesta:**
  - **El viaje llegó al listener.** La app creó la oferta `drivers/{uid}/tripOffers/prueba_prueba-viaje-01_1_economy` a las 05:49:32.261, 57,6 s después de `createdAt`, y la app solo crea ofertas para viajes recibidos de la suscripción.
  - **La oferta venció y la solicitud se ocultó.** Se rechazó automáticamente por vencimiento a las 05:49:48.967, 16,7 s después. Desde entonces el filtro de ofertas rechazadas la oculta para ese conductor, como está diseñado. Los otros viajes pendientes también tenían su oferta vencida; uno del lote se aceptó y completó.
  - **Causas descartadas:** la consulta, las reglas, la estructura del documento, la elegibilidad del conductor, la categoría y la antigüedad de `createdAt`. También el reloj del dispositivo: un retraso de 16,7 s corresponde a 15 s más el segundo de margen.
  - **Hipótesis principal del retraso de 57,6 s:** la app no estaba en primer plano.
- **Qué se usó / modificó / descartó:** consultas de solo lectura al proyecto real con la sesión de Firebase CLI, con máscaras de campos y sin datos personales, y reproducción local del filtro de la app con los datos reales. No se modificó código ni datos.

- **Archivos modificados:** ninguno en la consulta. Esta entrada se registró después, junto con la entrada #37.
- **Commit:** `c69bf0e` (documentación). Esta entrada no tuvo cambios de código.
- **Verificación:** consultas de solo lectura (`runQuery` de la consulta del listener y de las ofertas del conductor) y reproducción de `selectVisibleTripRequests` en cuatro momentos de la cronología.
- **Aprendizaje:** La oferta que la app guarda para cada solicitud recibida permite distinguir «no llegó al listener» de «llegó y se descartó» sin acceder al dispositivo.

Entrada #36

- **Fecha:** 2026-11-14
- **Integrante:** Julian Cardona
- **Herramienta:** Claude Code (Anthropic), modelo Claude Opus 5.5
- **Objetivo:** Auditar en solo lectura el estado de los pagos y diseñar la integración de Mercado Pago Checkout Pro según la sección 4 de la guía.
- **Prompt (texto literal):**

```
Claude, revisé la guía oficial «UberClone — Versión Conductor», especialmente la sección 4,
«Pasarela de pagos».

La guía establece que es obligatorio integrar una pasarela (Mercado Pago o Stripe) en modo de
prueba, con backend, checkout, confirmación y actualización del estado del viaje en Firestore.

Quiero implementar **Mercado Pago Checkout Pro con su API real y credenciales de prueba**, sin
procesar dinero real.

Realiza una auditoría de solo lectura del proyecto actual:

1. Revisa los servicios y componentes de pagos existentes.
2. Determina qué funcionalidades están simuladas y cuáles utilizan una API real.
3. Analiza la mejor arquitectura para integrar Checkout Pro con Expo SDK 58, React Native,
JavaScript y Firebase.
4. Evalúa si necesitamos Firebase Cloud Functions, su plan de facturación y las alternativas de
backend.
5. Identifica los cambios mínimos necesarios en Firestore, reglas, servicios y pantallas.
6. Diseña el flujo de creación del pago, checkout, webhook verificado, confirmación y actualización
de ganancias.
7. Conserva íntegramente el flujo de pagos en efectivo y los viajes ya registrados.
8. Propón fases de implementación, archivos nuevos y modificados, pruebas y requisitos de
seguridad.
9. Revisa cómo documentarlo en `docs/AI_LOG.md`, manteniendo las entradas consecutivas y
«Consideraciones generales» al final.

**Restricciones:** no modificar archivos, no instalar dependencias, no crear transacciones, no
desplegar, no hacer commits ni push. No solicitar ni exponer credenciales secretas en el chat.

Entrégame primero un diagnóstico y un plan detallado para revisión.
```

- **Resumen de la respuesta:**
  - **Diagnóstico:** el cobro en efectivo y el desglose de tarifa eran reales, pero no existía ninguna integración de pagos digitales; `mercadopago` y `stripe` solo eran valores y etiquetas. El saldo no incluía las ganancias cobradas por la pasarela.

    - **Facturación:** en solo lectura se comprobó que el proyecto no tiene facturación activa (`billingEnabled=false`) y que las APIs de Cloud Functions y Secret Manager están desactivadas. Según la documentación de Firebase, desplegar funciones exige el plan Blaze, aunque el emulador funciona sin él.
    - **Arquitectura propuesta:** función callable para crear la preferencia, `expo-web-browser` para el checkout, webhook con firma y confirmación con `GET /v1/payments/{id}`, actualización idempotente con Admin SDK y la app escuchando Firestore. Las reglas no se modifican.
    - **Plan:** cinco fases y las decisiones pendientes del integrante.
- **Qué se usó / modificó / descartó:** documentación oficial de Mercado Pago (Checkout Pro, notificaciones, cuentas y tarjetas de prueba de Colombia), de Firebase (requisitos de Cloud Functions y del plan Blaze) y de Expo SDK 58 (`expo-web-browser`). Se descartaron los enlaces de retorno a la app en Expo Go y un backend externo con clave de cuenta de servicio.
- **Archivos modificados:** ninguno en la auditoría. Esta entrada se registró después, junto con la entrada #37.
- **Commit:** `c69bf0e` (documentación). Esta entrada no tuvo cambios de código.
- **Verificación:** lectura de la guía, inventario del código con `grep` y consultas de solo lectura de facturación y APIs.
- **Aprendizaje:** Comprobar el plan de facturación antes de diseñar el backend evita elegir una arquitectura que no se puede desplegar.

Entrada #37

- **Fecha:** 2026-11-14
- **Integrante:** Julian Cardona
- **Herramienta:** Claude Code (Anthropic), modelo Claude Opus 5.5
- **Objetivo:** Implementar las fases 1 a 3 de la integración de Mercado Pago Checkout Pro en modo de prueba (backend con Cloud Functions, app y pruebas), sin desplegar ni activar el plan Blaze.
- **Prompt (texto literal):**

```
Claude, confirmo que la entrega definitiva de UberClone Driver es HOY, 14 de noviembre de 2026, a
las 8:30 a. m. Tenemos muy poco tiempo.

Autorizo iniciar la implementación de las fases 1 a 3 de Mercado Pago Checkout Pro, siguiendo tu
auditoría anterior.

**Objetivo prioritario:** integrar Mercado Pago en modo de prueba sin romper las funcionalidades
existentes.

Condiciones:

1. Trabaja en una rama de funcionalidad, nunca directamente sobre `main`.
2. Implementa el backend con Firebase Functions y pruebas automatizadas, utilizando el emulador
cuando sea posible.
3. Integra el flujo en Expo SDK 58 y Firestore, conservando el pago en efectivo.
4. Implementa verificaciones de seguridad, idempotencia y confirmación del pago desde la API.
5. No introduzcas credenciales reales ni secretos en el código.
6. Ejecuta las pruebas disponibles y reporta cuáles no pudiste ejecutar.
7. Documenta el uso de IA en `docs/AI_LOG.md`, con entradas consecutivas y «Consideraciones
generales» al final.
8. No hagas commits, push ni despliegues sin mi autorización.
9. No actives Blaze ni modifiques la facturación.
```

```
**Trabaja de manera autónoma en la implementación y las pruebas locales.** Solo detente si necesitas una decisión o acción que no puedas realizar sin mi autorización.

Al finalizar, entrega un resumen de archivos modificados, pruebas realizadas, errores pendientes y pasos exactos para habilitar el pago de prueba real.

Prioriza tener una versión estable y entregable antes que añadir funcionalidades adicionales.
```

- **Resumen de la respuesta:**
  - **Rama:** `feat/pasarela-mercadopago`.
  - **Backend:** nuevo directorio `functions/`, con Node 22, `firebase-functions` 7 y `firebase-admin` 14, y tres funciones:
    - `createPaymentPreference` (callable): exige sesión y que el conductor sea el del viaje, `completed` y `mercadopago`. Calcula el monto con el desglose guardado (el de la app solo se compara), crea la preferencia con `external_reference = tripId` y la reutiliza mientras está vigente.
    - `mercadoPagoWebhook`: verifica la firma `x-signature` con HMAC-SHA256 según la plantilla del validador del SDK oficial de Node de Mercado Pago, porque la documentación pública no la publica. Consulta el pago en la API y lo aplica en una transacción idempotente.
    - `paymentReturn`: página de retorno que también confirma el pago con la API.
  - **Resultado del pago:**
    - `approved`: el viaje pasa a `paid`, con `paymentStatus`, `paymentId` y `paidAt`, y se crea el asiento `mp_{tripId}` (`gateway_earning`).
    - `pending` y `rejected`: solo se actualiza `paymentStatus`.
    - Un pago aprobado no retrocede y un segundo pago aprobado no se aplica.
    - La auditoría queda en `paymentEvents`.
  - **Secretos:** `MP_ACCESS_TOKEN` y `MP_WEBHOOK_SECRET` se definen con `defineSecret` (Secret Manager); los parámetros no secretos tienen valores por defecto.
  - **App:**
    - Tarjeta «Cobro con Mercado Pago» en la pestaña Viajes, con el checkout en `expo-web-browser` y el resultado en tiempo real.
    - Saldo = ganancias de la pasarela – comisiones de efectivo pendientes (guía, sección 5). Reutiliza el índice compuesto ya desplegado gracias a que el asiento tiene la misma forma que el del efectivo.
    - Pantalla Pagos y etiquetas actualizadas.
  - **Sin cambios:** `firestore.rules` y el flujo del pago en efectivo.
- **Qué se usó / modificó / descartó:**
  - **Usado:** la documentación oficial consultada en la entrada #36 y el validador de firma del SDK oficial de Node de Mercado Pago. Para la página de retorno se usa `back_urls` sin enlaces profundos a la app.
  - **Descartado:**
    - Stripe, fuera de alcance.
    - Enlaces profundos de retorno en Expo Go.
    - La liquidación de las ganancias al conductor.
    - Modificar `firestore.rules`: no es necesario, porque las rutas sin regla están denegadas y el conductor no puede escribir pagos digitales.
- **Archivos modificados:**

    - **Backend:** `functions/` (`index.js`, `src/paymentLogic.js`, `src/signature.js`, `src/mercadoPagoClient.js`, `src/handlers.js`, `test/`, `package.json`, `package-lock.json`, `.env.example`, `.gitignore`).
    - **App, archivos nuevos:** `src/utils/gatewayPaymentLogic.js`, `src/services/gatewayPaymentService.js`, `src/hooks/useGatewayPayments.js`, `src/components/payments/PendingGatewayPayments.js`.
    - **App, archivos modificados:** `src/config/firebase.js`, `src/components/trips/TripRequests.js`, `src/screens/home/PaymentSummaryScreen.js`, `src/services/cashPaymentService.js`, `src/utils/cashPaymentLogic.js`.
    - **Pruebas:** `tests/gatewayPayment.test.cjs`, `tests/gatewayPayment.integration.test.cjs`, `tests/payments.rules.test.cjs`, `tests/cashPayment.test.cjs`, `tests/pagination.test.cjs`.
    - **Configuración y documentación:** `firebase.json`, `package.json`, `package-lock.json` (`expo-web-browser` ~58.0.8, instalado con `npx expo install`), `README.md` y `docs/AI_LOG.md`.
- **Commit:** `202ac3e` (backend), `66c2d4c` (app y pruebas) y `c69bf0e` (README y bitácora). Los cambios de las entradas #37 a #39 se confirmaron juntos en esos commits, agrupados por archivos.
- **Verificación:**
    - **Suites:**
        - `npm run test:unit`: 146/146.
        - `npm run test:integration`: 41/41, incluido el flujo de extremo a extremo: la app completa el viaje con las reglas, el backend real con Admin SDK aprueba el pago con la API simulada, y la app ve el viaje pagado y el saldo.
        - `npm run test:rules`: 156/156, 0 omitidas, sin avisos «maximum of 1000 expressions».
        - `npm run test:functions`: 10/10 de lógica y firma, y 10/10 de manejadores contra el emulador, con concurrencia, idempotencia, desajustes de monto, firma y retorno.
    - **Emulador de Functions de Firebase CLI:** cargó las tres funciones con secretos ficticios temporales, borrados al terminar. `paymentReturn` respondió 200, el webhook rechazó GET con 405 y la callable sin sesión respondió 401.
    - **Expo:** `npx expo-doctor` 20/20 y `npx expo export --platform ios` con 1006 módulos.
    - **Bundle de la app:** no contiene los nombres de los secretos ni llamadas a `api.mercadopago.com`.
    - **No ejecutado:** el despliegue, la prueba con la API real de Mercado Pago y la prueba en Expo Go, porque requieren el plan Blaze y las credenciales de prueba.
- **Limitaciones:**
    - El despliegue requiere activar Blaze, cargar los secretos y configurar el webhook en el panel de Mercado Pago.
    - Falta confirmar con la API real qué campo de checkout usar con credenciales de prueba (`init_point`, el valor por defecto, o `sandbox_init_point`) y la plantilla de la firma; se puede cambiar a `MP_WEBHOOK_SIGNATURE_MODE=report` sin perder seguridad, porque el estado siempre se confirma con la API.
    - Stripe no está integrado, y los reembolsos y contracargos se registran pero no se aplican.
- **Aprendizaje:** Si el estado del pago se toma siempre de la API de la pasarela y no de la notificación, el webhook queda protegido aunque la firma falle. Inyectar Firestore y el cliente de la pasarela en los manejadores permitió probar el backend real contra el emulador sin credenciales.

Entrada #38

- **Fecha:** 2026-11-14

- **Integrante:** Julian Cardona
- **Herramienta:** Claude Code (Anthropic), modelo Claude Opus 5.5
- **Objetivo:** Poder presentar el cobro digital completo desde Expo Go sin credenciales de Mercado Pago ni plan Blaze, con un modo de demostración local, claramente identificado y aislado del entorno real.
- **Prompts (texto literal):**

Primer prompt (diagnóstico):

```
Claude, antes de cerrar el proyecto necesito aclarar algo.

Como no conseguimos las credenciales de Mercado Pago, quiero saber si podemos aprovechar la API simulada que ya utilizaste en las pruebas para demostrar un pago digital completo durante la presentación.

Haz primero un diagnóstico breve:

1. ¿La API simulada funciona únicamente dentro de las pruebas automatizadas o también podemos utilizarla desde la aplicación Expo Go?
2. ¿Podemos demostrar desde la interfaz un viaje con Mercado Pago, abrir un checkout simulado, aprobar el pago y observar cómo Firestore actualiza el viaje y las ganancias?
3. ¿Cuánto trabajo adicional necesitaríamos para conseguirlo?
4. ¿Podemos hacerlo sin Blaze, sin credenciales de Mercado Pago y sin afectar el flujo de efectivo?
5. ¿Cómo impedirías que el modo de demostración se utilice accidentalmente en producción?

No modifiques archivos todavía. No hagas commits ni despliegues.

Necesito una respuesta concreta y una estimación de tiempo, porque la entrega es hoy a las 8:30 a. m . chaygpt tambien dice que  el quiere mantener esta condicion ; Y hay una condición que quiero mantener: la simulación debe estar claramente identificada como modo de demostración, nunca como un pago auténtico de Mercado Pago.
```

Segundo prompt (autorización):

```
Claude, autorizo el PLAN A con la salida de emergencia del PLAN B.

La entrega es hoy, 14 de noviembre, a las 8:30 a. m. Necesitamos una versión estable y lista para presentar.

Condiciones:

1. Empieza exclusivamente por la comprobación de conectividad entre Expo Go, Firebase Auth y los emuladores locales. Si no funciona en un máximo de 15–20 minutos, detente e infórmame antes de continuar.
2. Si funciona, implementa el modo demostración desde Expo Go, siguiendo tu arquitectura y las protecciones que propusiste.
3. Mantén completamente separado el modo demostración del entorno real. Nunca permitas que una simulación escriba pagos aprobados en Firestore de producción.
4. Identifica claramente todas las pantallas y pagos simulados con «MODO DEMOSTRACIÓN».
5. No modifiques el flujo de pagos en efectivo ni las funcionalidades estables.
6. Ejecuta las pruebas automatizadas y verifica que el modo demostración esté deshabilitado fuera de los emuladores.
7. Actualiza README y docs/AI_LOG.md con los cambios realizados y las limitaciones reales.
8. No actives Blaze, no despliegues funciones, no utilices credenciales reales y no hagas commits
```

```
ni push.
9. Si el Plan A no funciona o se vuelve demasiado complejo, detente y proponme ejecutar el Plan B.
No cambies de estrategia silenciosamente.

Cuando termines, necesito los comandos exactos para iniciar la demostración desde mi Mac y abrirla
en Expo Go.

Prioriza seguridad, estabilidad y facilidad de presentación. No agregues funcionalidades ajenas al
modo demostración.
```

- **Resumen de la respuesta:**
  - **Diagnóstico:** la API simulada solo existía dentro de las pruebas. Para usarla desde Expo Go hacían falta los emuladores locales y un modo de demostración (plan A, unas 3 h), con un plan B por terminal como salida de emergencia.
  - **Comprobación de conectividad (plan A, en 1 minuto):**
    - IP local `192.168.1.5`, firewall de macOS desactivado y Metro accesible en la red.
    - Firebase CLI 15.36 ejecuta las funciones del emulador con `skipTokenVerification`.
    - Desde la IP local: Firestore aplicó las reglas con el uid de un token con la forma de Firebase Auth (200 en el documento propio, 403 en el ajeno), y la callable aceptó la sesión (404 «El viaje no existe») y rechazó la ausencia de token (401).
  - **Backend (`functions/src/demoMode.js`):**
    - Un cliente con la misma interfaz que el de Mercado Pago guarda las preferencias y los pagos de demostración en el emulador (`demoPreferences` y `demoPayments`).
    - `demoCheckout` es una página identificada como «MODO DEMOSTRACIÓN — no es Mercado Pago», con los botones Aprobar, Pendiente y Rechazar.
    - `processPayment` aplica el resultado con las mismas validaciones y la misma idempotencia, y marca el viaje, el asiento y la auditoría con `paymentMode: "demo"` e IDs `DEMO-`.
    - Protecciones: `PAYMENTS_DEMO_MODE=true` solo se acepta con `FUNCTIONS_EMULATOR=true` y `FIRESTORE_EMULATOR_HOST`. En cualquier otro entorno, `index.js` no carga y un despliegue fallaría. En modo demostración no se exportan el webhook ni el retorno reales, no se declaran secretos, y el webhook real no acepta IDs `DEMO-`.
  - **App:**
    - `src/config/paymentsDemo.js` activa el modo solo con `__DEV__`, `EXPO_PUBLIC_PAYMENTS_DEMO=true` y una IP privada o `localhost`.
    - `src/config/firebase.js` conecta Firestore y Functions a los emuladores; la autenticación anónima sigue siendo la real.
    - Aviso fijo `DemoModeBanner` y etiquetas «MODO DEMOSTRACIÓN» en la tarjeta de cobro, el resultado, el historial y la pantalla Pagos.
    - El checkout `http://` solo se acepta para el emulador configurado.
  - **Herramienta:**
    - `scripts/dev/paymentDemo.cjs`, con los comandos `emuladores`, `preparar`, `solicitud` y `app` y sus atajos `npm run demo:*`.
    - `firebase.demo.json` define los puertos 8181 y 5002 en la red local, separados del emulador de pruebas.
    - El modo del backend se activa con `functions/.env.local` (solo lo leen los emuladores; excluido de Git), que el script borra al terminar.
- **Qué se usó / modificó / descartó:**

    - **Hallazgo durante la implementación:** el emulador de Functions no pasa las variables de la terminal a la fase en que descubre las funciones, pero sí a su ejecución. Por eso se usó `.env.local`, que se aplica en ambas fases.
    - **Corrección durante la prueba:** el lanzador ahora reenvía Ctrl+C y SIGTERM a los emuladores y espera a que terminen.
    - **Descartado:** escribir pagos simulados en el Firestore real, imitar la marca de Mercado Pago y el plan B, porque el plan A funcionó.
    - **Sin cambios:** `firestore.rules`, el flujo de efectivo y el comportamiento sin el modo demostración.
- **Archivos modificados:**
    - **Backend:** `functions/src/demoMode.js` (nuevo), `functions/src/handlers.js`, `functions/index.js`, `functions/test/demoMode.test.cjs` (nuevo), `functions/test/handlers.emulator.test.cjs`, `functions/package.json`.
    - **App:** `src/config/paymentsDemo.js` (nuevo), `src/components/DemoModeBanner.js` (nuevo), `App.js`, `src/config/firebase.js`, `src/services/gatewayPaymentService.js`, `src/utils/gatewayPaymentLogic.js`, `src/utils/cashPaymentLogic.js`, `src/components/payments/PendingGatewayPayments.js`, `src/screens/home/PaymentSummaryScreen.js`.
    - **Herramienta y configuración:** `scripts/dev/paymentDemo.cjs` (nuevo), `firebase.demo.json` (nuevo), `package.json`.
    - **Pruebas y documentación:** `tests/gatewayPayment.test.cjs`, `README.md`, `docs/AI_LOG.md`.
- **Commit:** `202ac3e` (backend), `66c2d4c` (app, script y pruebas) y `c69bf0e` (README y bitácora), compartidos con las entradas #37 y #39.
- **Verificación:**
    - **Suites:**
        - `npm run test:unit`: 149/149.
        - `npm run test:integration`: 41/41.
        - `npm run test:rules`: 156/156, sin avisos «maximum of 1000 expressions».
        - `npm run test:functions`: 13/13 unitarias y 12/12 contra el emulador.
    - **Protecciones del modo:** sin el modo, `index.js` exporta las funciones reales; con el modo pedido fuera del emulador, la carga falla; en el emulador, solo `createPaymentPreference` y `demoCheckout`.
    - **Prueba de extremo a extremo con `npm run demo:emuladores`, por la IP local:**
        1. `preparar` y `solicitud` crearon los datos.
        2. La consulta real de la app, con reglas, mostró el cobro pendiente.
        3. La callable, con la sesión del conductor, devolvió el checkout de demostración.
        4. La página se identificó como «MODO DEMOSTRACIÓN».
        5. La aprobación pasó el viaje a `paid` con `paymentMode: "demo"` e ID `DEMO-`, y la app lo vio en tiempo real.
        6. El saldo, calculado con reglas, incluyó la ganancia.
    - **Cierre:** al detener los emuladores con SIGTERM, el puerto quedó libre y `.env.local` se borró.
    - **Expo:** `npx expo-doctor` 20/20. `npx expo export --platform ios` en producción (1008 módulos, sin nombres de secretos) y en desarrollo con las variables de demostración (1127 módulos).
    - **Sin ejecutar:** la prueba en el móvil con Expo Go. El tramo del móvil a la IP del Mac no se pudo comprobar desde el equipo; se confirma al abrir la demostración.
- **Limitaciones:**
    - La demostración necesita el Mac y el móvil en la misma red Wi-Fi y los tres procesos en marcha.

    - Los datos viven solo en el emulador y se borran al detenerlo. El perfil del conductor es de demostración y se crea con `preparar`, usando el UID que muestra Metro.
    - No demuestra la integración con la API real de Mercado Pago, que sigue pendiente de Blaze y credenciales (entrada #37).
- **Aprendizaje:** Una simulación es aceptable en una presentación si queda aislada por diseño (solo emuladores, protecciones al cargar, sin despliegue posible) y se identifica en cada pantalla y en cada dato. Comprobar la conectividad antes de construir evitó invertir horas en un plan que podía no funcionar en el móvil.

Entrada #39

- **Fecha:** 2026-11-14
- **Integrante:** Julian Cardona
- **Herramienta:** Claude Code (Anthropic), modelo Claude Opus 5.5
- **Objetivo:** Corregir dos problemas visuales detectados en un iPhone real con Expo Go: importes cortados en el historial de ganancias y el aviso «MODO DEMOSTRACIÓN» encima de la barra de estado.
- **Prompt (texto literal):**

```
Claude, estamos realizando las pruebas finales de UberClone Driver en un iPhone real mediante Expo Go.

**El flujo de solicitudes, viajes y pago simulado funciona correctamente**, pero detecté dos problemas visuales que necesito corregir.

### 1. Historial de ganancias: importe cortado

En la pantalla de historial de ganancias, un valor positivo en verde, por ejemplo **$26.640**, aparece demasiado desplazado hacia la derecha y se corta, mostrando algo como `26.6...`.

Necesito que:
- Localices el componente exacto que renderiza los importes del historial.
- Corrijas la distribución horizontal para que los importes se vean completos.
- Mantengas la alineación, los colores y la jerarquía visual.
- Permitas que el texto descriptivo ocupe menos espacio o se distribuya en varias líneas si es necesario.
- Evites desbordamientos en pantallas pequeñas y con importes mayores.
- No cambies el cálculo de tarifas, comisiones ni ganancias.

### 2. Safe Area del modo demostración

El aviso naranja «MODO DEMOSTRACIÓN» aparece encima de la hora y la batería del iPhone.

Corrige su posición para respetar el Safe Area superior de iOS y Android, sin tapar los elementos del sistema ni perjudicar la navegación.

### Restricciones

- Realiza cambios mínimos y localizados.
- No alteres Firebase, los emuladores, la lógica de pagos ni los viajes.
- No hagas commits ni push.
- No modifiques archivos `.env`.
- Ejecuta las verificaciones pertinentes.
- Si utilizas IA para modificar código, actualiza `docs/AI_LOG.md` con una entrada consecutiva,
```

```
antes de `### Consideraciones generales`, y los comentarios de referencia correspondientes.
- Al terminar, indícame los archivos modificados, las verificaciones ejecutadas y cualquier
limitación de pruebas visuales en un dispositivo real.

No necesito un rediseño completo, sino dos correcciones visuales profesionales y seguras.
```

- **Resumen de la respuesta:**
  - **Importe cortado:** la fila de cada viaje en `EarningsScreen.js` tiene dos columnas, una de texto y otra con el importe en verde. La columna de texto no tenía `flex: 1`, y la etiqueta larga «Pago aprobado (MODO DEMOSTRACIÓN)», añadida en la entrada #38, la ensanchaba hasta encoger y cortar el importe.
    - Ahora la columna de texto ocupa el espacio libre (`flex: 1`, `minWidth: 0`) y se ajusta en varias líneas; la dirección conserva una sola línea con puntos suspensivos y pierde el ancho fijo de 180.
    - La columna del importe no se encoge (`flexShrink: 0`, `maxWidth: '55%'`, alineada a la derecha).
    - Se aplicó la misma corrección a «Movimientos del saldo» de `PaymentSummaryScreen.js`, que tenía el mismo patrón.
  - **Aviso de demostración:** el aviso usaba el `SafeAreaView` de la librería sin un `SafeAreaProvider`, y en el iPhone no aplicó el margen superior.
    - Ahora, solo en MODO DEMOSTRACIÓN, `App.js` envuelve la app en un `SafeAreaProvider` raíz, con `initialWindowMetrics`, y el aviso toma el margen de `useSafeAreaInsets`.
    - La navegación va en un `SafeAreaProvider` anidado debajo del aviso, que mide sus márgenes desde su posición: no se duplica el margen superior y la navegación conserva el comportamiento de siempre.
    - Sin el modo demostración, `App.js` renderiza exactamente lo mismo que antes.
- **Qué se usó / modificó / descartó:** se usó `react-native-safe-area-context` 5.11.1, ya instalado; se revisó en su código que el proveedor anidado parte de los márgenes del padre y que ambos ocupan el espacio con `flex: 1`. Se descartaron un margen superior fijo y un `SafeAreaProvider` raíz también fuera del modo demostración, para no cambiar la app estable. No se modificaron cálculos, Firebase, emuladores, pagos ni viajes.
- **Archivos modificados:** `src/screens/home/EarningsScreen.js`, `src/screens/home/PaymentSummaryScreen.js`, `src/components/DemoModeBanner.js`, `App.js` y `docs/AI_LOG.md`.
- **Commit:** `66c2d4c` (Pagos, aviso de demostración y `App.js`, junto con la entrada #38), `7f19f73` (Ganancias) y `c69bf0e` (bitácora).
- **Verificación:**
  - **Suites:** `npm run test:unit` 149/149 y `npm run test:integration` 41/41, sin cambios de comportamiento.
  - **Expo:** `npx expo-doctor` 20/20. `npx expo export` genera los bundles de iOS en producción (1008 módulos), de iOS en desarrollo con las variables del MODO DEMOSTRACIÓN (1127 módulos) y de Android (1012 módulos).
  - **Repositorio:** `git diff --check` sin errores.
- **Limitaciones:** no se pudo comprobar la presentación visual en un dispositivo desde el equipo de desarrollo, porque no hay pruebas de renderizado en el proyecto. El integrante debe confirmarla en el iPhone: el importe completo con la etiqueta larga y el aviso debajo de la hora y la batería. La prueba en Android también queda pendiente.
- **Aprendizaje:** En una fila de dos columnas, la columna de texto debe ser la que cede espacio (`flex: 1`, `minWidth: 0`) y la del importe no debe encogerse; si no, un texto más largo empuja y corta el dato principal. En React Native, el margen seguro de un elemento fijo superior debe salir de un `SafeAreaProvider`, y el contenido situado debajo necesita su propio proveedor para no duplicar el margen.

Entrada #40

- **Fecha:** 2026-11-14
- **Integrante:** Julian Cardona
- **Herramienta:** Claude Code (Anthropic), modelo Claude Opus 5.5
- **Objetivo:** Completar el historial de carreras según la sección 6 de la guía (filtros, ordenación y paginación correctas, detalle y mapa), ejecutar todas las verificaciones, documentar el cierre y preparar commits locales sin push ni despliegues.
- **Prompt (texto literal):**

```
**IMPORTANTE — ESTADO INICIAL VERIFICADO**
Antes de modificar archivos, ejecuta git status --short --branch y revisa los cambios
preexistentes. No descartes ni sobrescribas trabajo anterior. Como esta rama contiene
funcionalidades ya probadas, conserva todo su contenido. Si creas commits, verifica previamente que
no incluyan credenciales ni archivos .env.
Estoy en la rama `feat/pasarela-mercadopago` y tengo cambios sin confirmar correspondientes a la
implementación de Mercado Pago, el modo demostración, las correcciones visuales, Firebase
Functions, pruebas y documentación.

Todos esos cambios deben conservarse.

Antes de trabajar:

1. Inspecciona `git status` y los archivos modificados.
2. No descartes ni sobrescribas cambios preexistentes.
3. Revisa que `functions/`, los archivos nuevos y los archivos de configuración no contengan
secretos.
4. Verifica que `.env`, credenciales y `node_modules` estén excluidos de los commits.
5. Conserva la funcionalidad del modo demostración y las correcciones visuales ya probadas.

Puedes crear commits locales organizados una vez que las verificaciones correspondientes pasen,
pero NO puedes hacer push, fusionar ramas ni desplegar Firebase.

A continuación, ejecuta el plan completo de cierre del proyecto que te proporcionaré.

Claude, te autorizo a realizar la **fase final de desarrollo y preparación de entrega de UberClone
Driver**.

Tengo muy poco tiempo para presentar el proyecto y necesito que trabajes de forma autónoma hasta
completar los requisitos pendientes del historial, ejecutar las verificaciones y preparar la
documentación.

La prioridad es entregar una aplicación funcional, estable y lo más alineada posible con la guía
académica.

## CONTEXTO

Proyecto: UberClone Driver.

Tecnologías: React Native, Expo SDK 58, JavaScript, Firebase Authentication, Firestore, Firebase
Functions, Cloudinary y emuladores Firebase.

Rama actual: `feat/pasarela-mercadopago`.
```

La aplicación ya fue probada en un iPhone real. Funcionan el registro, disponibilidad, solicitudes, aceptación, viaje, finalización y pagos simulados.

Las correcciones visuales de Safe Area e importes ya fueron verificadas satisfactoriamente.

El modo demostración debe permanecer aislado de Firebase real.

IMPORTANTE: No debes convertir el checkout simulado en una supuesta integración real de Mercado Pago. Documenta esa limitación claramente.

FASE 1 — HISTORIAL COMPLETO SEGÚN EL APARTADO 6

Implementa el historial de carreras de forma profesional y funcional.

1. Listado de viajes

Utiliza `FlatList` y muestra:

- Fecha y hora.
- Origen y destino.
- Categoría del vehículo.
- Tarifa total.
- Ganancia neta del conductor.
- Estado del viaje.
- Método y estado del pago.

Los importes deben mostrarse completos en pantallas pequeñas.

2. Filtros

Implementa:

- Hoy.
- Semana.
- Mes.
- Rango personalizado de fechas.
- Método de pago.

Los filtros deben combinarse correctamente.

Reutiliza el DateTimePicker ya instalado.

Incluye pruebas de límites de fechas, combinaciones de filtros y resultados vacíos.

3. Ordenación y paginación

Corrige el problema detectado en `getDriverTripHistory`.

Actualmente se consultan hasta 50 documentos sin ordenación de Firestore y después se ordenan en JavaScript, lo que puede excluir viajes recientes.

Implementa una estrategia correcta y estable:

- Orden descendente por fecha.
- Paginación o carga incremental.

- Consultas eficientes.
- Compatibilidad con viajes existentes.
- Prevención de duplicados.
- Manejo de errores.
- Revisión de índices y reglas de Firestore.

Si hacen falta índices, prepara los archivos de configuración correspondientes, pero NO los despliegues.

No debilites las reglas de seguridad.

4. Detalle individual del viaje

Al tocar un viaje, debe abrirse una vista de detalle con:

- Fecha y hora.
- Origen y destino.
- Categoría.
- Distancia.
- Duración.
- Tarifa.
- Comisión del 20 % aplicada.
- Propina.
- Ganancia neta.
- Método y estado del pago.
- Calificación del pasajero, si existe.

Si no existe una calificación real, mostrar «Sin calificación».

No inventar datos.

5. Mapa y ruta del historial

El detalle debe mostrar un mapa con el origen, destino y la ruta correspondiente.

Reutiliza `DriverMap`, `routeService` y la infraestructura de mapas existente siempre que sea técnicamente seguro.

Si el viaje no conserva la geometría de la ruta, calcula una ruta de referencia utilizando las coordenadas disponibles.

No presentes una ruta recalculada como si fuera necesariamente el recorrido histórico exacto.

Si no hay coordenadas o falla el servicio de rutas, muestra un mensaje claro y conserva visible el resto del detalle.

No modifiques la navegación ni la lógica del viaje activo salvo que sea absolutamente indispensable.

6. Calidad visual

Mantén el estilo existente.

Respeta el Safe Area de iOS y Android.

Evita textos cortados, desbordamientos, botones inaccesibles y errores de navegación.

No rediseñes toda la aplicación.

FASE 2 — PRUEBAS Y ESTABILIDAD

Cuando termines la implementación:

1. Ejecuta todas las pruebas unitarias.
2. Ejecuta las pruebas de integración.
3. Ejecuta las pruebas de reglas Firestore disponibles.
4. Ejecuta las pruebas de Functions.
5. Ejecuta `npx expo-doctor`.
6. Genera bundles de iOS y Android.
7. Ejecuta `git diff --check`.
8. Comprueba imports, dependencias, navegación y errores de sintaxis.
9. Verifica que las consultas del historial respeten las reglas de seguridad.
10. Añade pruebas automatizadas para los nuevos filtros, paginación y datos del historial.

Si no puedes ejecutar una prueba por limitaciones del entorno, indícalo expresamente.

No inventes resultados ni afirmes haber probado visualmente un dispositivo físico.

Corrige los problemas encontrados dentro del alcance autorizado.

FASE 3 — DOCUMENTACIÓN FINAL

Actualiza:

- `README.md`.
- `docs/AI_LOG.md`.
- Documentación necesaria para ejecutar el modo demostración.
- Reporte de cumplimiento de requisitos.

En la bitácora:

- Mantén las entradas `#1` hasta la última en orden cronológico.
- No dupliques números.
- Inserta las entradas nuevas antes de `### Consideraciones generales`.
- Mantén esa sección siempre al final.
- Documenta las intervenciones de IA y los archivos modificados.
- Añade referencias `// AI-assisted: see docs/AI_LOG.md #N` cuando corresponda.

Documenta las limitaciones reales, especialmente la integración de Mercado Pago.

Si es viable, prepara también el PDF final de la bitácora, sin modificar el contenido histórico ni inventar entradas.

FASE 4 — COMMITS LOCALES

Te autorizo a crear commits locales, pero con condiciones estrictas.

Antes de comenzar:

1. Verifica la rama actual y `git status`.
2. Identifica los cambios preexistentes sin descartarlos.
3. Conserva todos los cambios válidos del proyecto.

Crea commits locales organizados por funcionalidad, sin mezclar cambios innecesarios.

Por ejemplo:

- Historial y filtros.
- Detalle y mapa.
- Paginación y pruebas.
- Documentación final.

Los nombres deben describir los cambios realmente realizados.

PROHIBIDO:

- `git push`.
- Crear o fusionar pull requests.
- `git reset --hard`.
- `git clean -fd`.
- Forzar cambios de rama descartando trabajo.
- Modificar archivos `.env`.
- Incluir secretos en Git.
- Desplegar Firebase.
- Activar Blaze o servicios de pago.
- Modificar datos reales de producción.
- Introducir cambios innecesarios en módulos que ya funcionan.

No crees commits si las verificaciones críticas fallan. En ese caso, conserva los cambios e informa el bloqueo.

FASE 5 — REPORTE FINAL

Al terminar, entrega un informe completo con:

1. Funcionalidades implementadas.
2. Archivos creados y modificados.
3. Pruebas ejecutadas y resultados exactos.
4. Requisitos de la guía cumplidos, parciales y pendientes.
5. Limitaciones técnicas.
6. Commits locales creados, con sus hashes.
7. Estado final de Git.
8. Índices o configuraciones pendientes de despliegue.
9. Comandos para ejecutar y probar la aplicación en mi iPhone.
10. Instrucciones para revisar y publicar los commits posteriormente.

Trabaja de forma autónoma, avanzando por fases, sin solicitar confirmación por cada archivo.

Si encuentras un problema de alto riesgo que no puedes resolver sin modificar producción, credenciales o componentes críticos, no realices esa acción. Documenta el bloqueo y continúa con las tareas independientes.

No sacrifiques la estabilidad de las funcionalidades existentes para completar un requisito secundario.

Cuando termines las fases autorizadas, detente.

```
El objetivo es que al regresar encuentre el proyecto implementado, probado, documentado y con los commits locales preparados para la entrega.
```

- **Resumen de la respuesta:**
  - **Estado inicial:** se revisaron `git status`, posibles secretos y exclusiones. No había secretos; `.env`, `functions/.env.local`, `.secret.local` y `node_modules` están excluidos. El trabajo pendiente de las entradas #37 a #39 se confirmó primero, agrupado por archivos, después de pasar la regresión completa.
  - **Causa del fallo de ordenación:** `getDriverTripHistory` pedía 50 viajes sin orden de Firestore y los ordenaba en la app. Una prueba con 60 viajes, cuyos IDs crecen al revés que sus fechas, confirmó que la versión anterior devolvía los 50 más antiguos y omitía los 10 más recientes.
  - **Consulta corregida (`tripHistoryService.js`):**
    - Viajes finalizados propios, ordenados por `createdAt` descendente, con el periodo y el método de pago filtrados en Firestore y páginas de 20 con cursor.
    - Cumple la regla existente (`limit <= 50`), así que no hubo que cambiar `firestore.rules`.
    - Requiere dos índices compuestos, declarados en `firestore.indexes.json` y no desplegados.
    - Si Firestore responde `failed-precondition`, la app lee todos los viajes finalizados con la consulta ya indexada y los filtra y ordena localmente.
    - `getDriverTripHistory` devuelve ahora los 50 más recientes; lo usan Ganancias y Pagos.
  - **Lógica (`tripHistoryLogic.js`):** periodos Hoy, Semana (lunes a domingo), Mes y Rango (días completos e incluidos), con fin exclusivo y hora local. Incluye filtros combinados, orden estable, páginas sin duplicados, datos de la lista y del detalle (sin inventar: «No disponible» y «Sin calificación») y coordenadas válidas para el mapa.
  - **Interfaz:**
    - **Historial (`TripHistoryScreen.js`):** muestra fecha y hora, origen y destino, categoría, tarifa, ganancia neta, estado y pago. Incluye filtros (`HistoryFilters.js`, con el `DateTimePicker` existente), «Cargar más», actualizar al deslizar, mensajes según el filtro y error con reintento.
    - **Detalle (`TripHistoryDetail.js`):** se abre en un `Modal` con `SafeAreaProvider` propio.
    - **Mapa (`TripRouteMap.js`):** origen, destino y ruta de referencia OSRM, identificada como tal.
    - `DriverMap` no se reutilizó directamente porque depende de la posición GPS y del viaje activo; solo se exportó `getMapProvider`.
  - **Documentación:** README (historial, índices pendientes, ejecución en iPhone), `docs/CUMPLIMIENTO.md` (reporte de cumplimiento por sección de la guía) y `docs/AI_LOG.pdf`, generado desde el Markdown con `marked` (ya presente en `node_modules`) y Chrome sin interfaz.
  - **Campo «Commit»:** se completó en las entradas #35 a #39 con los hashes reales. Las entradas #30 a #34 conservan «Pendiente», porque corresponden a commits anteriores del integrante.
- **Qué se usó / modificó / descartó:**
  - **Reutilizado:** `routeService.getDrivingRoute`, el proveedor de mapas de `DriverMap`, `getTripEarnings`, `getTripPaymentLabel`, `fetchAllPages` y el patrón de `DateTimePicker` del registro.
  - **Descartado:**
    - Modificar `firestore.rules`.
    - Desplegar los índices.
    - Cambiar la navegación: se usó un `Modal`.
    - Guardar la geometría de la ruta en los viajes: cambiaría el ciclo del viaje.
- **Archivos modificados:**

    - **Nuevos:** `src/utils/tripHistoryLogic.js`, `src/services/tripHistoryService.js`, `src/hooks/useTripHistory.js`, `src/components/history/HistoryFilters.js`, `src/components/history/TripHistoryDetail.js`, `src/components/history/TripRouteMap.js`, `tests/tripHistory.test.cjs`, `tests/tripHistory.integration.test.cjs` y `docs/CUMPLIMIENTO.md`.
    - **Modificados:** `src/screens/home/TripHistoryScreen.js`, `src/services/tripService.js`, `src/components/map/DriverMap.js` (solo `export`), `firestore.indexes.json`, `tests/activeTripService.test.cjs`, `package.json`, `README.md` y `docs/AI_LOG.md`.
    - **Generado:** `docs/AI_LOG.pdf`.
- **Commit:** `2cda7e4` (orden, paginación y pruebas), `ec76dcb` (detalle y mapa), `d285c5c` (pantalla y filtros) y el commit de documentación final, que contiene esta entrada.
- **Verificación:**
    - **Suites:**
        - `npm run test:unit`: 159/159.
        - `npm run test:integration`: 46/46, incluida la paginación completa sin duplicados y los filtros contra el emulador con reglas.
        - `npm run test:rules`: 156/156, sin avisos «maximum of 1000 expressions».
        - `npm run test:functions`: 13/13 y 12/12.
    - **Regresión del fallo original:** con 60 viajes, la versión anterior omitía el más reciente y la nueva devuelve exactamente los 50 más recientes.
    - **Expo:** `npx expo-doctor` 20/20. `npx expo export` genera los bundles de iOS (1014 módulos), Android (1018) e iOS en desarrollo con el MODO DEMOSTRACIÓN (1133).
    - **Repositorio:** `git diff --check` sin errores.
    - **No ejecutado:** la prueba visual en un dispositivo físico de la nueva pantalla del historial y del detalle, y la consulta ordenada contra el proyecto real (los índices no están desplegados y el emulador no los exige).
- **Limitaciones:**
    - La ruta del detalle es de referencia, no el recorrido real.
    - No hay calificación del pasajero.
    - En producción, mientras no se desplieguen los índices, se usa el respaldo, que lee todos los viajes finalizados.
    - La integración con la API real de Mercado Pago sigue sin probarse (entrada #37); la presentación usa el MODO DEMOSTRACIÓN, que no es Mercado Pago.
- **Aprendizaje:** Limitar una consulta antes de ordenarla en el servidor devuelve un subconjunto arbitrario, no los más recientes. El orden debe aplicarse en Firestore antes del límite. Un respaldo automático ante un índice ausente permite publicar el cambio sin romper la app mientras se despliega el índice.

Consideraciones generales

- Las implementaciones fueron revisadas y ejecutadas dentro del proyecto. Las funcionalidades que todavía no han sido integradas o verificadas se registran como pendientes, sin presentarlas como terminadas.
- Las herramientas se identifican como: **ChatGPT (OpenAI)**, **Claude (Anthropic)** para el uso conversacional de las entradas #10, #11 y #13, **Gemini CLI (Google)** y **Claude Code (Anthropic)**. Las atribuciones de las entradas #1 a #22 se basan en la declaración de esta bitácora y en los comentarios del código; no existen exportaciones de las conversaciones en el repositorio que permitan verificarlas de forma independiente. Las

entradas #23, #24, #26, #27, #28, #29, #30, #31, #32, #33, #34, #35, #36, #37, #38, #39 y #40 se realizaron con Claude Code en el repositorio local.

- Cuando un prompt no se conserva literalmente, el campo se identifica como resumen.
- Los archivos generados por la plantilla de Expo (entrada #1) y por npm (`package-lock.json`) no constituyen código generado por IA.
- Los commits de documentación `e7f8588`, `8748a20`, `f5ef7a9`, `04788d3`, `74b621a`, `3669693`, `8e6ef79`, `eff416b` y `8cf42fd` solo actualizan esta bitácora o el README; no consta el uso de IA en ellos.